Classification

Classification Problem

류영표 강사
ryp1662@gmail.com



© 2025 Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.

류 영 표

AI & Applied Mathematics Researcher · Lecturer

ryp1662@gmail.com
github/youngpyoryu



학력

- 동국대학교 일반대학원 응용수학 석사 수료 (2016-2019)
- 동국대학교 수학과 졸업 (2010-2016)

심사 및 전문가 자문 활동

- 인공지능 학습용 데이터 구축 사업 품질검증 전문가 (2023)
- 2023 신뢰할 수 있는 인공지능 개발 안내서(안)_고도화 전문위원
- 2023 자율주행/의료/공공사회 분야 신뢰할 수 있는 인공지능 개발 안내서(안) 자문위원

연구 및 프로젝트

- 부산대학교 병원: 개인 맞춤형 당뇨병 예방관리 인공지능 시스템 외주 프로젝트 (2022)
- 연세대학교 병원: 영상의학과 FLASK 고도화 연구 (2023, 2024)
- 산업수학 스터디 그룹 (국가수리과학연구소): 피부암/유방암 분류, 유전자 정보 분석
- 폐플라스틱 이미지 객체 검출 경진대회 3위 (2021)
- 양자 AI, XAI 연구 진행 중

논문

- Quantum computing for the optimization of CT image reconstruction, IEEE ICTC 2022

강의 및 교육 활동

대학 강의

- 고려대학교, 인천대학교, 동양미래대학교, 목포대학교
- 이젠 아카데미: 파이썬/ADSP 과정

기업 교육

- 현대자동차 연구원
- 한화, 하나금융, 한전 KDN, IBK 기업은행, 아가방엔컴퍼니, IM증권

공공/국비 과정

- LG 헬로비전 DX 데이터 스쿨 (4기)
- 보건·바이오 AI 활용 전문가 양성과정 (1-3기)
- 공공데이터 청년 인턴 멘토
- 충청 ICT 취창업 역량강화 프로그램 (SQLD)
- 서울아이티고등학교

아카데미 & 부트캠프

- 모두의연구소 Aiffel 퍼실리테이터 (1기)
- Upstage AI Boostcamp 멘토 (1-6기)
- 패스트캠퍼스/멀티캠퍼스 조교 및 멘토

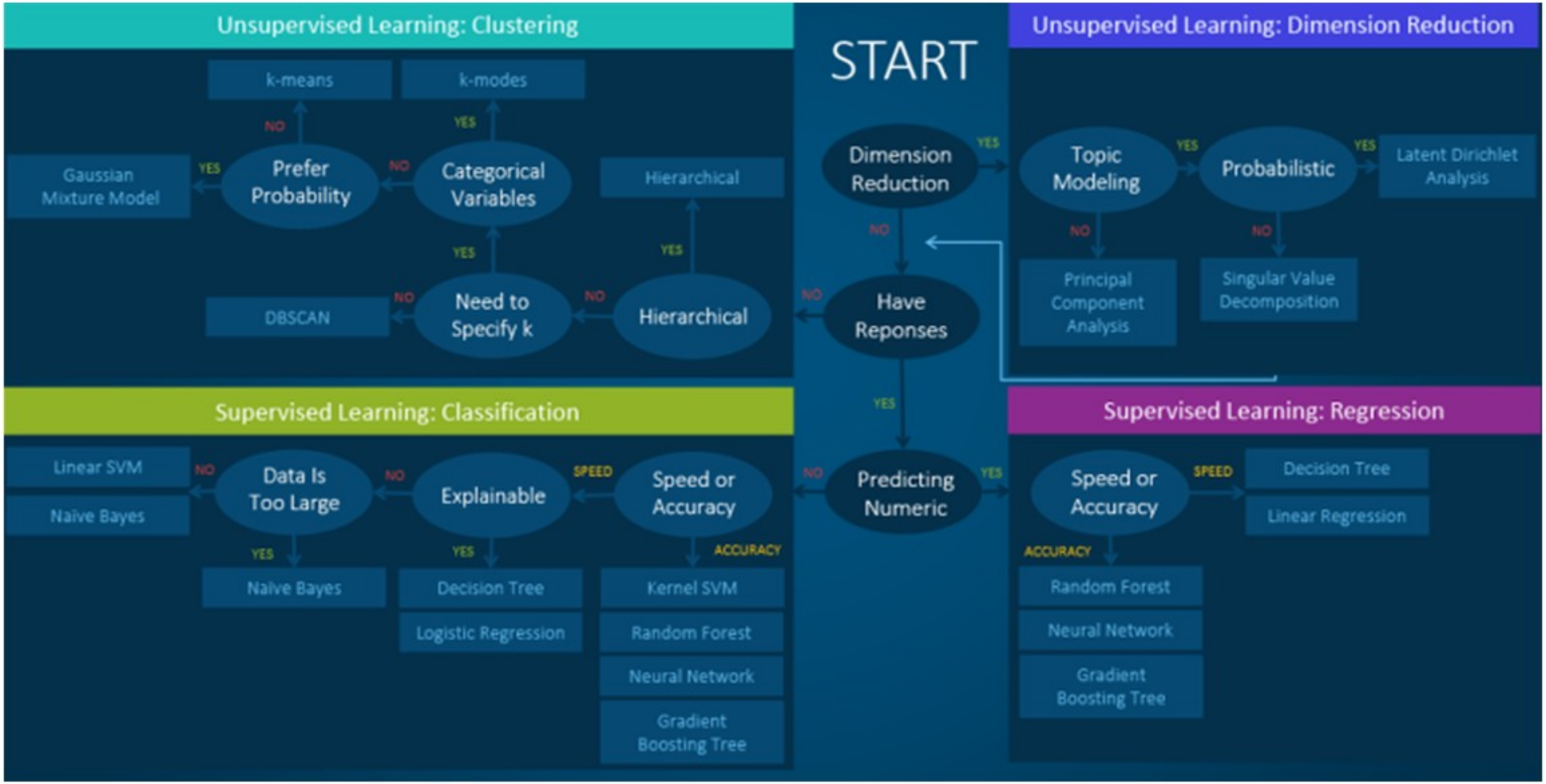
INDEX

1. 분류 성과지표
2. K-Nearest Neighbor (K-NN)
3. Naïve Bayes
4. Decision Tree
5. Random Forest
6. Boosting 계열
7. Ensemble learning



출처 : https://realpython.com/python-keras-text-classification/

Machine learning Algorithms Cheat Sheet



출처 : blogsaskorea.com/49

분류(Classification)란?

- Supervised Learning(지도학습)의 일종으로 기존에 존재하는 데이터의 Category관계를 파악하고, 새롭게 관측된 데이터의 Category를 스스로 판별하는 과정



- 학습데이터로 주어진 데이터의 피처와 레이블 값(결정 값, 클래스 값)을 머신러닝 알고리즘으로 학습해 모델을 생성하고, 이렇게 생성된 모델에 새로운 데이터 값이 주어졌을 때 미지의 레이블 값을 예측 하는 것

Classification 종류

✓ 베이즈(Bayes) 통계와 생성 모델에 기반한 **나이브 베이즈(Naïve Bayes)**

✓ 독립변수와 종속변수의 선형 관계성에 기반한 **로지스틱 회귀(Logistic Regression)**

✓ 데이터의 균일도에 따른 규칙 기반의 **결정트리(Decision Tree)**

✓ 개별 클래스 간의 최대 분류 마진을 효과적으로 찾아주는 **서포트 벡터 머신(Support Vector Machine)**

✓ 근접 거리를 기준으로 하는 **최소 근접(K-Nearest Neighbor)**

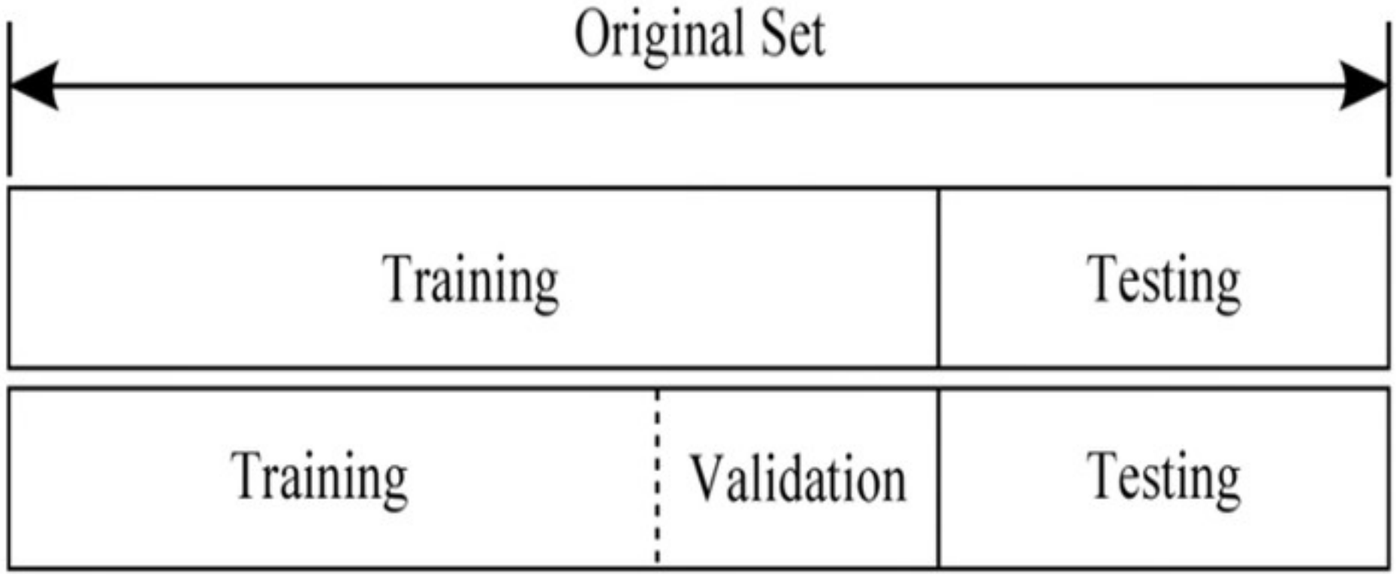
✓ 심층 연결 기반의 **신경망(Neural Network)**

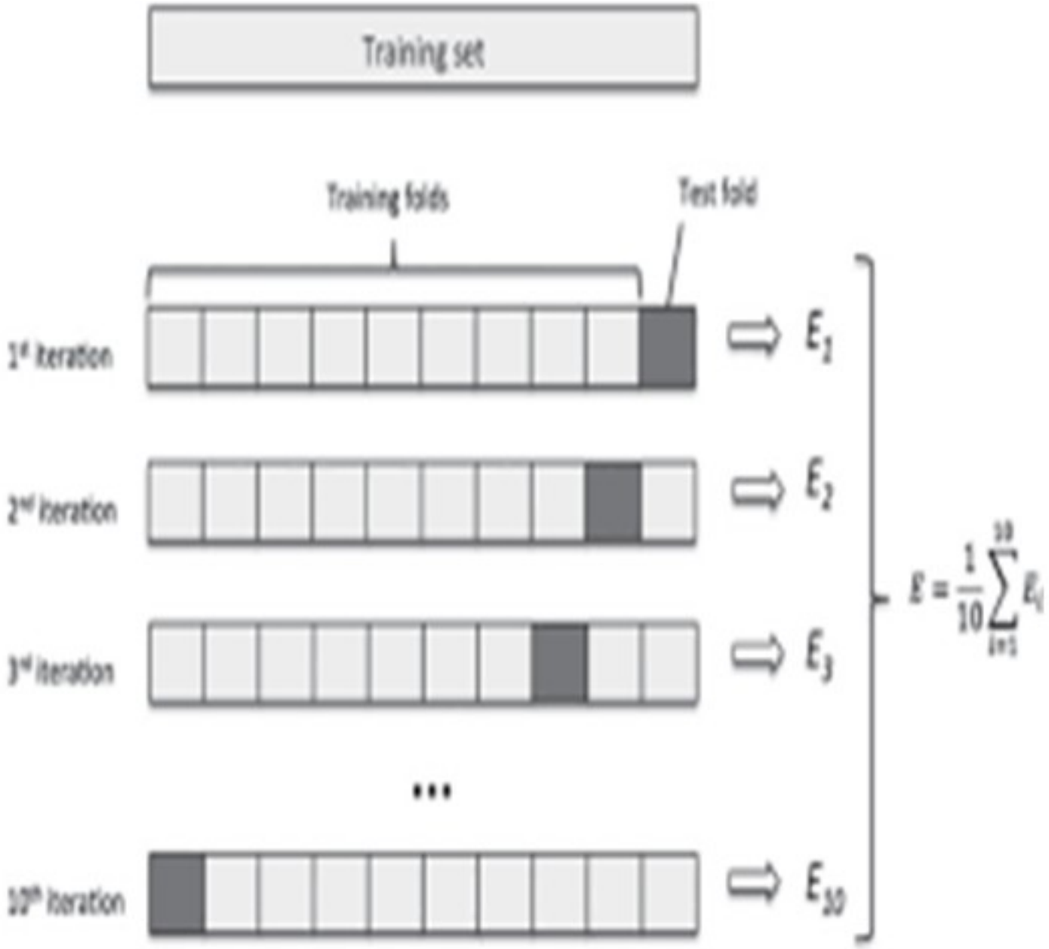
✓ 서로 다른(또는 같은) 머신러닝 알고리즘을 결합한 **앙상블(Ensemble)**

1. 분류 성과 지표

- Confusion Matrix
- Accuracy (정확도)
- Recall (재현도)
- Precision (정밀도)
- Specificity (특이도)
- F1 Score
- ROC와 AUC

데이터마이닝을 위한 데이터 분할





오분류표(Confusion Matrix)

		Actual Values	
		Positive	Negative
Predicted Values	Positive	TP	FP
	Negative	FN	TN

- **TP**(True Positive) : 실제 positive값을 positive로 올바르게 예측한 경우
- **TN**(True Negative) : 실제 negative값을 negative로 올바르게 예측한 경우
- **FP**(False Positive or Type I error(1종 오류)) : 실제 negative값을 positive로 잘못 예측한 경우
- **FN**(False Negative or Type II error(2종 오류)) : 실제 positive값이 negative로 잘못 예측한 경우

정확도(Accuracy)

		Actual Values	
		Positive	Negative
Predicted Values	Positive	TP	FP
	Negative	FN	TN

- Accuracy는 정확도로 전체 예측한 것 중에 올바른 예측을 얼마나 했는지를 지표로 구하는 것.

$$\frac{TP + TN}{TP + TN + FP + FN}$$

- Accuracy값이 높을수록 예측 정확도가 높아질 수 있음.
- Error Rate = 1−Accuracy
- 편향의 함정이 있음.

정밀도(Precision)

		Actual Values	
		Positive	Negative
Predicted Values	Positive	TP	FP
	Negative	FN	TN

- Precision은 정밀도로 Positive로 예측한 것 중 실제로 맞춘 비율.

$$\frac{TP}{TP + FP}$$

- Positive로 예측한 것 중 실제로 Positive가 얼마나 되는지를 보여주는 지표
- Positive predictive value

민감도(Sensitivity, recall)

		Actual Values	
		Positive	Negative
Predicted Values	Positive	TP	FP
	Negative	FN	TN

- True Positive Rate로 실제 Positive를 얼마나 잘 예측했는지를 나타내는 지표.

$$\frac{TP}{TP + FN}$$

- 실제 Positive 중 Positive로 예측한 것이 얼마나 되는지를 보여주는 지표.

특이도(Specificity)

		Actual Values	
		Positive	Negative
Predicted Values	Positive	TP	FP
	Negative	FN	TN

- Specificity는 특이도로 실제 Negative를 얼마나 잘 예측했는지를 나타내는 지표.

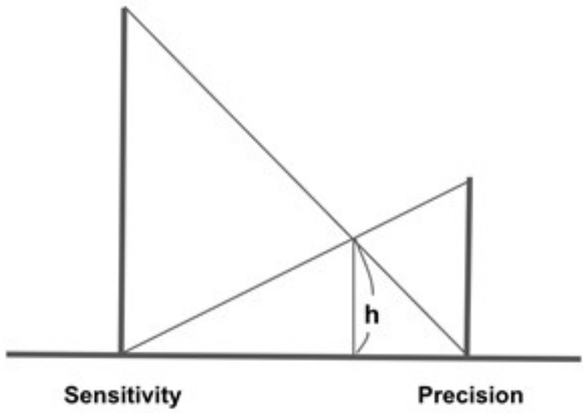
$$\frac{TN}{FP + TN}$$

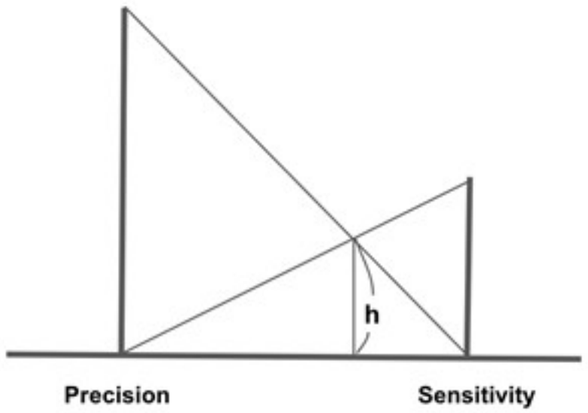
- 특이도를 통해서 False Positive Rate를 구할 수 있음.

  False Positive Rate = 1 − Specificity

F1-Score

- 보통 불균형 분류문제에서 평가척도 주로 사용됨.
- 데이터가 불균형한 상태에서 Accuracy로 성능을 평가하기엔 데이터 편향성이 너무 크게 나타나 올바르게 측정하기 힘들 때 사용.
- Sensitivity와 Precision을 이용한 조화평균

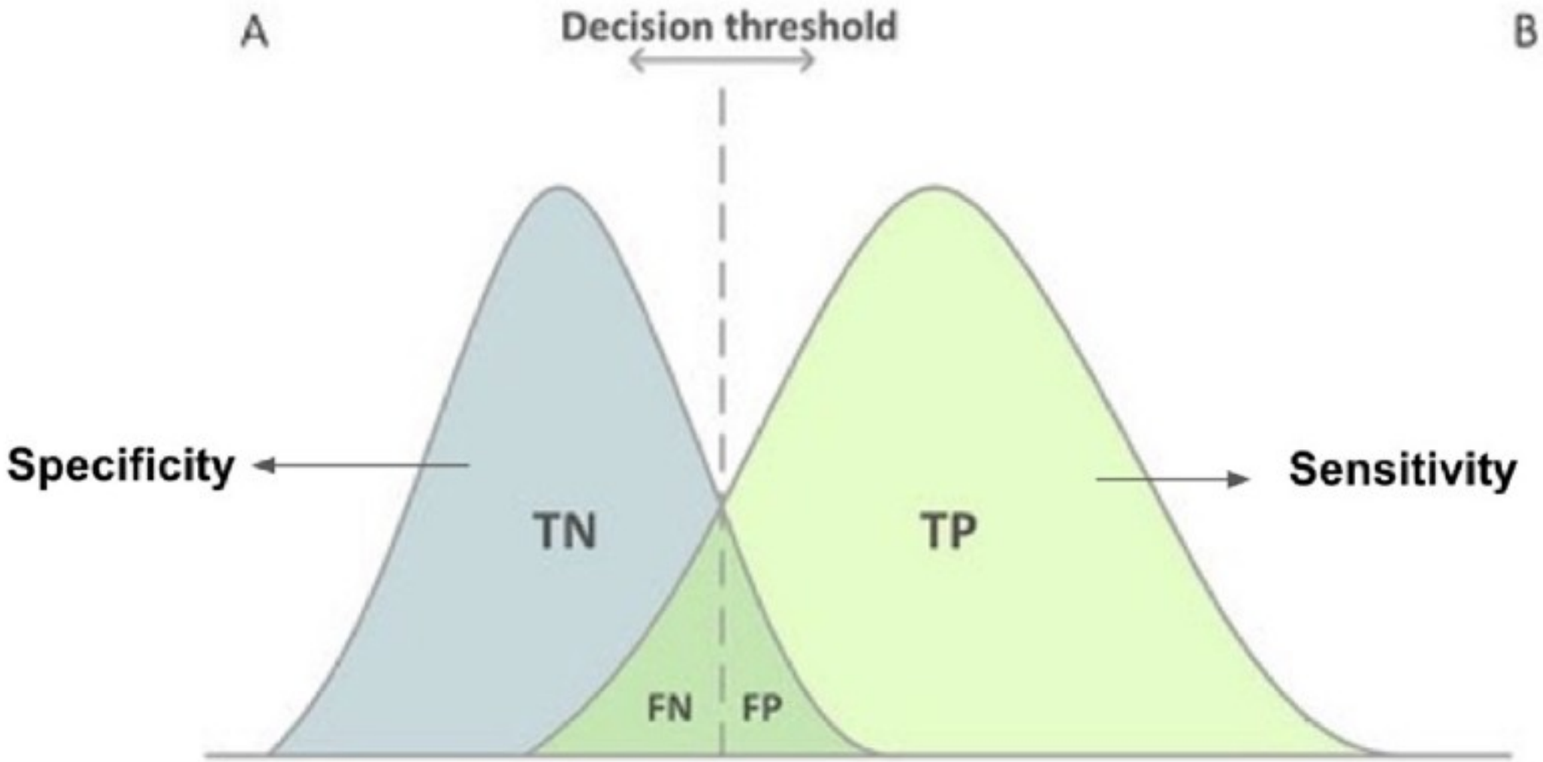




$$2 * \frac{\text{Sensitivity} * \text{Precision}}{\text{Sensitivity} + \text{Precision}}$$

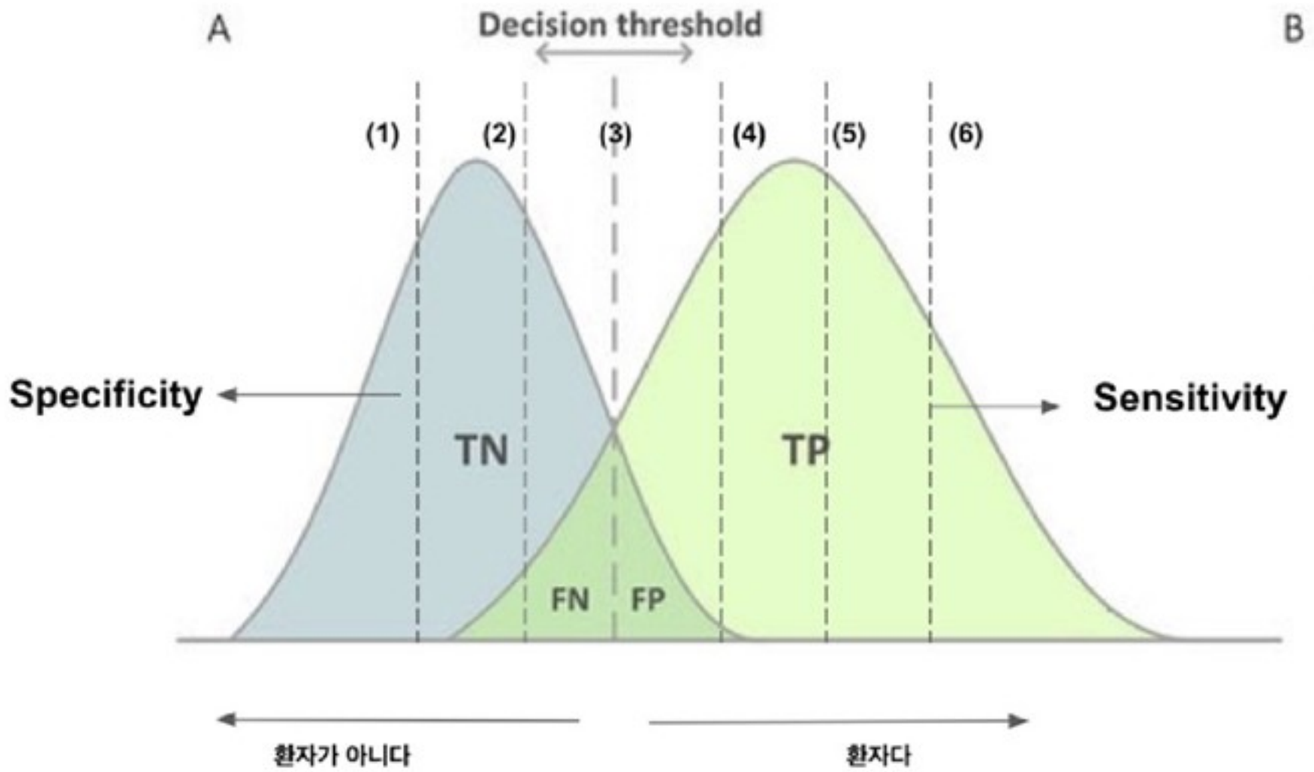
ROC(Receiver Operating Characteristic)

- FPR(False Positive Rate)과 TPR(True Positive Rate)을 각각 x, y축으로 놓은 그래프.
- TPR(True Positive Rate)-〉 실제 Positive를 얼마나 잘 예측했는지를 나타내는 지표(Sensitivity)
- FPR(False Positive Rate) -〉 1 - Specificity(실제 Negative를 얼마나 잘 예측했는지를 나타내는 지표)



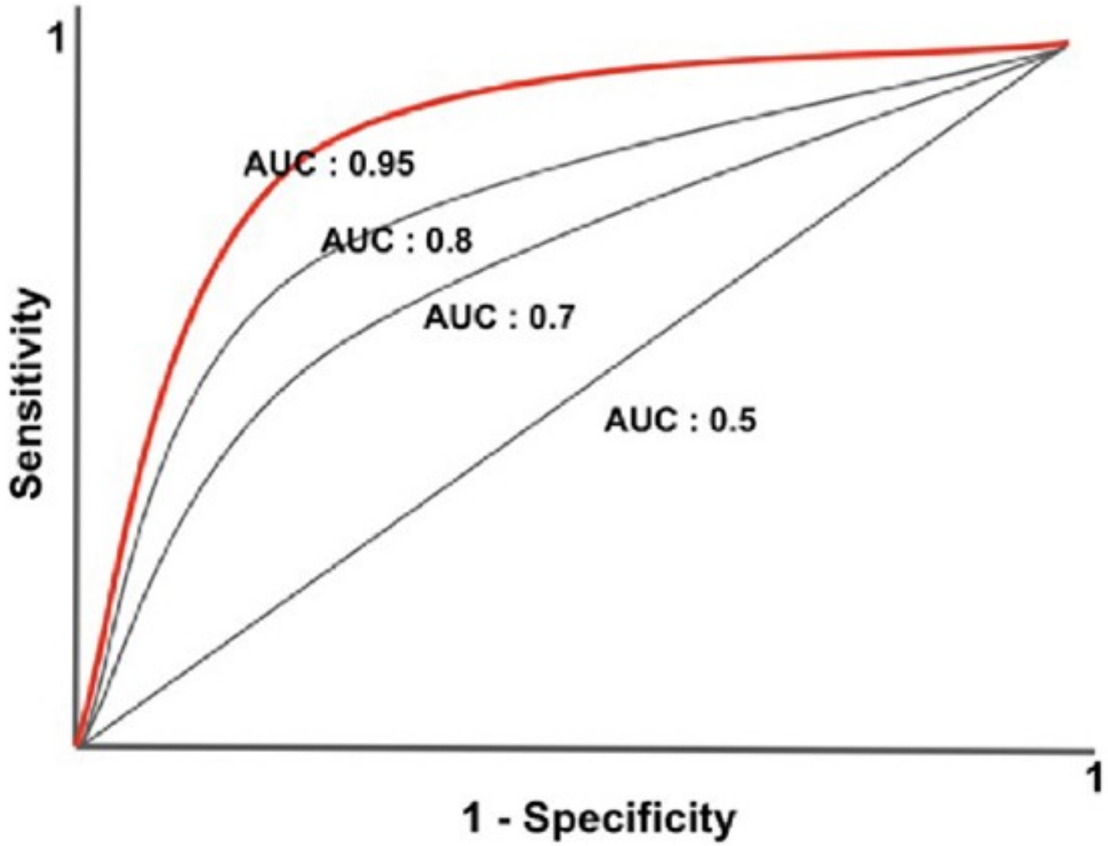
ROC(Receiver Operating Characteristic)

- FPR(False Positive Rate)과 TPR(True Positive Rate)을 각각 x, y축으로 놓은 그래프.
- TPR(True Positive Rate)-〉 실제 Positive를 얼마나 잘 예측했는지를 나타내는 지표(Sensitivity)
- FPR(False Positive Rate) -〉 1 - Specificity(실제 Negative를 얼마나 잘 예측했는지를 나타내는 지표)



AUC(Area Under the Curve)

- FPR(False Positive Rate)과 TPR(True Positive Rate)을 각각 x, y축으로 놓은 그래프.
- TPR(True Positive Rate)-〉 실제 Positive를 얼마나 잘 예측했는지를 나타내는 지표(Sensitivity)
- FPR(False Positive Rate) -〉 1 - Specificity(실제 Negative를 얼마나 잘 예측했는지를 나타내는 지표)



오분류표 예제

		Actual Values	
		Positive	Negative
Predicted Values	Positive	10	90
	Negative	5	895

- 정확도?
- 정밀도?
- 재현율(민감도)
- 특이도?
- FP Rate?
- F1 은?

오분류표 예제

		Actual Values	
		Positive	Negative
Predicted Values	Positive	30	40
	Negative	40	60

- 정확도?
- 정밀도?
- 재현율(민감도)
- 특이도?
- FP Rate?
- F1 은?

2. K-Nearest Neighbor (K-NN)

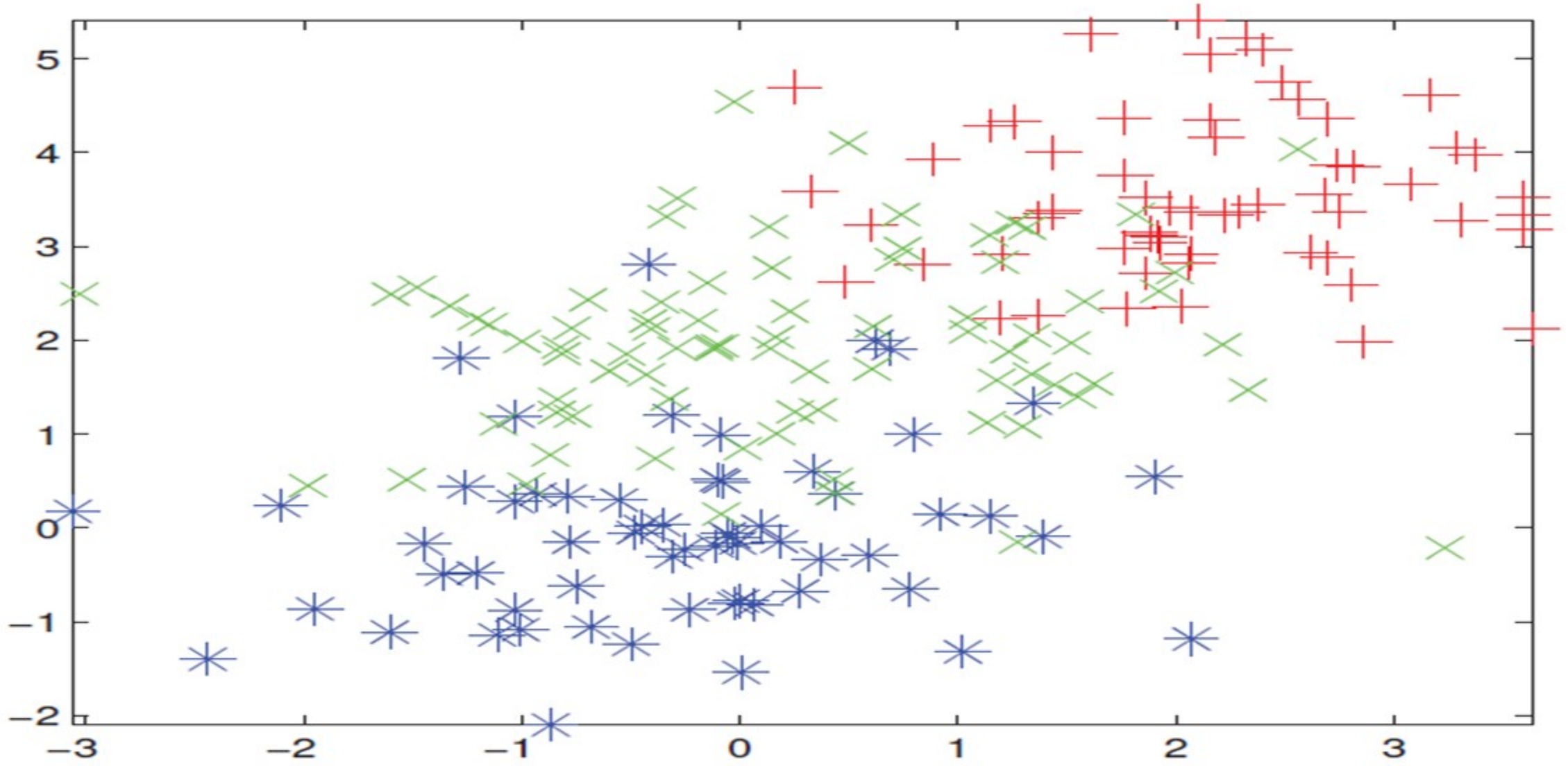
- K-Nearest Neighbor (K-NN)
- Metric Function의 종류
- Voronoi Tessellation

K-Nearest Neighbor (K-NN)



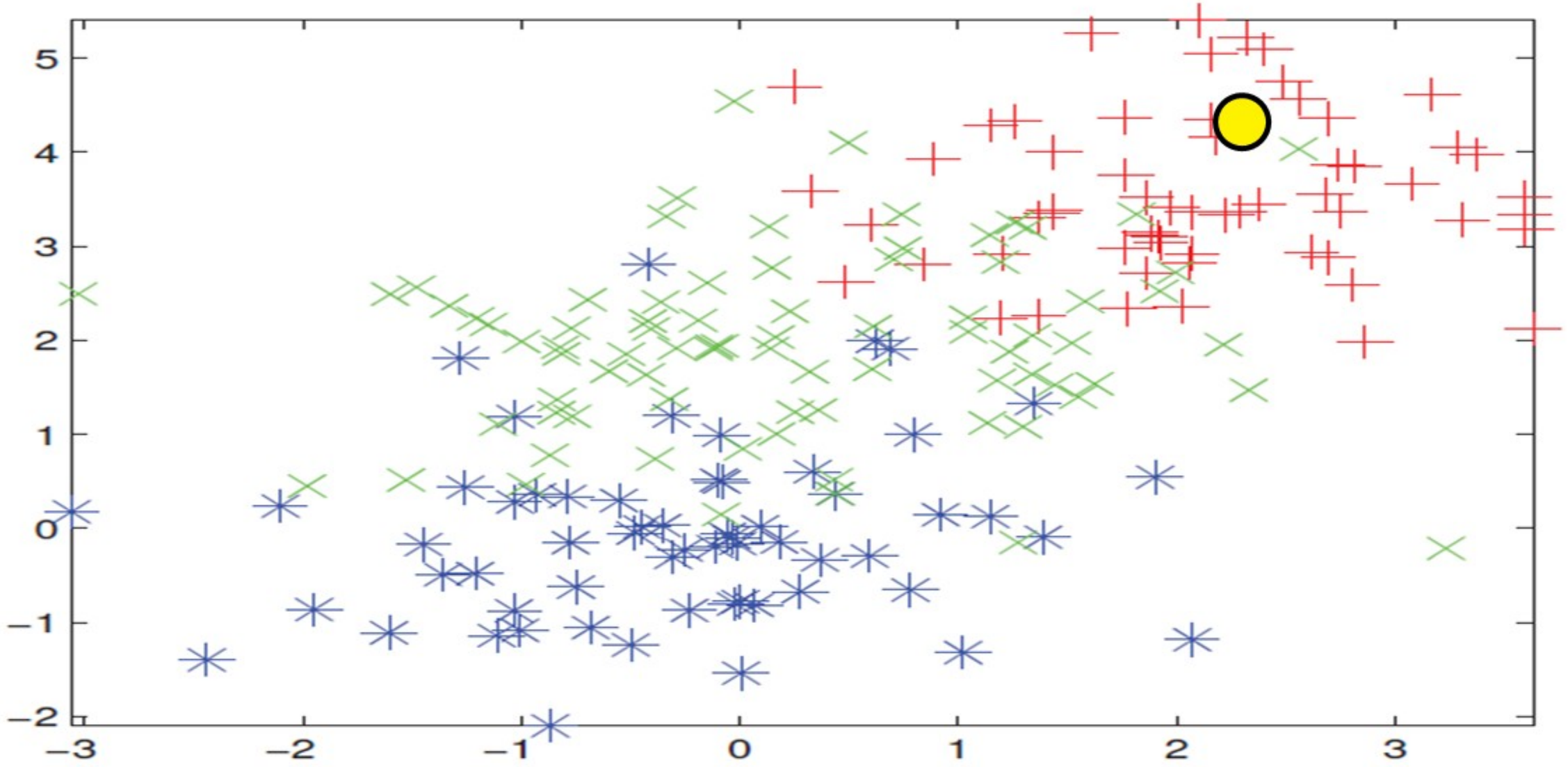
- 레이블이 있는 데이터를 사용하여 분류 작업을 하는 것
- 데이터로부터 거리가 가장 가까운 K개의 다른 데이터의 레이블을 참조하여 분류하는 알고리즘
- 주로 거리를 측정할 때 유클리디안 거리(Euclidean Distance) 계산법을 이용하여 거리를 측정함. 벡터의 크기가 커지면 계산이 복잡해짐.

K-Nearest Neighbor (K-NN)



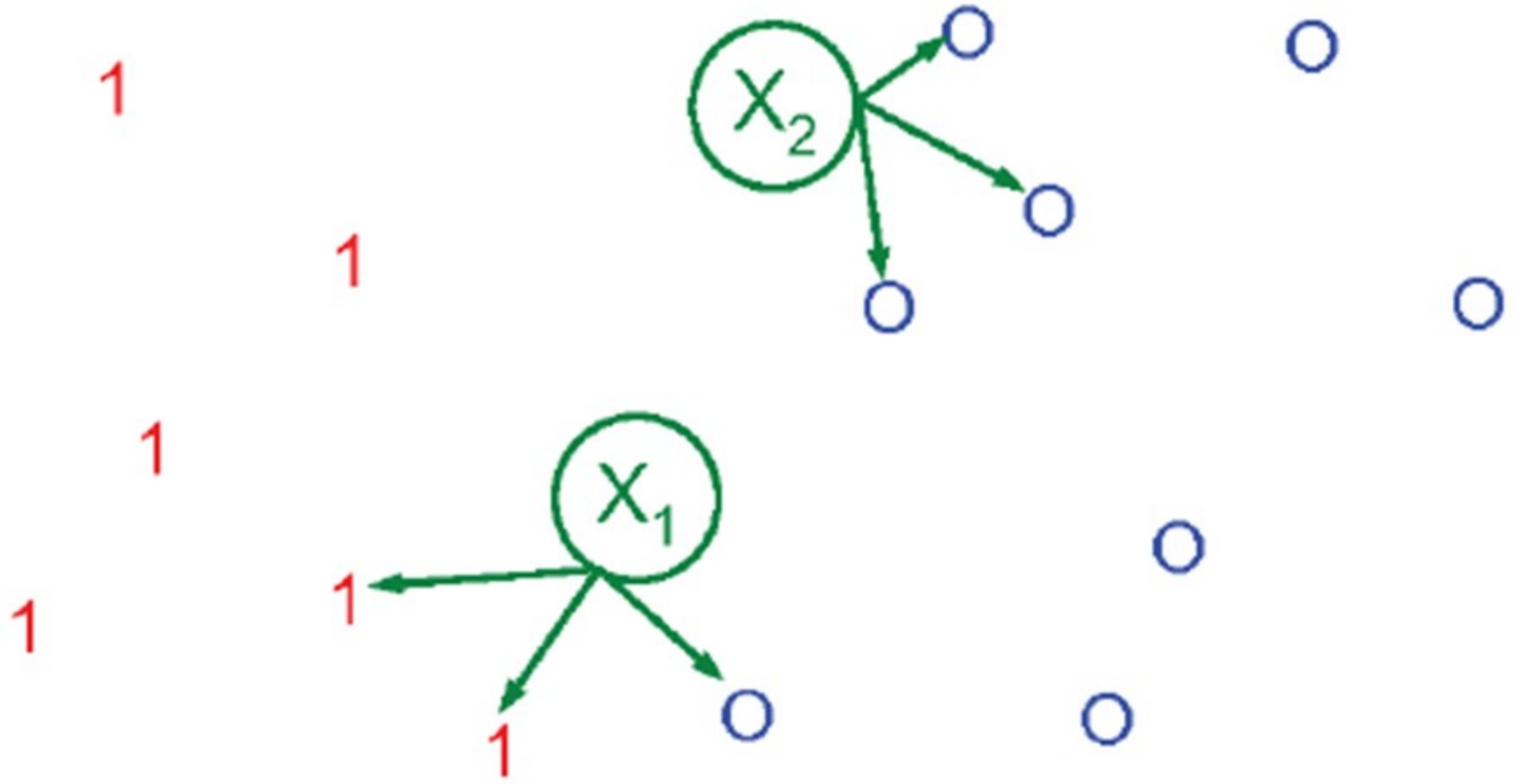
출처 : process-mining.tistory.com/132

K-Nearest Neighbor (K-NN)



출처 : process-mining.tistory.com/132

K-Nearest Neighbor (K-NN)



출처 : process-mining.tistory.com/132

K-Nearest Neighbor (K-NN)

주위에 K개의 점들이 label을 보고, 가장 많은 label로

해당 input의 label을 예측하는 것을 식으로 나타내면, 다음과 같다.

$$p(y = c|\mathbf{x}, \mathcal{D}, K) = \frac{1}{K} \sum_{i \in N_K(\mathbf{x}, \mathcal{D})} \mathbb{I}(y_i = c)$$

K-nearest neighbor의 식 표현. I는 indicator function,

N_K는 x에 대한 K-nearest point들을 의미한다.

Definition

Let Ω (Omega) be a set with x as an outcome of a random variable X. Define the set A be a subset of Ω. The indicator function of the set A as $1_A(\cdot)$ and define it as:

$$1_A(x) = \begin{cases} 1 & \text{if } x \in A \\ 0 & \text{if } x \notin A \end{cases}$$

If x is in the set A then the indcator function is 1 and if x is not in the set A then the indicator function is zero.

Metric Function의 종류

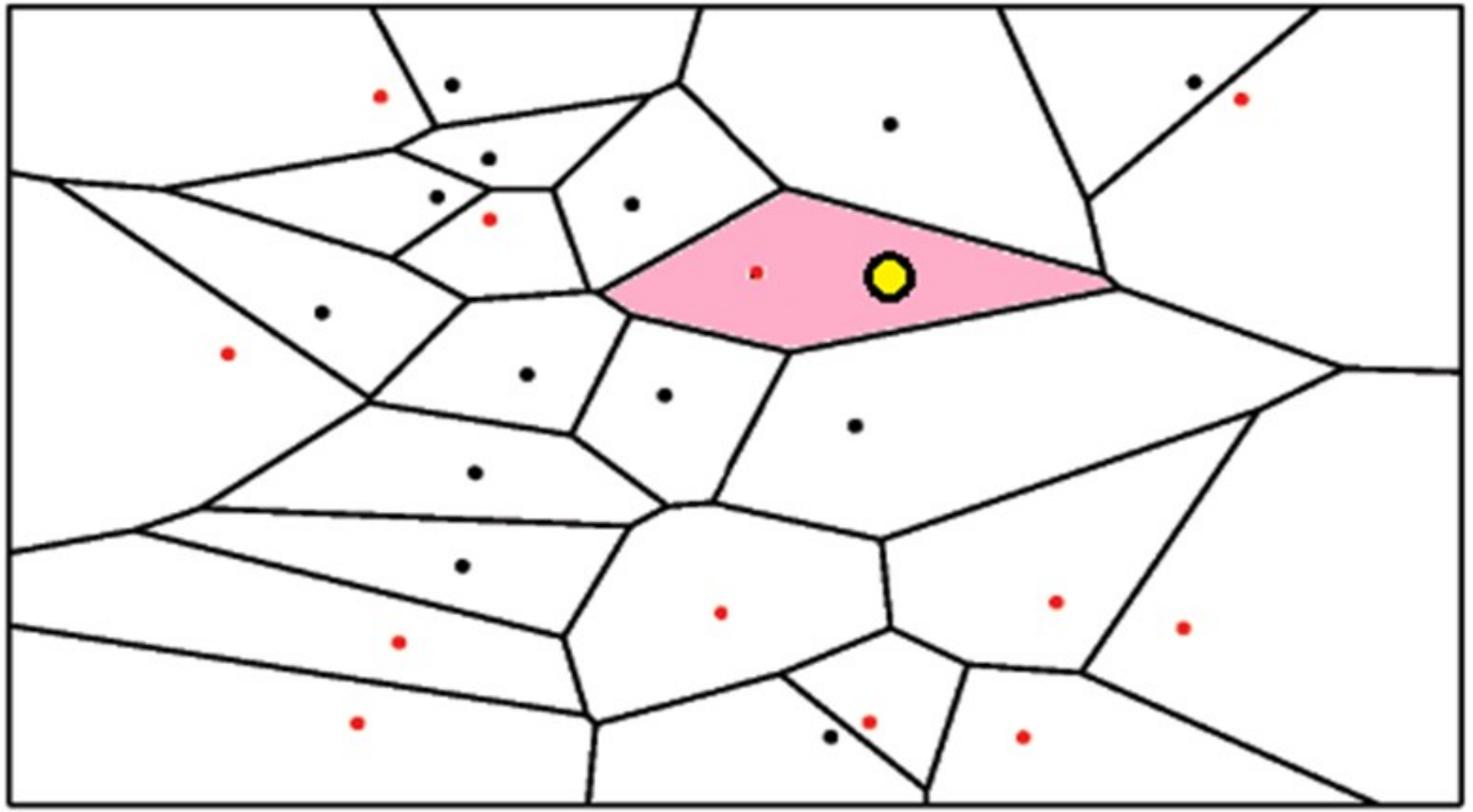
- 자주 쓰이는 거리함수

함수이름	함수식
2차 노름(유클리디안 거리)	$d_E(x,y) = \|x-y\|_2 = \sqrt{(x-y)^T(x-y)} = \sqrt{\sum_{i=1}^{n}(x_i-y_i)^2}$
1차 노름	$d_1(x,y) = \|x-y\|_1 = \sum_{i=1}^{n}\|x_i-y_i\|$
p차 노름	$d_p(x,y) = \sqrt[p]{\sum_{i=1}^{n}(x_i-y_i)^p}$
내적	$d_{IN}(x,y) = x \cdot y = \sum_{i=1}^{n} x_i y_i$
코사인거리	$d_{cos}(x,y) = 1 - \frac{x \cdot y}{\|x\| \|y\|}$
정규화된 유클리디안 거리	$d_{NE}(x,y) = \sqrt{\sum_{i=1}^{n}\frac{(x_i-y_i)^2}{\sigma_i^2}}$ (σ_i^2는 데이터의 분산)
마할노비스 거리	$d_M(x,y) = \sqrt{(x-y)^T\Sigma^{-1}(x-y)}$ (Σ는 데이터의 공분산행렬)

출 처 : iskim3068.tistory.com/70

K-Nearest Neighbor (K-NN)

- Voronoi Tessellation
  - K값이 1인 K-nearest neighbors를 우리는 Voronoi Tessellation이라고 부름.
    가장 가까운 곳에 있는 점의 label이 무엇인지에 따라 바로 그 점의 label을 결정.



출처 : process-mining.tistory.com/132

3. Naïve Bayes

- Naïve Bayes
- 날씨와 비의 관계표

Naïve Bayes

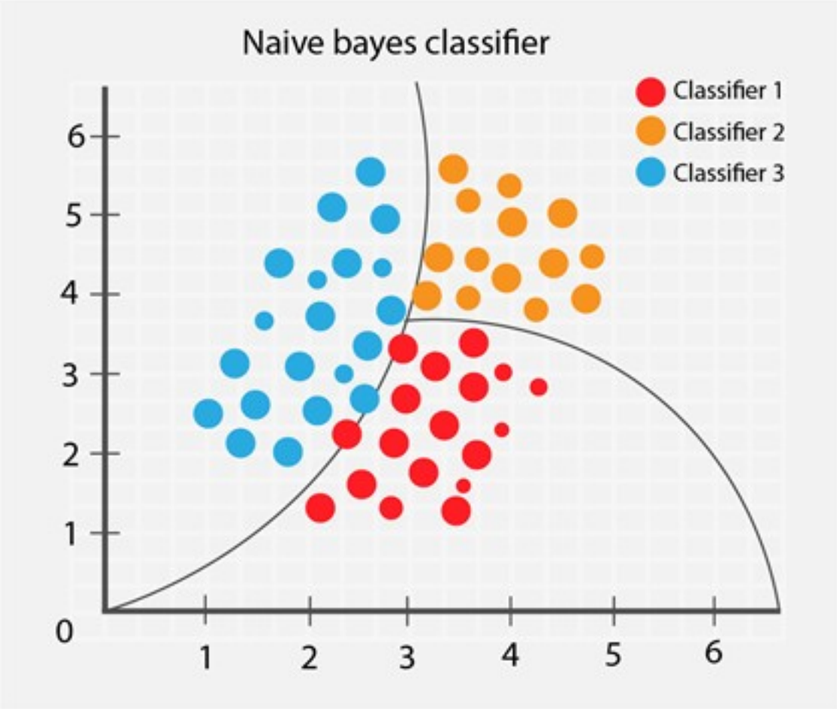
thatware.co

In machine learning, naive Bayes classifiers are a family of simple "probabilistic classifiers" based on applying Bayes' theorem with strong (naive) independence assumptions between the features.

$$P(A|B) = \frac{P(B|A)\ P(A)}{P(B)}$$

using Bayesian probability terminology, the above equation can be written as

$$\text{Posterior} = \frac{\text{prior x likelihood}}{\text{evidence}}$$



출처 : thatware.co/naive-bayes

Naïve Bayes

날씨와 비의 관계표			
	비 왔는가?		
	옴	안옴	합계
맑은 날	3	7	10
흐린 날	5	5	10
합계	8	12	20

- P(비|맑은날) 얼마일까?

Naïve Bayes

날씨와 비의 관계표			
	비 왔는가?		
	옴	안옴	합계
맑은 날	P(맑은날|비) = 3/8 ≈0.42	7	P(맑은날) =10/20 = 0.5
흐린 날	5	5	10
합계	P(비) =8/20 = 0.35	12	20

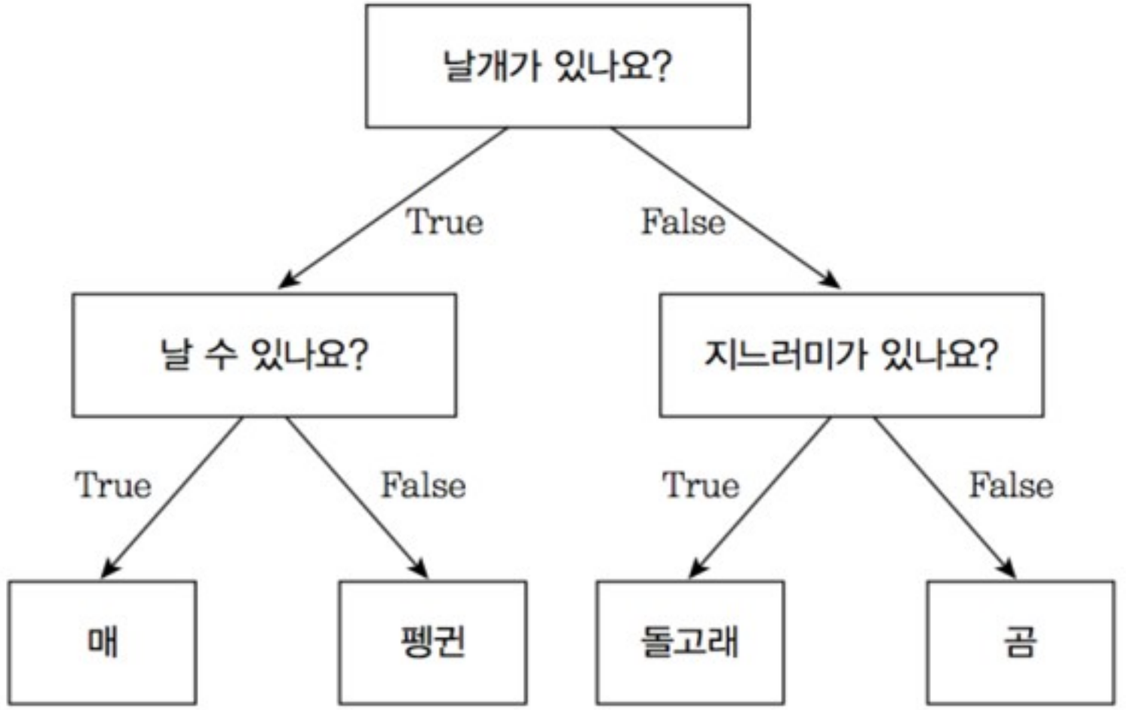
- P(비|맑은날) = P(맑은날|비) * (P(비) / P(맑은날)) = (3/7)* (0.375/0.4) ≈ 0.35

4. Decision Tree

- Decision Tree (결정 트리)
- Impurity
- Gini Index
- Information Gain
- 결정 트리의 종류
- 결정 트리의 모델 학습

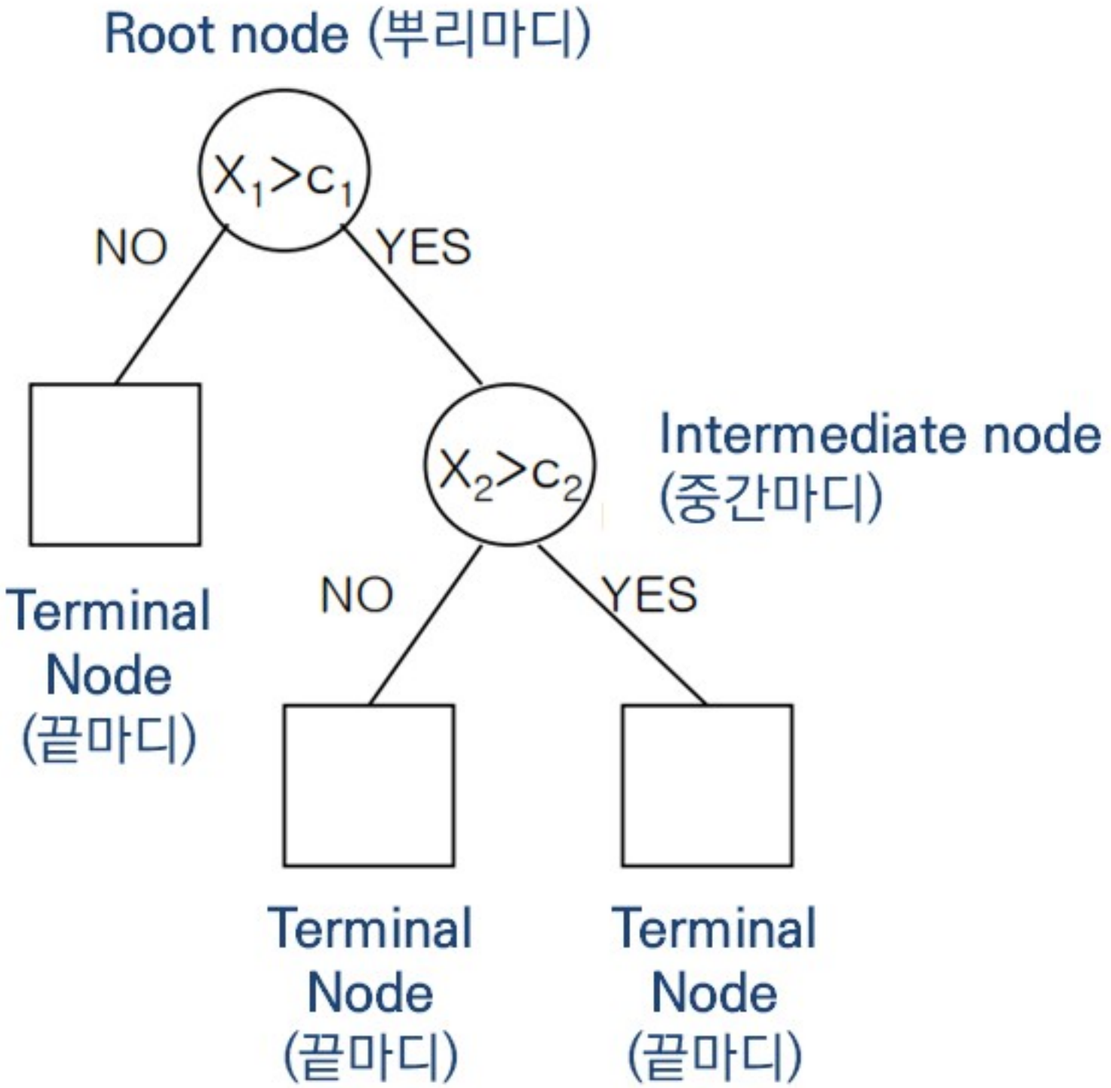
Decision Tree (결정 트리)

- 일반적으로 쉽게 표현하는 if/else로 스무고개 게임을 한다고 생각하면 된다.
- 결정 트리(Decision Tree, 의사결정 트리, 의사결정 나무라고도 함)는 분류(Classification)와 회귀(Regression) 모두 가능한 지도 학습 모델 중 하나입니다. 결정 트리는 스무 고개를 하듯 '예/아니오' 질문을 이어가며 학습합니다



출 처 : bkshin.tistory.com/entry/%EB%A8%B8%EC%8B%A0%EB%9F%AC%EB%8B%9D-4-%EA%B2%B0%EC%A0%95-%ED%8A%B8%EB%A6%ACDecision-Tree

Decision Tree (결정 트리)

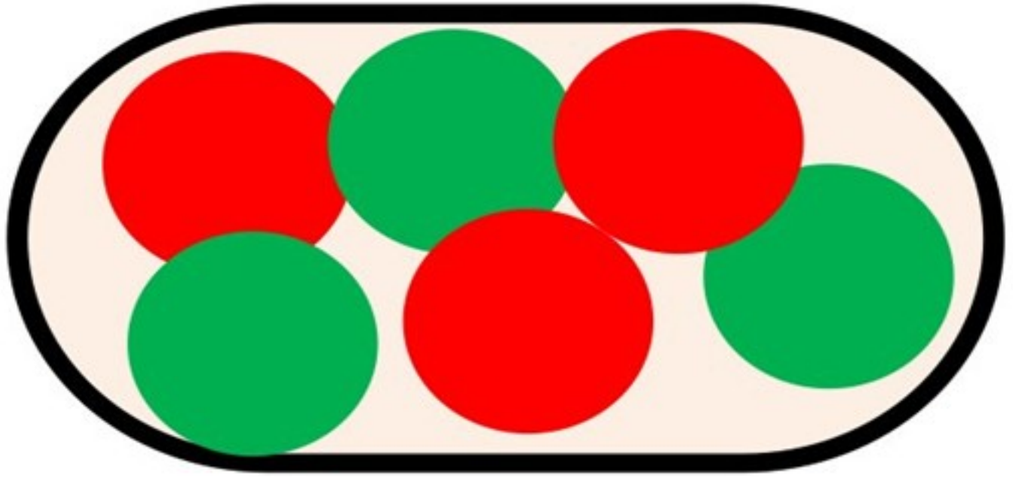


출 처 : ratsgo's blog

Decision Tree (결정 트리)

〈기본 아이디어〉

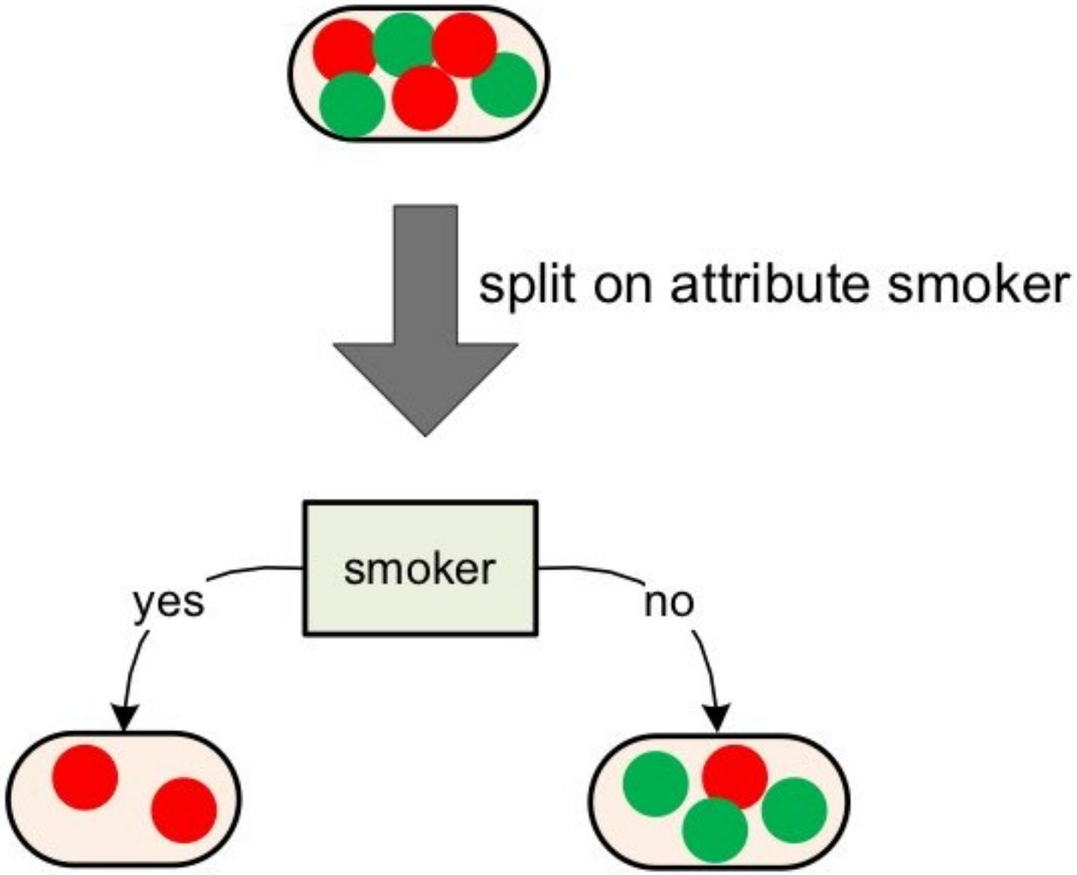
- Decision Tree의 기본 아이디어는 terminal node(끝마디)가 가장 섞이지 않은 상태로 완전히 분류 하는 것.
- 복잡성(Entropy)이 낮도록 만드는 것.

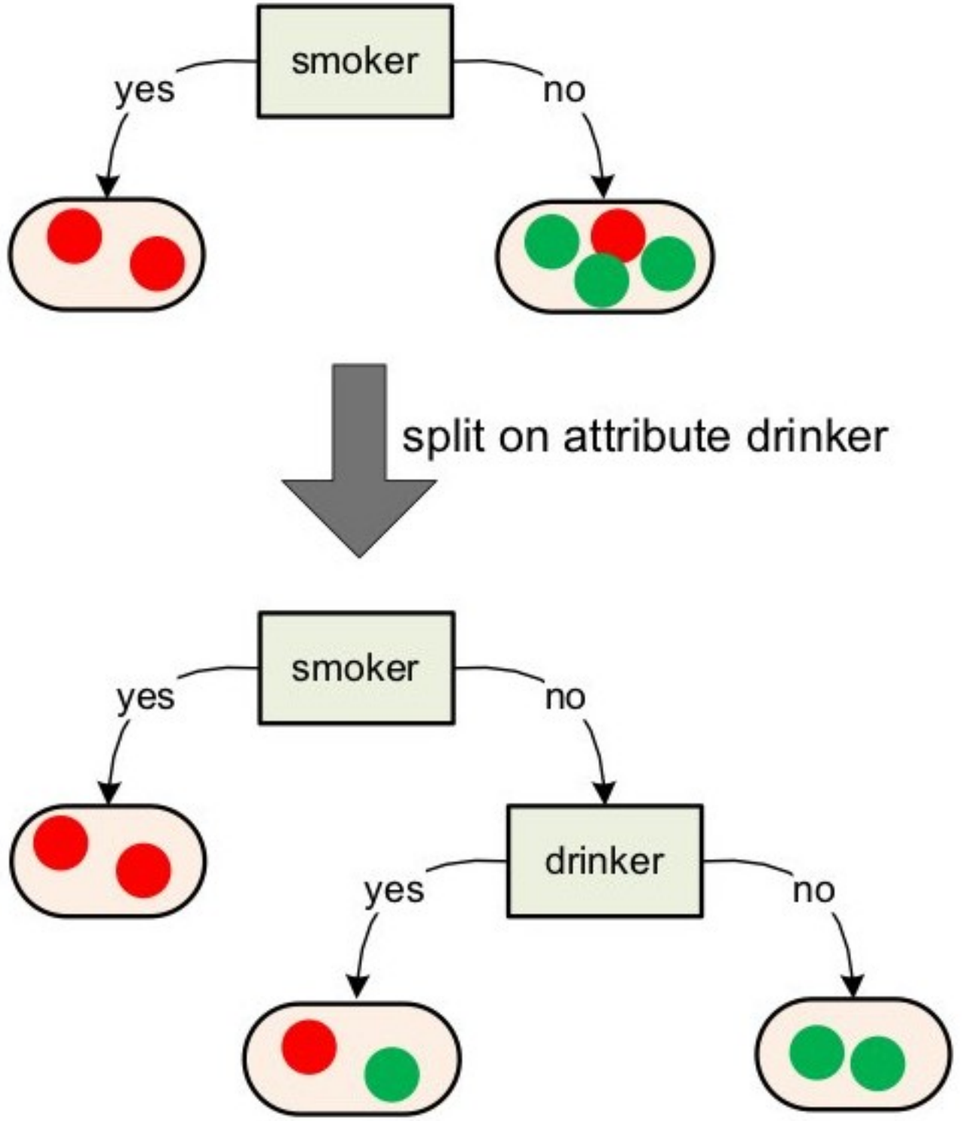


high entropy

출 처 : process-mining.tistory.com/42

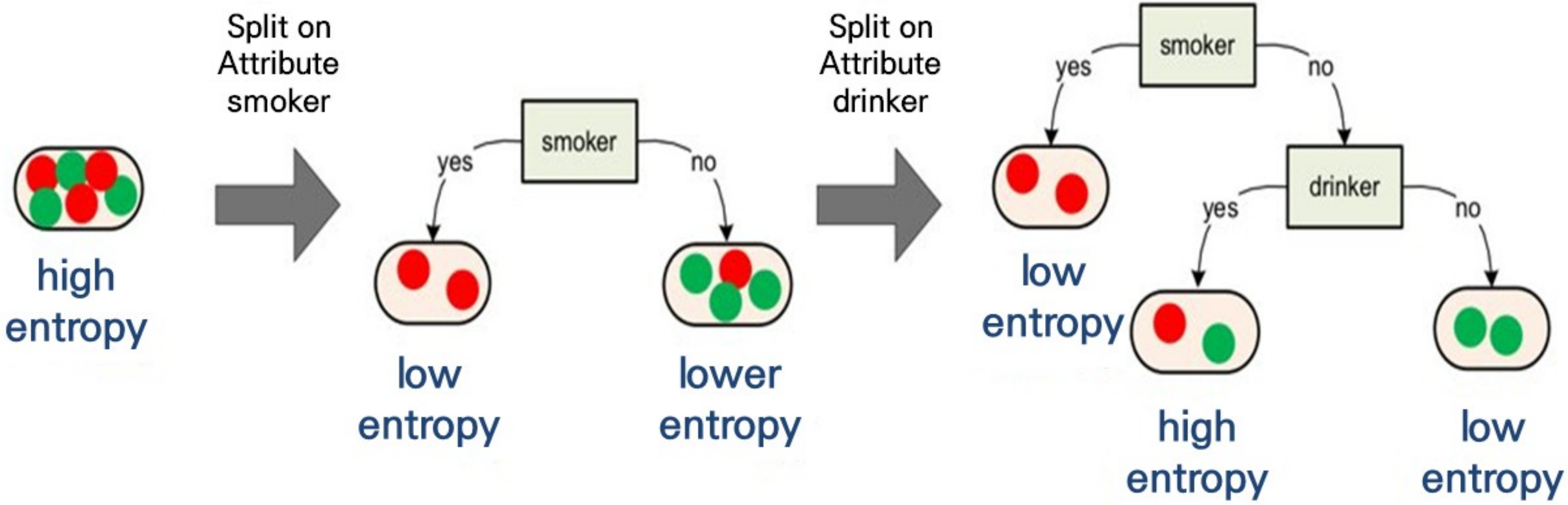
Decision Tree (결정 트리)





출처 : process-mining.tistory.com/42

Decision Tree (결정 트리)



Entropy가 점점 낮아지는 모습

출처 : process-mining.tistory.com/42

Loss function

퍼셉트론 및 선형회귀 분석에서 사용한 손실 함수

Square Loss :

$$E(W) = \frac{1}{2} \sum_{d \in D} (y_d - \widehat{y_d})^2$$

y_d : Target

$\widehat{y_d}$: Output

Mean Square Loss :

$$E(W) = \frac{1}{2} \frac{1}{|D|} \sum_{d \in D} (y_d - \widehat{y_d})^2$$

y_d : Target

$\widehat{y_d}$: Output

Information

어떤 사건을 수치로 나타낸 것.

확률을 이용한다.

확률사건 : Event

P(앞면)

P(앞면사건)

P(뒷면사건)

사건을 수치로 맵핑

$$\begin{cases} \text{앞면사건} \rightarrow 0 \\ \text{뒷면사건} \rightarrow 1 \end{cases} \qquad \mathrm{P}(\mathrm{X}=0) \qquad \mathrm{P}(\mathrm{X}=1)$$

확률 변수 : Random Variable

$$\mathrm{P}(\mathrm{X}=\mathrm{x}) \;=\; P_x(x) = P(x)$$ **다 같은 표현**

Information

지금부터 사건을 수치화 해보자.

사건이 드물게 발생할 수록 정보가 커야 한다. $\frac{1}{P(x)}$

두 사건에 대한 정보는 곱해야 하나 더해야 하나? $\frac{1}{P(x)}$ $\frac{1}{P(y)}$

Information

$$\frac{1}{P(x)} \times \frac{1}{P(y)}$$

두 사건에 대한 수치를 더할 수 있도록 다시 생각해보자.

Information

지금부터 사건을 수치화 해보자. $\frac{1}{P(x)}$ 사건이 드물게 발생할 수록 정보가 커야한다.

두 사건에 대한 정보는
곱해야 하나 더해야 하나? $\frac{1}{\mathrm{P(x)}} \times \frac{1}{\mathrm{P(y)}}$ 정보로 생각하면 더하는 것이 직관적이다.

$$log\frac{1}{\mathrm{P(x)}} + log\frac{1}{\mathrm{P(y)}}$$

로그는 고맙게도 곱셈을 덧셈으로 변경해준다.

Information

$$log\frac{1}{P(x)}$$

$$logP(x)^{-1}$$

$$-logP(x)$$

Entropy

정보량의 기댓값

확률 변수 X의 기댓값 계산 공식 $E[X] = \sum_{x} xP(x)$ $E[X] = \int x\,P(x)\,dx$

확률변수 X의 분포함수

확률질량함수 (Probability mass function)

확률밀도함수 (Probability density function)

Entropy

정보량의 기댓값

확률 변수 X의 기댓값 계산 공식

$$E[X] = \sum_x xP(x)$$

확률변수 X의 분포함수가 중요

$$E(aX + b) = \sum_x (ax + b)\, P(x)$$

$$E[f(X)] = \sum_x f(x)\, P(x)$$

일반화 시켜보면 ?

Entropy

정보량의 기댓값

$$E[X] = \sum_{x} xP(x)$$

확률변수 X의 분포함수가 중요

$$E[-logP(X)\ ] = \sum_{x} -logP(x)\quad P(x)$$

Entropy

$$\sum_{x} -logP(x)\quad P(x)$$

일반화 시켜보면 ?

Impurity

복잡성의 의미, 즉 해당 노드 안에서 섞여 있는 정도가 높을 수록 복잡성이 높고, 덜 섞여 있을수록 복잡성이 낮다.

먼저, 엔트로피(Entropy)에 대해 알아보면, 엔트로피(Entropy)는 불순도(impurity)를 수치적으로 나타냄

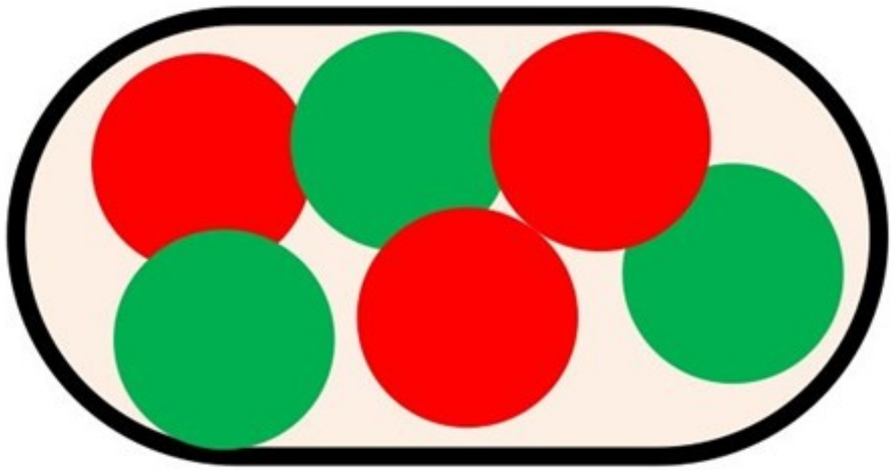
$$E = -\sum_{i=1}^{k} p_i \log_2(p_i)$$

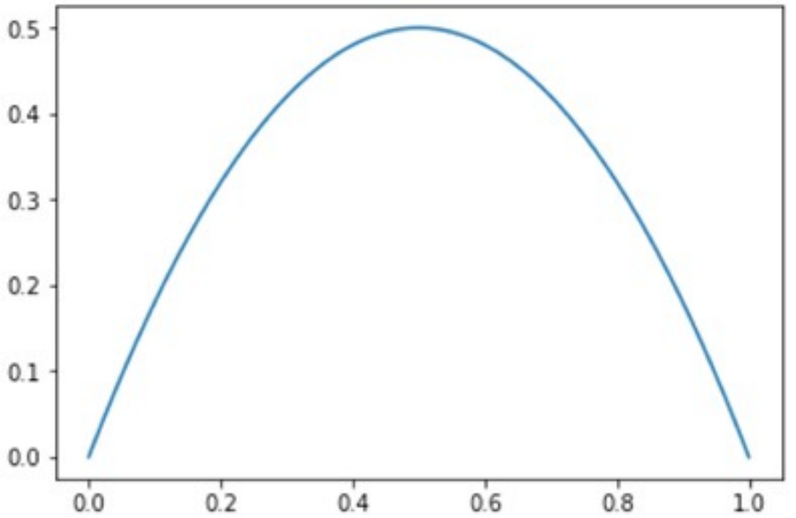
P_i = 한 영역 안에 존재하는 데이터 가운데 범주 i에 속하는 데이터의 비율

출 처 : process-mining.tistory.com/42

Impurity

Entropy





$$\text{entropy} = -(\frac{3}{6}\log_2\frac{3}{6}+\frac{3}{6}\log_2\frac{3}{6}) = -(\frac{1}{2}\log_2\frac{1}{2}+\frac{1}{2}\log_2\frac{1}{2}) = -\left(-\frac{1}{2}+\left(-\frac{1}{2}\right)\right) = 1$$

출처 : process-mining.tistory.com/42

Impurity

- **정보 이득(Information Gain)**

  정보 이득은 엔트로피라는 개념을 기반으로 함. 엔트로피는 주어진 데이터 집합의 혼잡도를 의미하는데,
  서로 다른 값이 섞여 있으면 엔트로피가 높고, 같은 값이 섞여 있으면 엔트로피가 낮음.
  정보 이득 지수는 1에서 엔트로피 지수를 뺀 값. 즉, 1-엔트로피 지수.
  결정 트리는 이 정보 이득 지수로 분할 기준을 정함. 즉, 정보 이득이 높은 속성을 기준으로 분할

- **지니 계수(Gini Index)**

  원래 경제학에서 불평등 지수를 나타낼 때 사용하는 계수.
  경제학자인 코라도 지니(Corrado Gini)의 이름에서 딴 계수로서 0이 가장 평등하고 1로 갈수록 불평등함.
  머신 러닝에 적용될 때는 지니 계수가 낮을수록 데이터 균일도가 높은 것으로 해석 되어,
  계수가 낮은 속성을 기준으로 분할.

Impurity

Impurity Criterion

Gini Index

$$I_G = 1 - \sum_{j=1}^{c} p_j^2$$

p_j: proportion of the samples that belongs to class c for a particular node

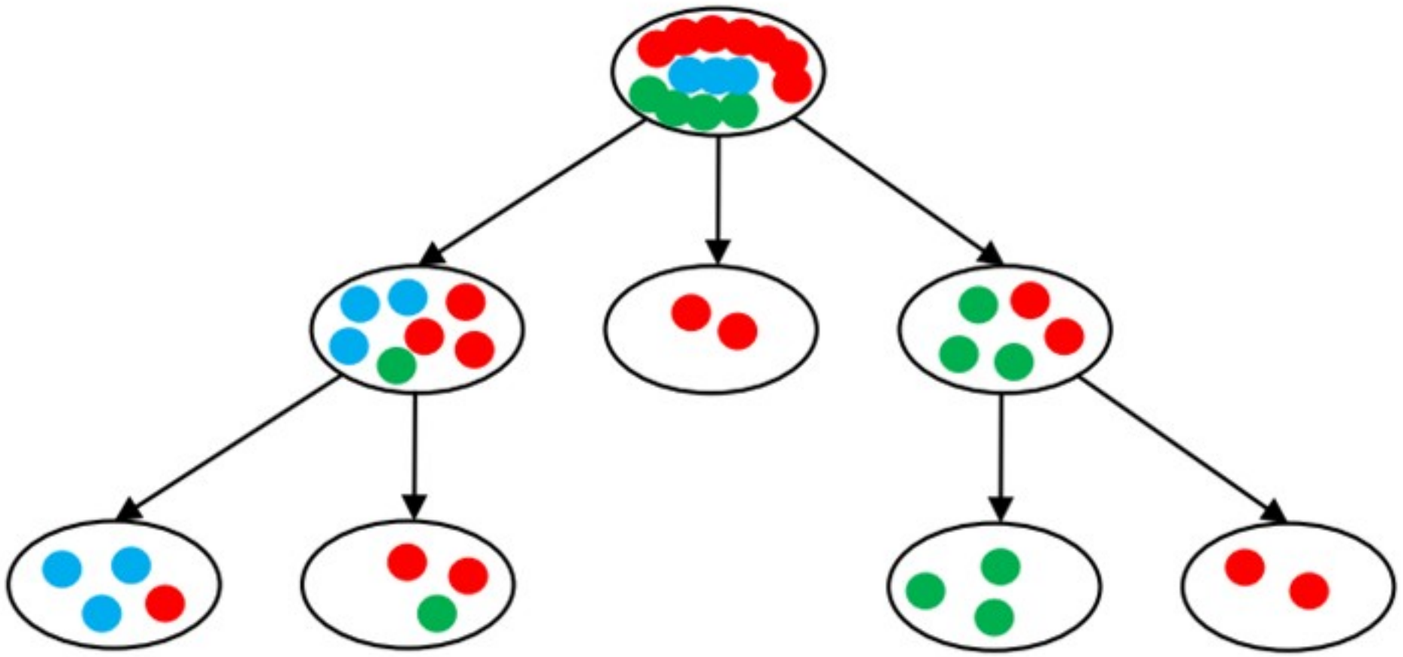
Entropy

$$I_H = -\sum_{j=1}^{c} p_j log_2(p_j)$$

p_j: proportion of the samples that belongs to class c for a particular node.

*This is the the definition of entropy for all non-empty classes ($p \neq 0$). The entropy is 0 if all samples at a node belong to the same class.

Impurity



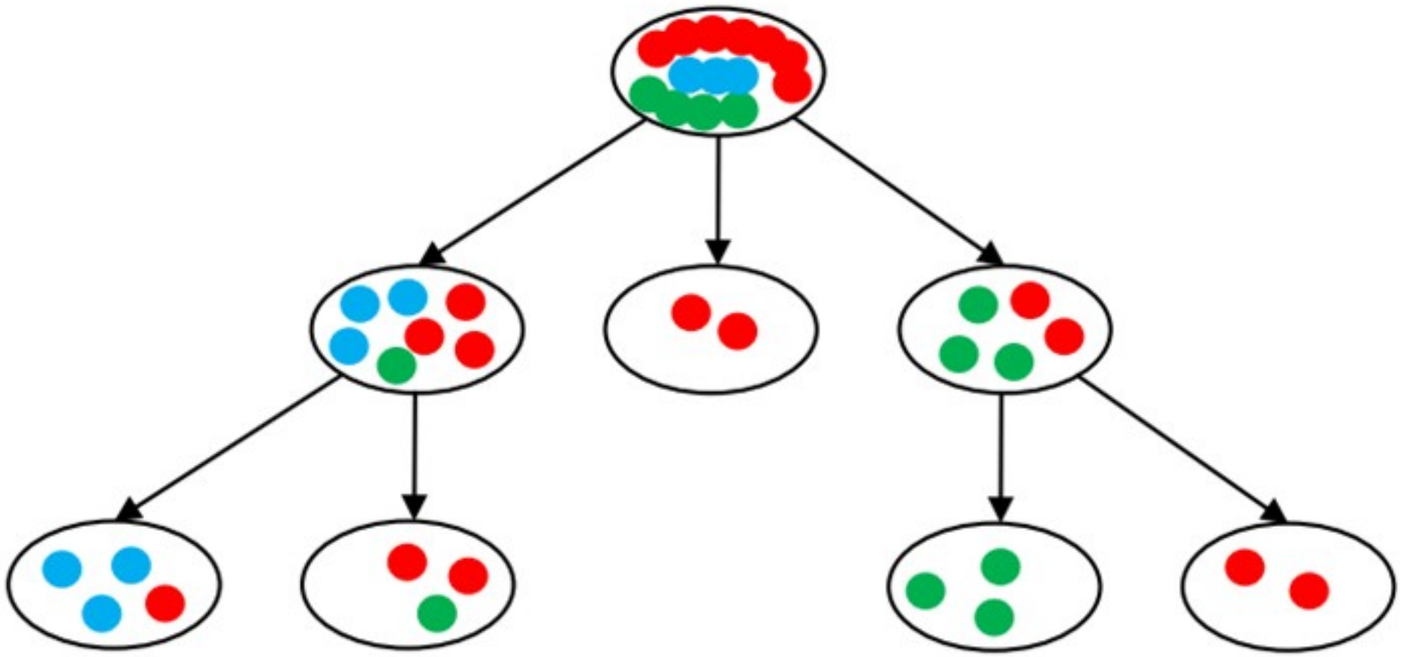
맨 위의 노드



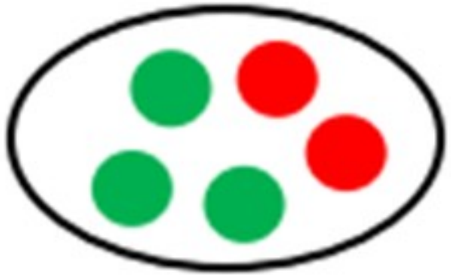
$$-\left(\frac{7}{14}\log_2\frac{7}{14}+\frac{3}{14}\log_2\frac{3}{14}+\frac{4}{14}\log_2\frac{4}{14}\right)=1.4926$$

출처 : process-mining.tistory.com/106

Impurity

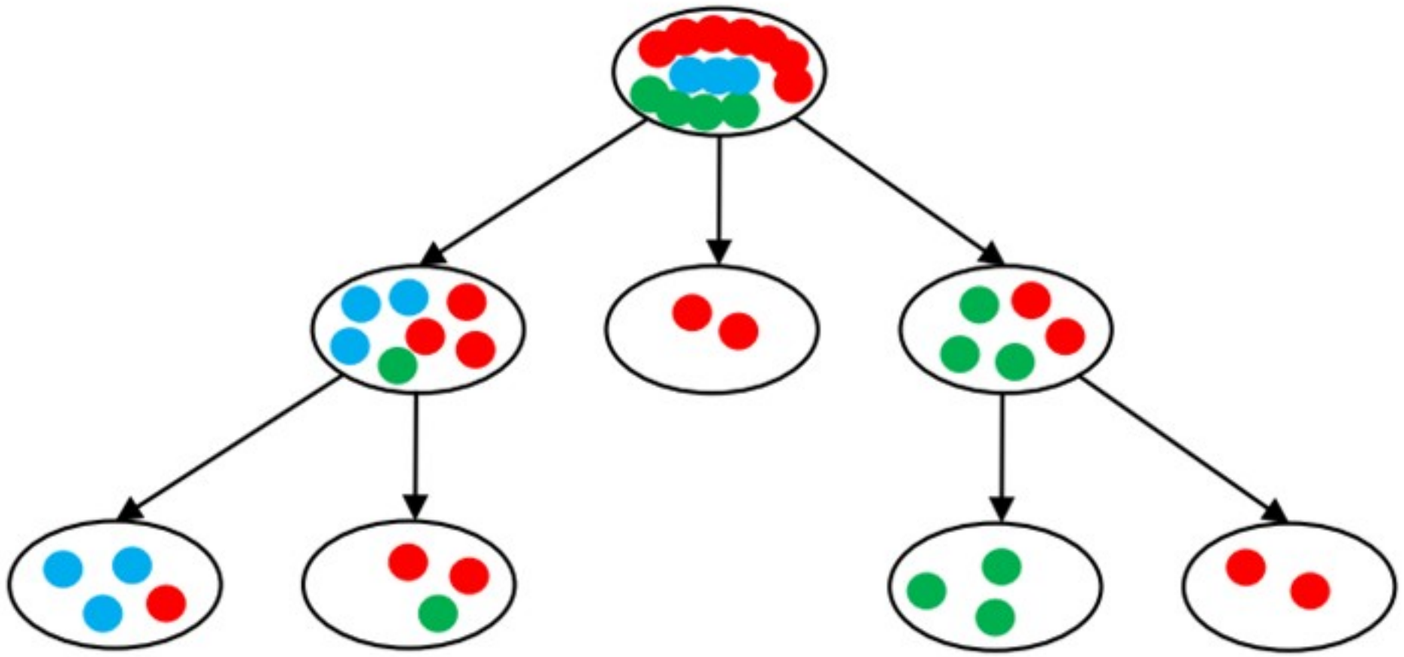


오른쪽, 위에서 두번째 노드



$$-\left(\frac{2}{5}\log_2\frac{2}{5}+\frac{3}{5}\log_2\frac{3}{5}\right)=0.910$$

Gini Index



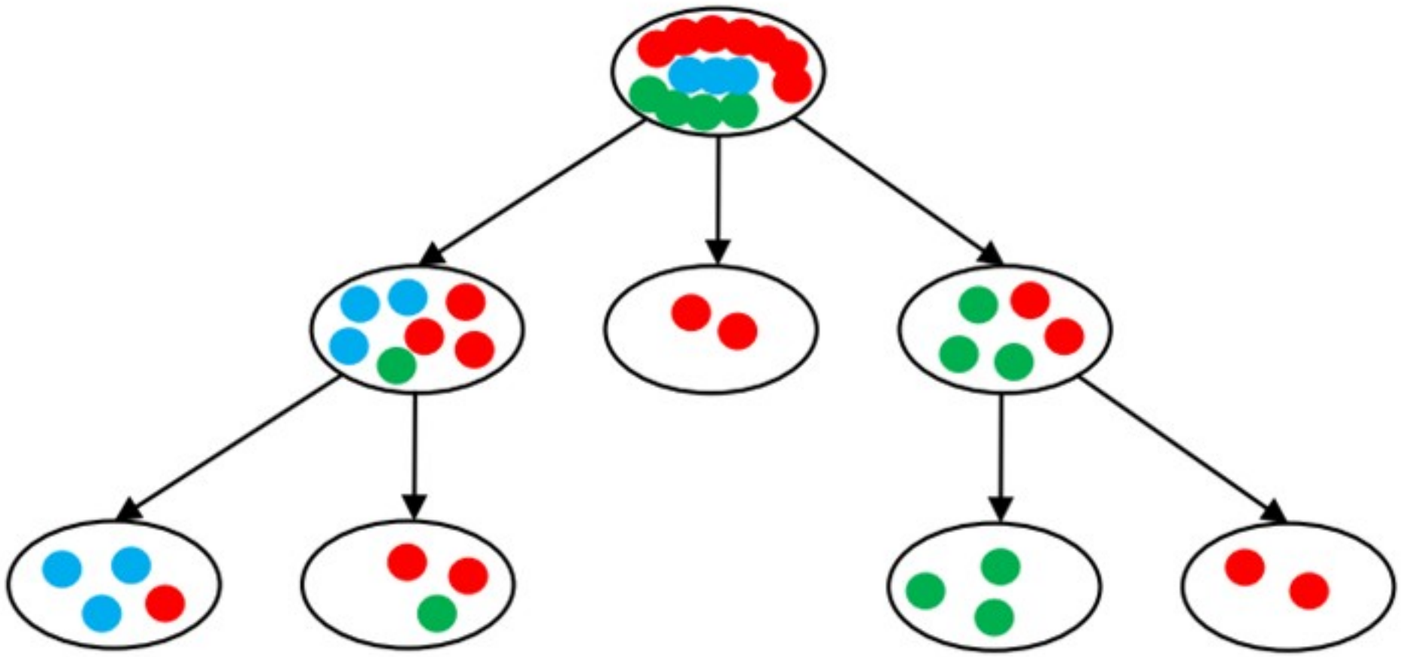
맨 위의 노드



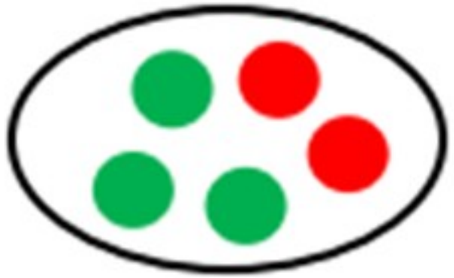
$$1 - \left(\left(\frac{7}{14}\right)^2 + \left(\frac{3}{14}\right)^2 + \left(\frac{4}{14}\right)^2\right)$$

$$= 0.6224$$

Gini Index



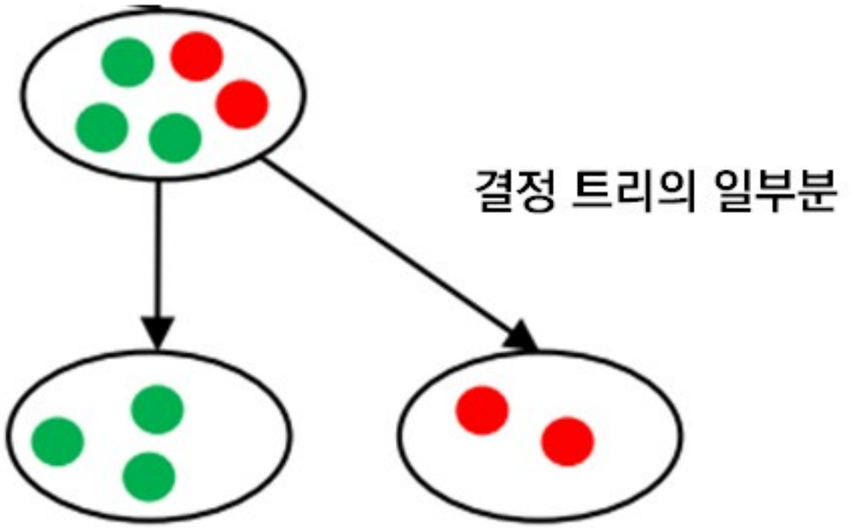
오른쪽 위에서 두 번째 노드



$$1 - \left(\left(\frac{2}{5}\right)^2 + \left(\frac{3}{5}\right)^2\right) = 0.52$$

Information Gain

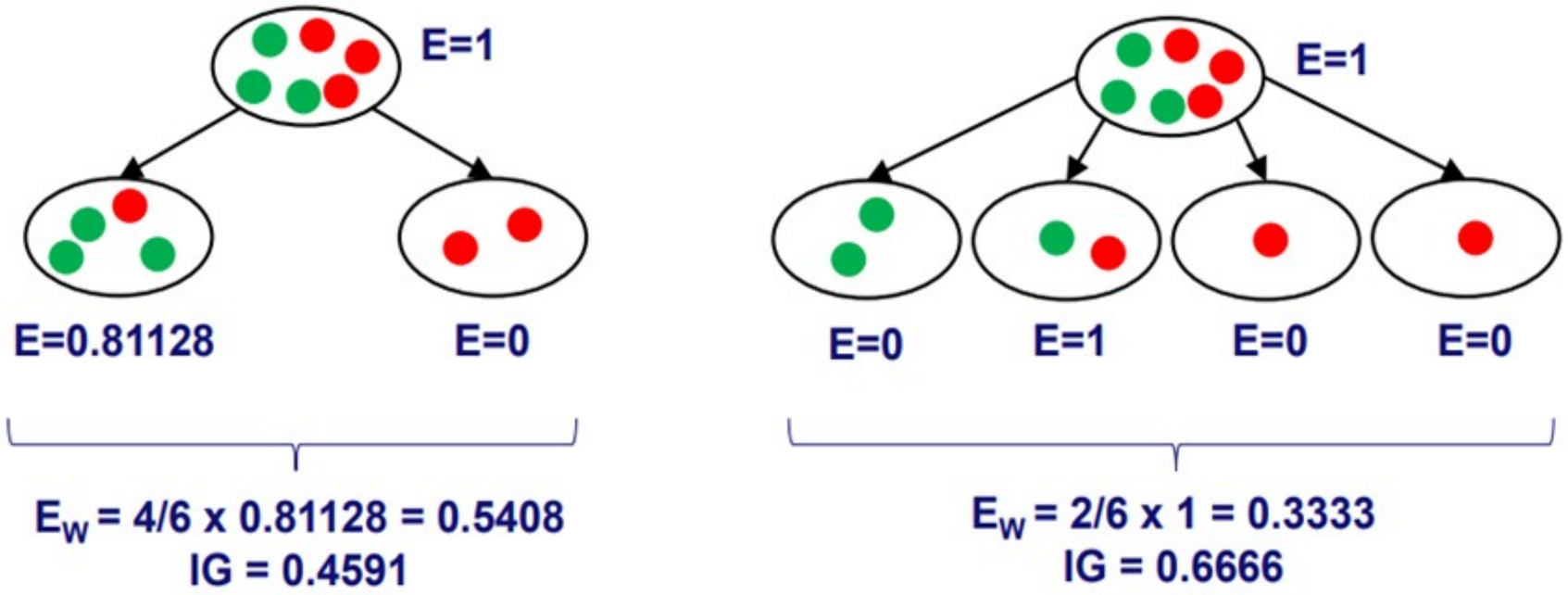
- 위에서 살펴본 Impurity 지표들을 바탕으로 우리는 각 node들의 복잡성을 계산할 수 있다.
  그리고 이 impurity를 바탕으로,
  (decision tree에 의해 나누기 전의 impurity – 나누어진 subset들의 impurity) 값을 통해
  impurity가 얼마나 개선 되었는지를 계산할 수 있고, 이를 information gain이라고 한다.



출처 : process-mining.tistory.com/106

Information Gain

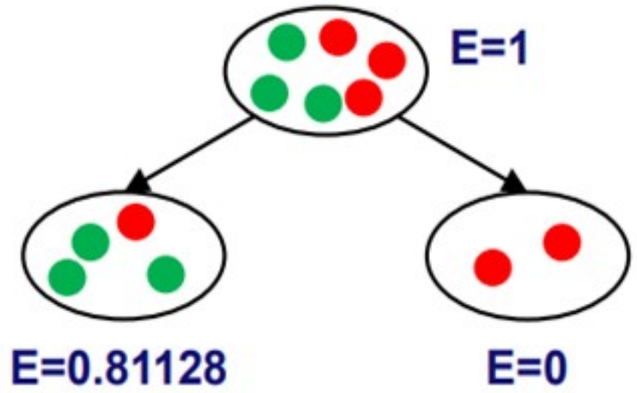
- impurity지표로 GINI index를 사용한다고 가정하자. 그러면 위 노드(parent node)의 GINI index는 우리가 위에서 계산했듯이 0.52가 되고, 아래 노드(child node)의 GINI index는 각각 0이므로 합하면 0이 된다. 그러므로 우리의 information gain은 0.52가 된다. 하지만 이러한 information gain은 한계점을 지닌다. 왜냐하면 split을 더 잘게 많이 할수록 information gain 값은 커질 수밖에 없기 때문이다.



출처 : process-mining.tistory.com/106

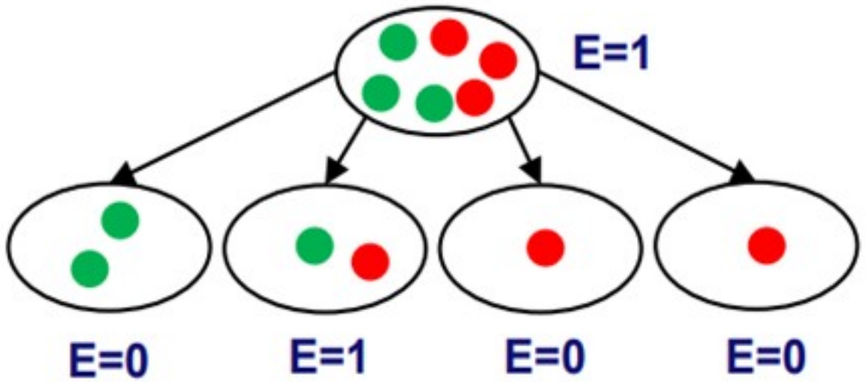
Information Gain Ratio

$$GR(d, \mathcal{D}) = \frac{IG(d, \mathcal{D})}{-\sum_{l \in levels(d)} (P(d = l) \times log_2(P(d = l)))}$$



E_W = 4/6 x 0.81128 = 0.5408
IG = 0.4591

$$-(\frac{4}{6}\log_2\frac{4}{6} + \frac{2}{6}\log_2\frac{2}{6}) = 0.91830$$



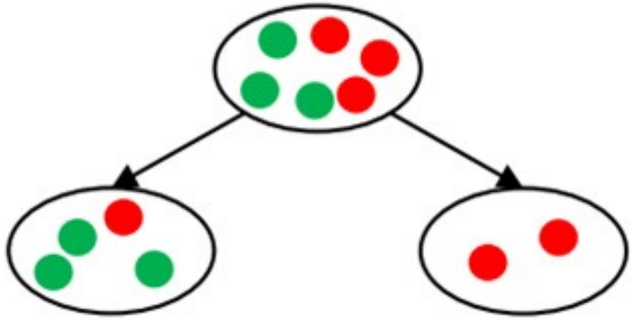
E_W = 2/6 x 1 = 0.3333
IG = 0.6666

$$-(\frac{2}{6}\log_2\frac{2}{6} + \frac{2}{6}\log_2\frac{2}{6} + \frac{1}{6}\log_2\frac{1}{6} + \frac{1}{6}\log_2\frac{1}{6}) = 1.91830$$

출처 : process-mining.tistory.com/106

Information Gain Ratio

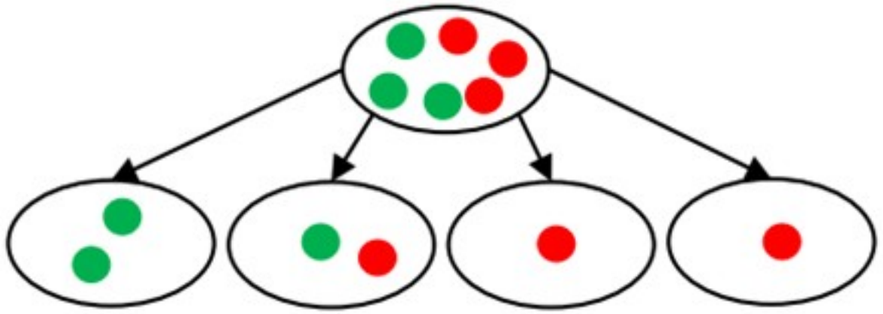
$$GR(d, \mathcal{D}) = \frac{IG(d, \mathcal{D})}{-\sum_{l \in levels(d)} (P(d = l) \times log_2(P(d = l)))}$$



(4,2) ⇒ E'=0,91830

IG = 0.4591

GR = 0.4591/E' = 0.50



(2,2,1,1) ⇒ E'=1,91830

IG = 0.6666

GR = 0.6666/E' = 0.34

결정 트리의 종류

알고리즘	평가지수 (선택방법)	비고
ID3	Entropy	다지분리(범주)
C4.5	Information Gain	다지분리(번주) 및 이진분리(수치)
C5.0	Information Gain	C4.5와 거의 유사 (차이점)
CHAID	카이제곱(범주), F검정(수치)	통계적 접근 방식
CART	Gini Index(범주), 분산의 차이(수치)	통계적 접근 방식, 항상 2진 분리

결정 트리의 모델 학습

- 입력 변수 영역을 두개로 구분하는 재귀적 분기(Recursive partitioning)와
  너무 자세하게 구분된 영역을 통합하는 가지치기(Pruning) 으로 구성 됨

- 먼저 재귀적 분기부터 알아보면,
  24개 가정을 대상으로
  소득(income),
  주택크기(Lot size),
  잔디깎기 구입 여부(Ownership)를
  조사한 데이터이다.

- 소득과 주택크기를 설명변수(X),
  기계 구입 여부를 종속변수(Y)로 하는
  분류 나무 모델 학습을 해보자.

Income	Lot size	Ownership	Income	Lot size	Ownership
60.0	18.4	Owner	75.0	19.6	Non-owner
85.5	16.8	Owner	52.8	20.8	Non-owner
64.8	21.6	Owner	64.8	17.2	Non-owner
61.5	20.8	Owner	43.2	20.4	Non-owner
87.0	23.6	Owner	84.0	17.6	Non-owner
110.1	19.2	Owner	49.2	17.6	Non-owner
108.0	17.6	Owner	59.4	16.0	Non-owner
82.8	22.4	Owner	66.0	18.4	Non-owner
69.0	20.0	Owner	47.4	16.4	Non-owner
93.0	20.8	Owner	33.0	18.8	Non-owner
51.0	22.0	Owner	51.0	14.0	Non-owner
81.0	20.0	Owner	63.0	14.8	Non-owner

출처 : rats'go

결정 트리의 모델 학습

- 먼저 한 변수 기준(예컨대 주택 크기)으로 정렬하고, 이후 가능한 모든 분기점에 대해 엔트로피/ 지니계수를 구해 분기 전과 비교해 정보획득을 조사함.

Income	Lot size	Ownership
51.0	14.0	Non-owner
63.0	14.8	Non-owner
59.4	16.0	Non-owner
47.4	16.4	Non-owner
85.5	16.8	Owner
64.8	17.2	Non-owner
108.0	17.6	Owner
84.0	17.6	Non-owner
49.2	17.6	Non-owner
60.0	18.4	Owner
66.0	18.4	Non-owner
33.0	18.8	Non-owner

Income	Lot size	Ownership
110.1	19.2	Owner
75.0	19.6	Non-owner
69.0	20.0	Owner
81.0	20.0	Owner
43.2	20.4	Non-owner
61.5	20.8	Owner
93.0	20.8	Owner
52.8	20.8	Non-owner
64.8	21.6	Owner
51.0	22.0	Owner
82.8	22.4	Owner
87.0	23.6	Owner

분기 전 엔트로피 $= -\frac{1}{2}\log_2(\frac{1}{2}) - \frac{1}{2}\log_2(\frac{1}{2}) = 1$

분기 후 엔트로피 $= \frac{1}{24}(-\log_2 1) + \frac{23}{24}\left(-\frac{12}{23}\log_2(\frac{12}{23}) - \frac{11}{23}\log_2(\frac{11}{23})\right) \approx 0.96$

정보획득 $= 1 - 0.96 = 0.04$

출처 : rats'go

결정 트리의 모델 학습

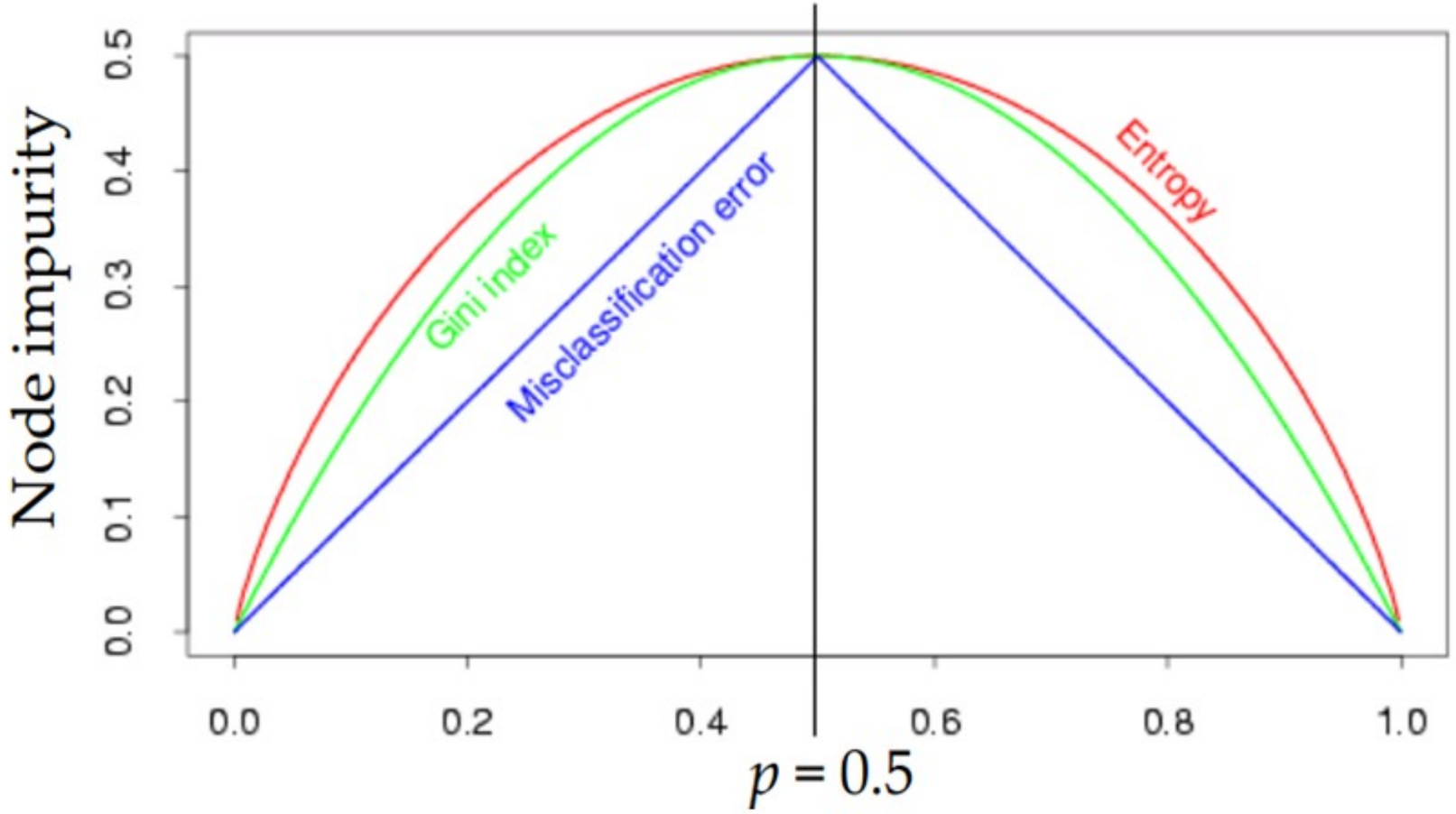


- 모든 terminal node의 순도가 100%인 상태를 Full tree라고 함.
  Full tree를 생성한 뒤 적절한 수준에서 terminal node를 결합해줘야 함.
  왜냐하면 과적합(Overfitting)할 염려가 생기기 때문.

- 의사결정나무의 분기 수가 증가 할 때 처음에는 새로운 데이터에 대한 오분류율이 감소하나 일정 수준 이상이 되면 오분류율이 되레 증가하는 현상이 발생함.
  이러한 문제를 해결하기 위해서 검증 데이터에 대한 오분류율이 증가하는 시점에서 적절히 가지치기를 수행해줘야 함.

출 처 : rats'go

결정 트리의 모델 학습



출처 : rats'go

5. Random Forest

- Random Forest
- Bagging
- Pasting

Random Forest



- 결정 트리 분류기 여러 개를 훈련시켜 그 결과들을 가지고 예측을 하는데, 각각의 결정 트리를 훈련시킬 때 **배깅**(Bagging)을 사용함.

- **앙상블 방식**을 사용하기 때문에 몇몇의 결정 트리들이 과대 적합하거나 과소 접합되어도 해당하는 오차가 상대적으로 무시되는 경향이 있어서 일반적으로 의사 결정 트리 하나를 사용한 모델보다 좋은 성능을 보임.

Random Forest



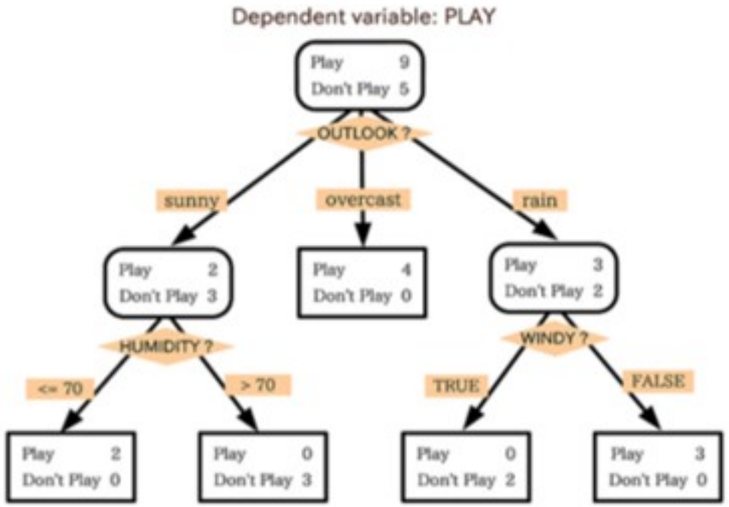
- **다수결의 원칙**에 따름
- 의견을 통합하거나 여러 가지 결과를 합치는 방식을 **앙상블**(Ensemble)이라고 함
- 하나의 거대한(깊이가 깊은) 결정 트리를 만드는 것이 아니라

  여러 개의 작은 결정 트리를 만드는 것.
- 여러 개의 작은 결정 트리가 예측한 값들 중 가장 많은 값(분류일 경우)

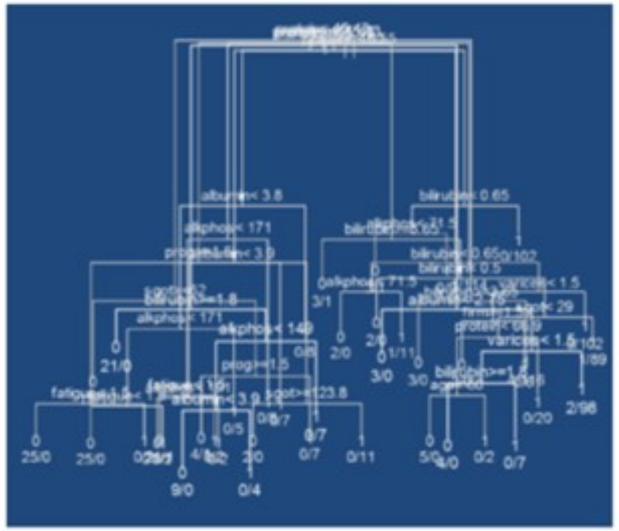
  혹은 평균값(회귀일 경우)을 최종 예측 값으로 정함 -〉 **집단지성의 힘**

Random Forest

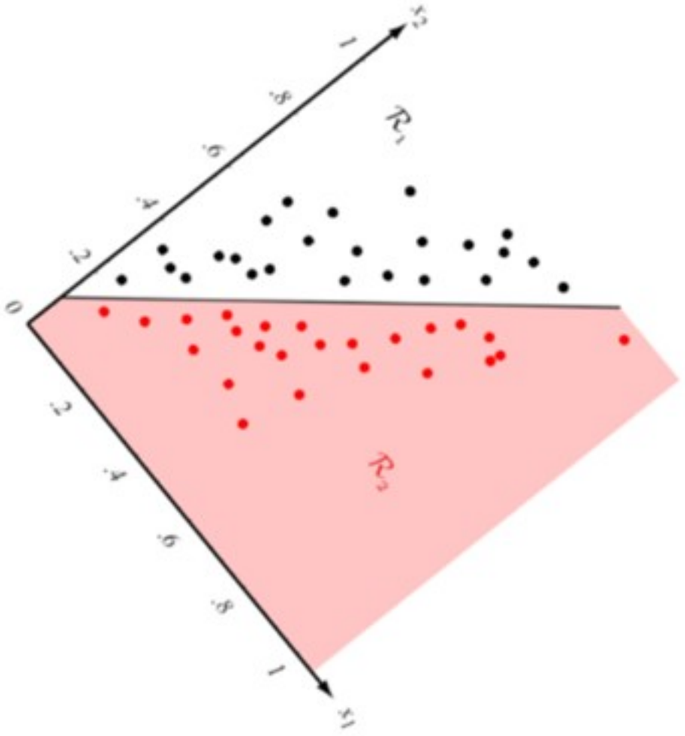
Tree



Random Forest



Rotation Forest



scikit learn

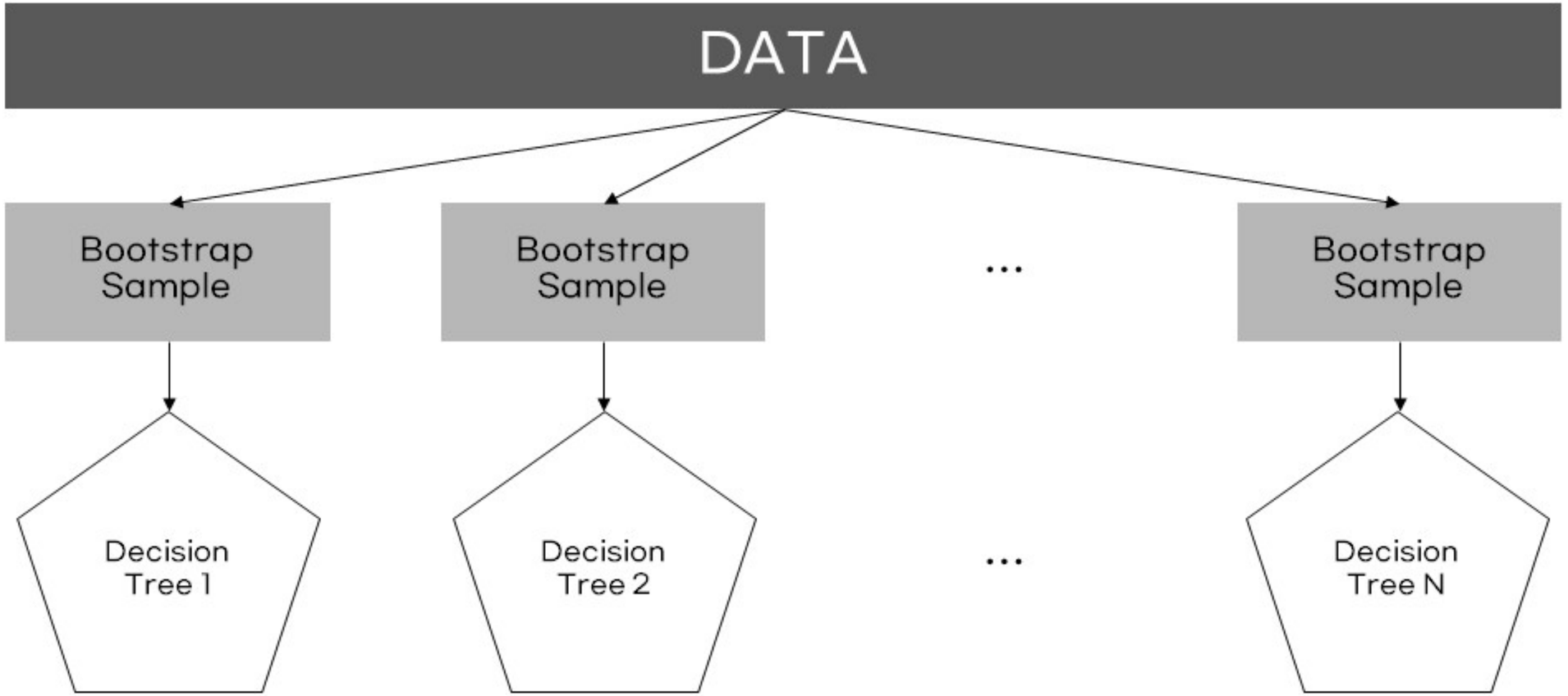
Random Forest



Bagging

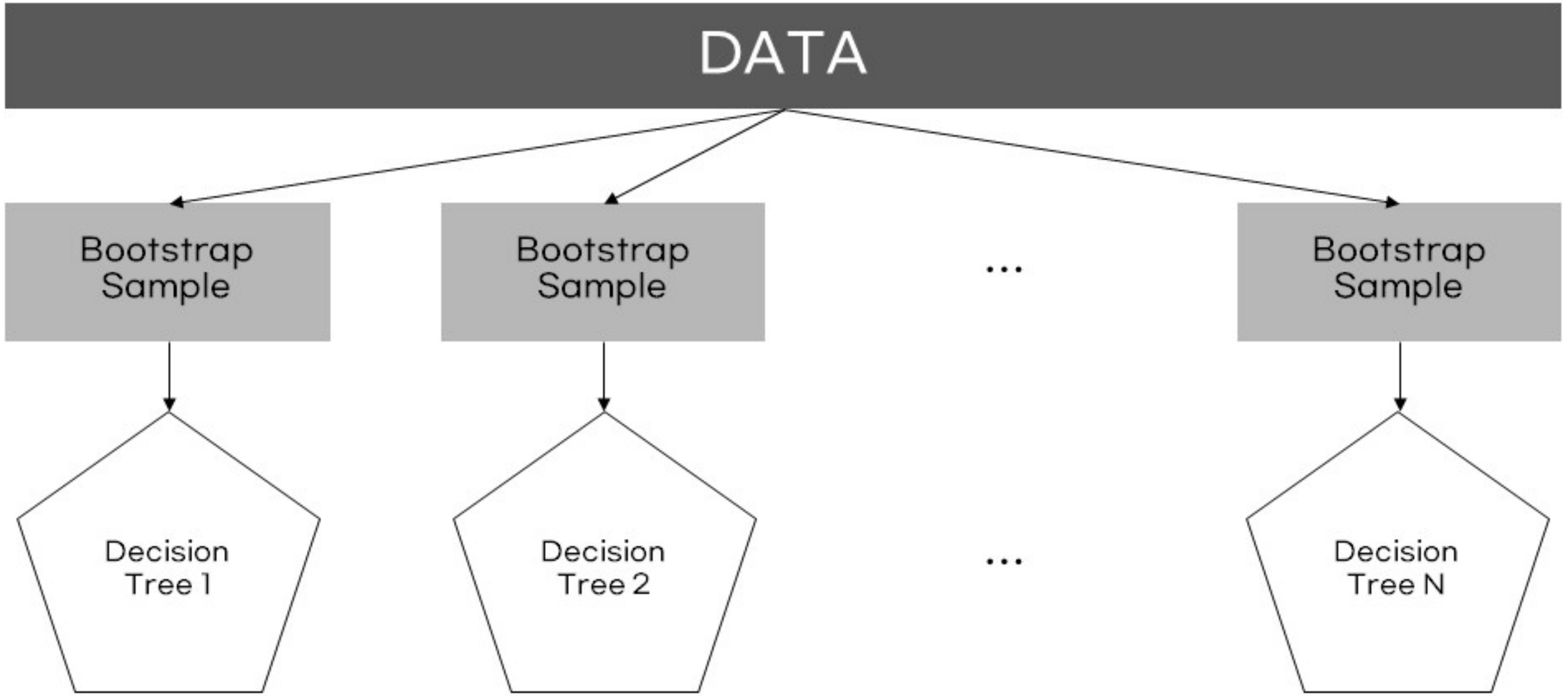
- Bagging은 Bootstrap Aggregation 의 약자로 훈련세트에서 **중복을 허용**하여 샘플링 하는 방식

  배깅(Bagging, Bootstrap Aggregation) 이라고 함



Pasting

- 훈련세트에서 **중복을 허용하지 않고** 샘플링 하는 방식을 페이스팅(Pasting)이라고 함



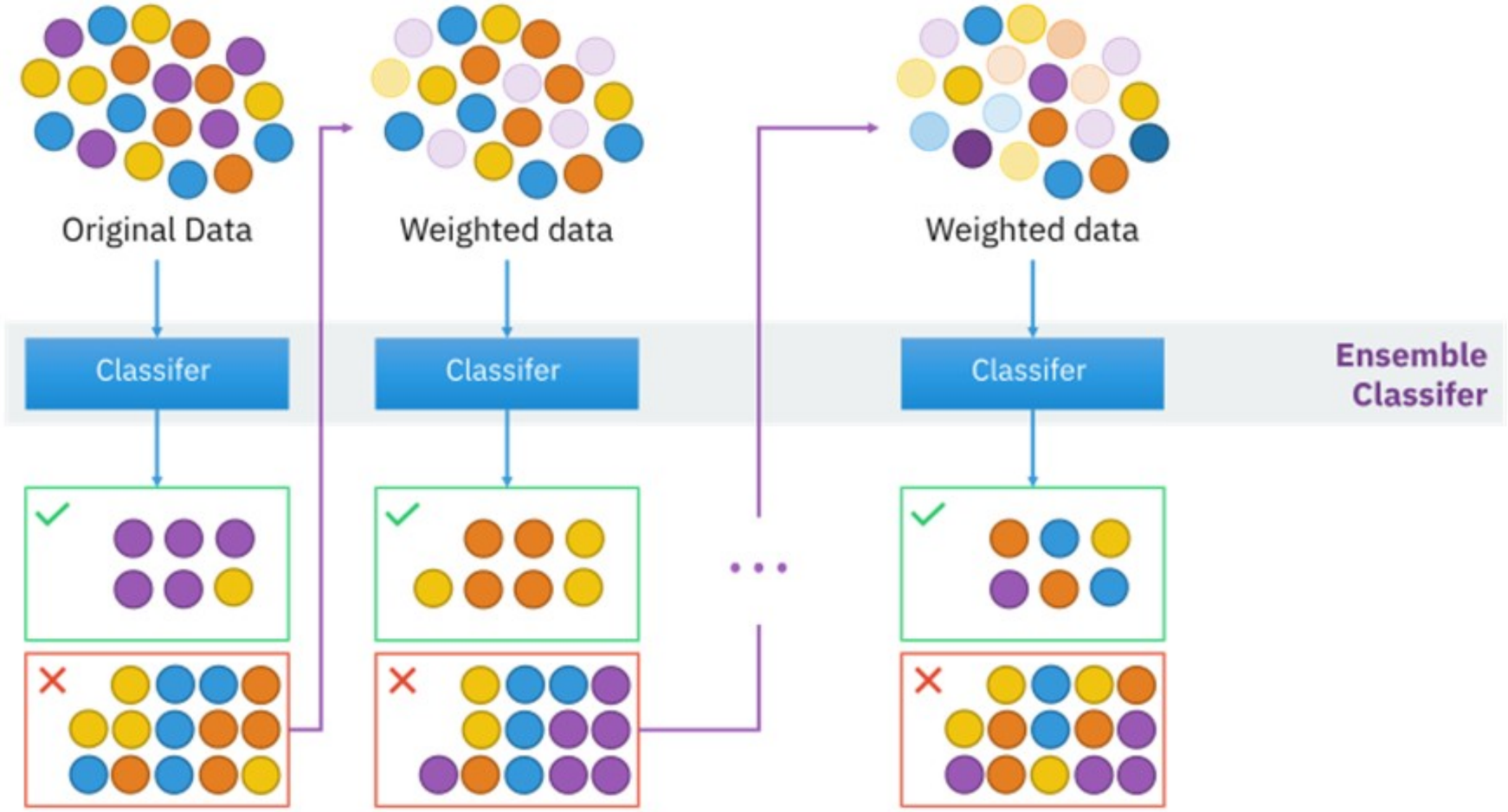
6. Boosting 계열

- Boosting
- Boosting Model
- Bagging VS Boosting
- Ada Boosting (Adaptive Boosting)
- Gradient Boosting
- Xgboost, A Scalable Tree Boosting System
- LightGBM, A highly Efficient Gradient Boosting Decision Tree
- Catboost, Unbiased Boosting with categorical features

Boosting

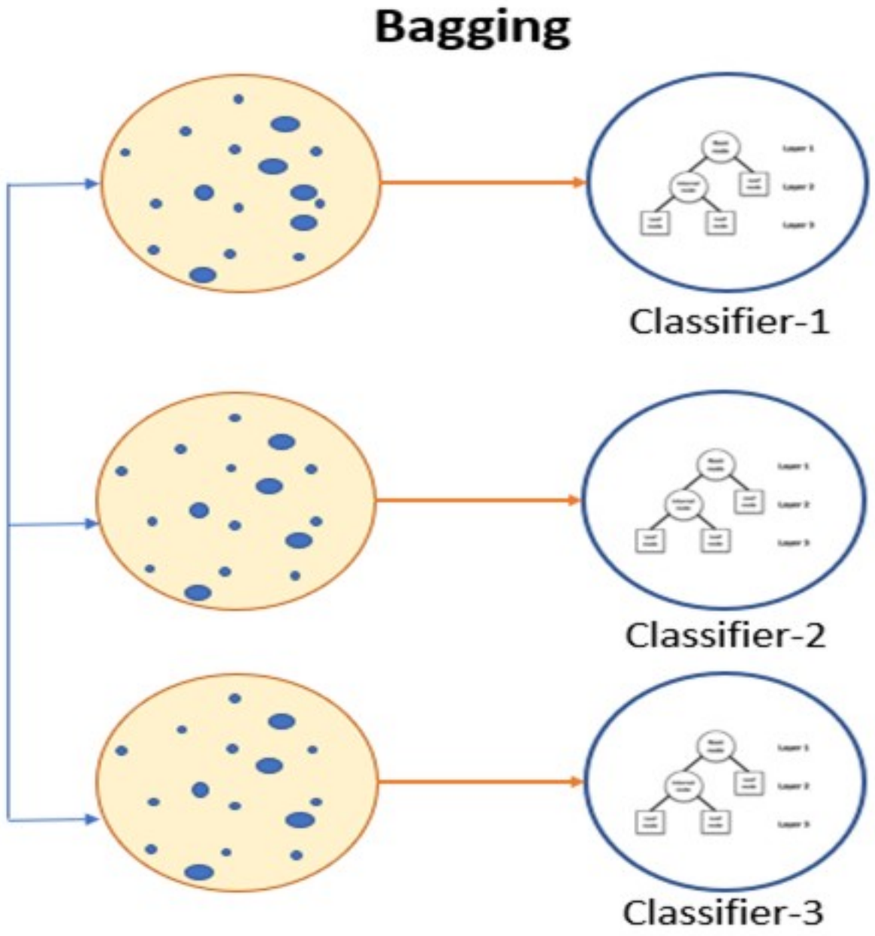
- 여러 개의 약한 학습기(Weak Learner)를 순차적으로 학습-예측하면서
  잘못 예측한 데이터에 가중치 부여를 통해 오류를 개선해 나가면서 학습하는 방식
- 순차적으로 하다보니 시간이 오래 걸림
- 대표적인 Ada Boosting, Gradient Boosting이 있음
- 대표적인 Weak Learner는 다음과 같음
  - Decision Stumps : depth가 1인 Decision Tree
  - Shallow Decision Trees
  - Naïve Bayes
  - Logistic Regression

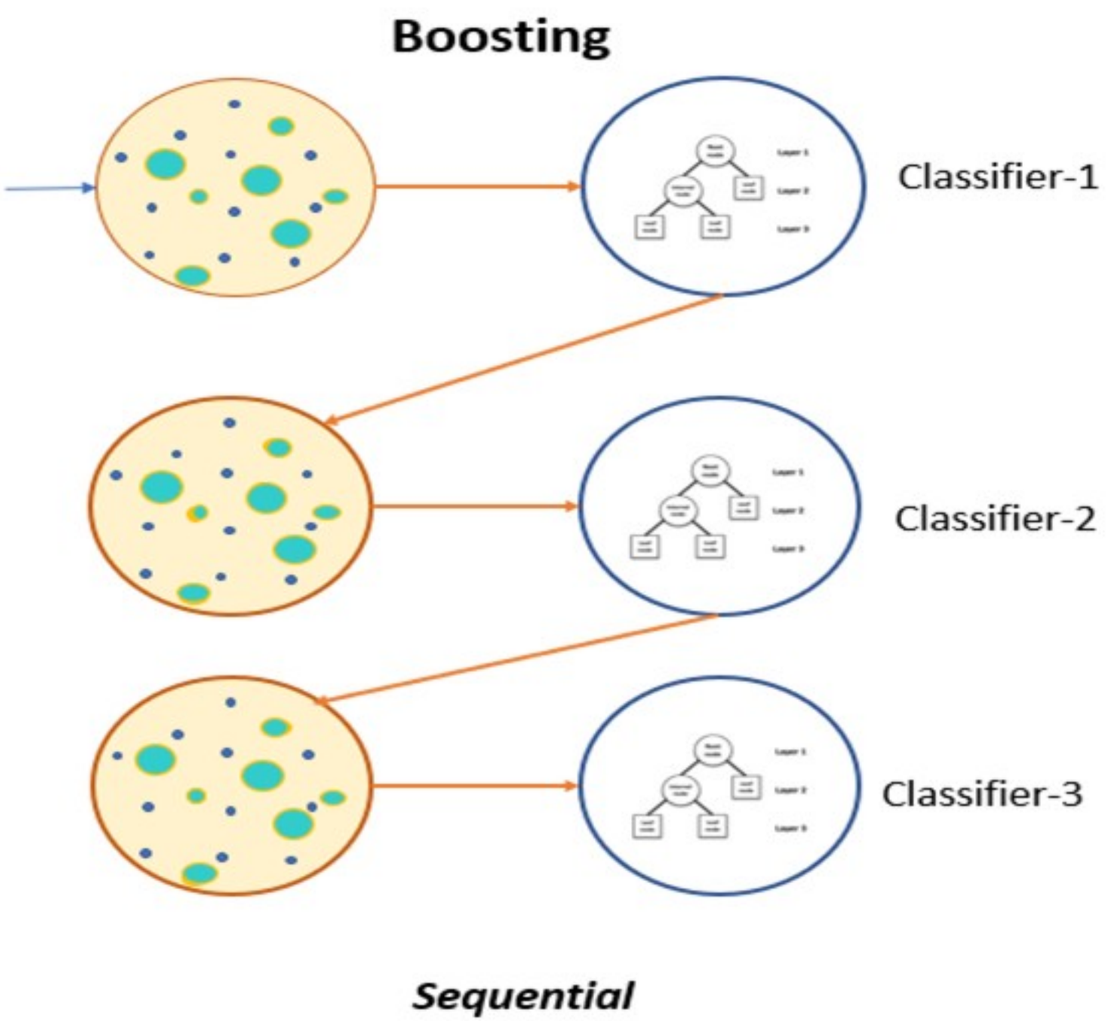
Boosting Model



출처 : Wikipedia

Bagging VS Boosting





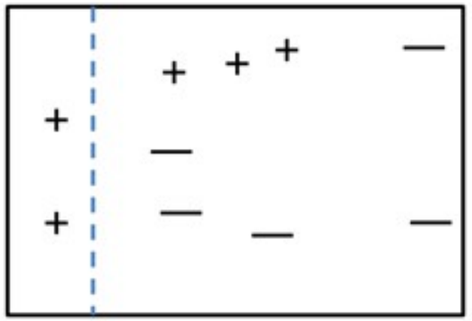
출처 : www.pluralsight.com/guides/ensemble-methods:-bagging-versus-boosting

Ada Boosting(Adaptive Boosting)

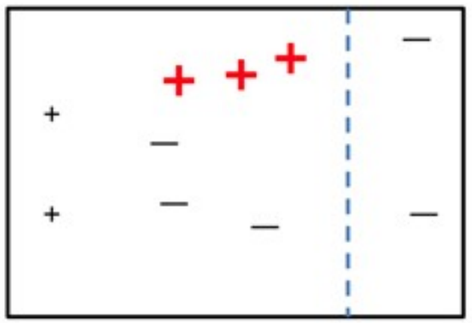
- Yoav Freund & Robert Schapire가 제안하였고, Weak Learner를 반복적으로 사용하고, 그 결과를 합하여 모델의 Accuracy를 향상 시킴.
- Adaptive+Boosting로 만들어진 단어.
- 약한 분류기(Weak Classifier)들이 상호보완 하도록 순차적(Sequential)으로 학습하고, 이들을 조합하여 최종적으로 강한 분류기(Strong Classifier)의 성능을 향상시키는 것.
- 예측 성능이 낮은 약한 분류기들을 조합하여 조금 더 성능이 좋은 강한 분류기 하나를 만드는 것. 약한 분류기들이 상호 보완적(Adaptive)으로 학습해 나가고, 이러한 약한 분류기들을 조합하여 하나의 분류기를 만들기 때문에 Boosting이 된다

Ada Boosting(Adaptive Boosting)

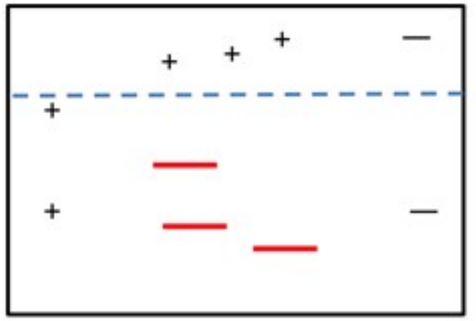
첫 번째 약 분류기(Decision stumps / 결정 크루터기)



두 번째 약 분류기



세 번째 약 분류기



오른쪽 3개 + : 오분류

왼쪽 3개 — : 오분류

weight 1 ×



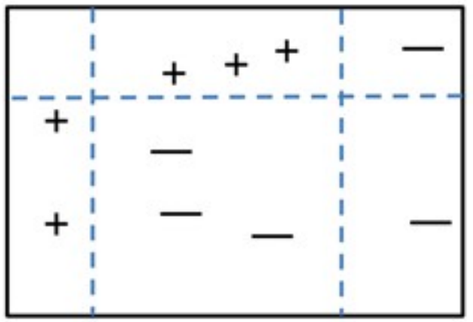
+ weight 2 ×



+ weight 3 ×



=



최종 (강) 분류기

Gradient Boosting

- Adaboost와 거의 비슷한 매커니즘을 가지고 있음.
- 일반적인 머신러닝 앙상블의 부스팅과 똑같이 약한 학습기(Weak Learner) 를 만드는 방식
- 가중치 부여->Gradient Descent를 사용.
- Gradient Boosting에서는 Gradient가 현재까지 학습된 Classifier의 약점(weak)을 알려주고, 이후 모델이 그것을 중점으로 해서 보완을 하는 방식.

Gradient Boosting

Input: training set $\{(x_i, y_i)\}_{i=1}^{n}$, a differentiable loss function $L(y, F(x))$, number of iterations M.

Algorithm:

1. Initialize model with a constant value:

$$F_0(x) = \arg\min_{\gamma} \sum_{i=1}^{n} L(y_i, \gamma).$$

2. For m = 1 to M:
    1. Compute so-called *pseudo-residuals*:

$$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F(x)=F_{m-1}(x)} \quad \text{for } i = 1, \ldots, n.$$

    2. Fit a base learner (or weak learner, e.g. tree) $h_m(x)$ to pseudo-residuals, i.e. train it using the training set $\{(x_i, r_{im})\}_{i=1}^{n}$.
    3. Compute multiplier γ_m by solving the following one-dimensional optimization problem:

$$\gamma_m = \arg\min_{\gamma} \sum_{i=1}^{n} L\left(y_i, F_{m-1}(x_i) + \gamma h_m(x_i)\right).$$

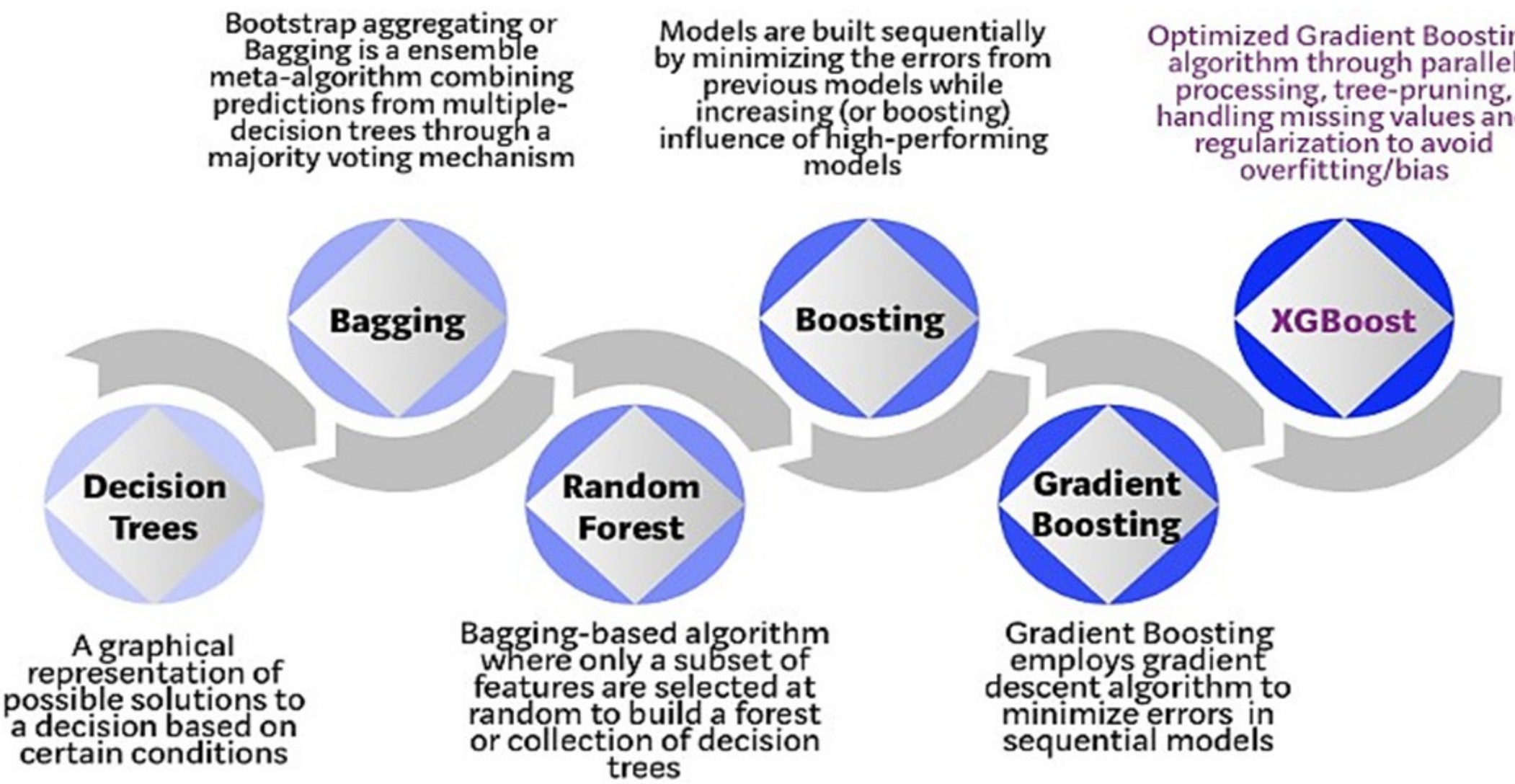
    4. Update the model:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x).$$

3. Output $F_M(x)$.

출 처 : 위키피디아

XGboost, A Scalable Tree Boosting System.



XGboost, A Scalable Tree Boosting System.



XGboost, A Scalable Tree Boosting System.

1.Introduction

2. Tree Boosting in a Nutshell
- 2.1 Regularized Learning Objective
- 2.2 Gradient Tree Boosting

3. Split Finding Algorithms
- 3.1 Basic Exact Greedy Algorithm
- 3.2 Approximate Algorithm
- 3.3 Weighted Qunatile Sketch
- 3.4 Sparsity-aware Split Finding

4. System Design
- 4.1 Colum Block for Parallel Learning
- 4.2 Cache-aware Access
- 4.3 Blocks for Out-of-Core Computation

5. Ralated Works

6. End to End Evaludation

XGboost, A Scalable Tree Boosting System.

1.Introduction

XGBoost

eXtreme Gradient Boosting

Scalability
확장성

- Massive on-line course dropout rate prediction
- Store sales prediction
- High energy physics event classification
- Web text classification
- Customer behavior prediction
- Motion detection
- Ad click through rate prediction
- Malware classification
- Product categorization
- Hazard risk prediction

Innovations

Algorithmatic

- Tree Boosting
- Split Finding Algorithms 'Regularized Gradient Boosting'

Systematic

- System Design

XGboost, A Scalable Tree Boosting System.

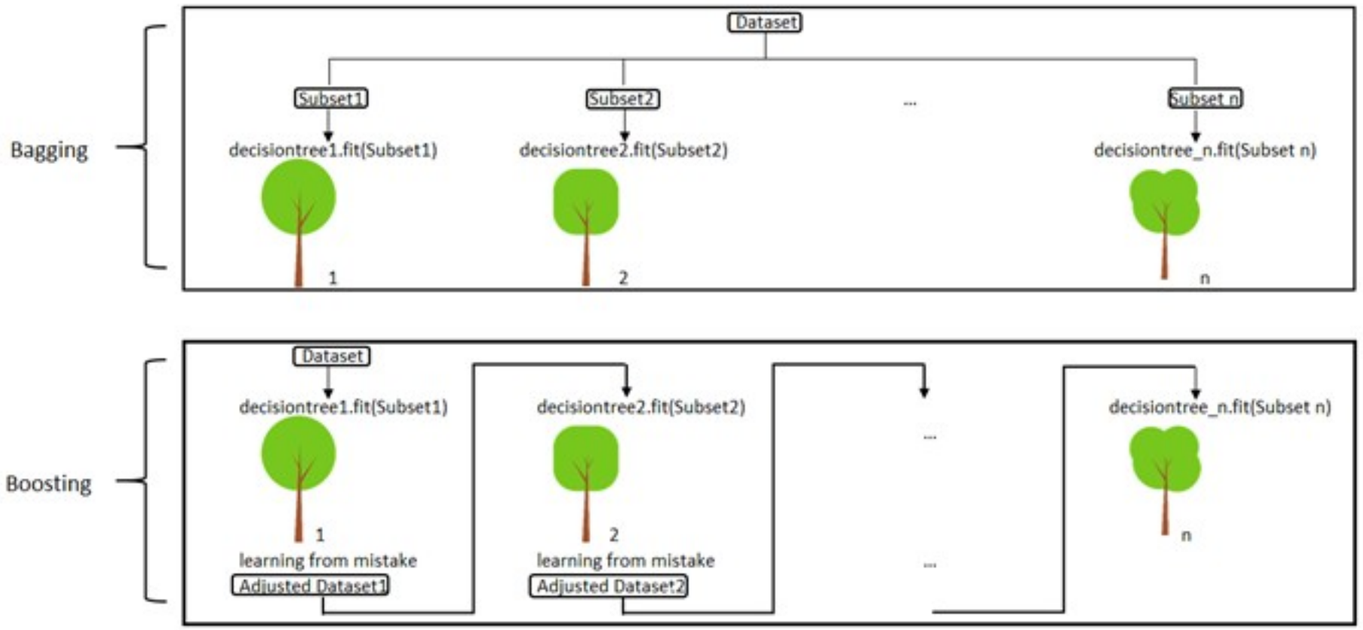
2. Tree boosting in a nutshell

기본적인 Tree Boosting

$D = \{(x_i, y_i)\}$ Given Data : n개의 Example과 m개의 Feature

$\hat{y} = \emptyset(x) = \sum_{k=1}^{K} f_k(x_i), f_k \in F$ 모든 함수들을 담고 있는 Function space

$= f_1(x_i) + f_2(x_i) + f_3(x_i) + \cdots + f_k(x_i)$



XGboost, A Scalable Tree Boosting System.

Tree Boosting in a Nutshell

n : examples, m : Feature를 가진 데이터 셋 $D = \{(x_i, y_i)\}(|D| = n, x_i \in R^m, y_i \in R)$이 주어져 있다고 하자,
트리 앙상블 모델(Figure 1)은 Output을 예측하기 위해 K개의 Additive 함수를 사용함

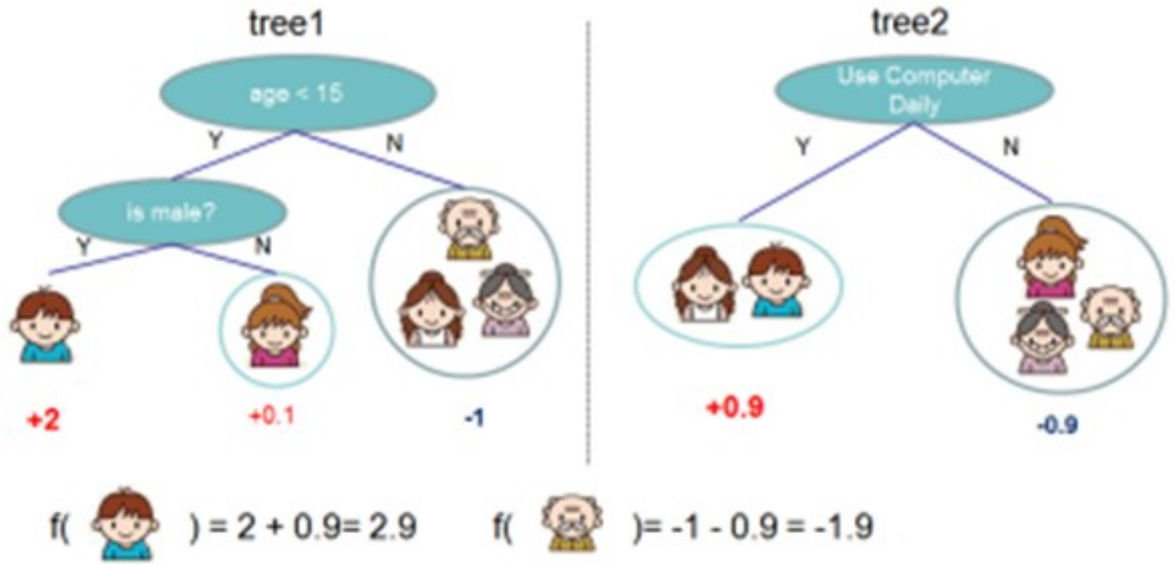


Figure 1: Tree Ensemble Model. The final prediction for a given example is the sum of predictions from each tree.

XGboost, A Scalable Tree Boosting System.

Tree Boosting in a Nutshell

$$\hat{y}_i = \phi(\mathbf{x}_i) = \sum_{k=1}^{K} f_k(\mathbf{x}_i), \quad f_k \in \mathcal{F}, \tag{1}$$

$$\mathcal{F} = \{f(\mathbf{x}) = w_{q(\mathbf{x})}\}(q : \mathbb{R}^m \rightarrow T, w \in \mathbb{R}^T)$$: 회귀 나무(CART)

q : 입력 벡터를 회귀 나무의 끝마디 인덱스를 매핑하는 함수.

T : 트리 잎 개수(끝마디 총 개수)

f_k : 독립적인 나무 구조 q&잎 가중치 w

Regularized 목적함수를 최소화 해야함

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k) \tag{2}$$

$$\text{where } \Omega(f) = \gamma T + \frac{1}{2}\lambda \|w\|^2$$

XGboost, A Scalable Tree Boosting System.

Tree Boosting in a Nutshell

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$$

이 Loss Function은

- Function을 파라미터로 사용하기 때문에,
- Numerical Vector가 아니기 때문에

Euclidean Space를 활용하여 Optimization이 불가능 함.

Solution : Additive Training(Boosting)

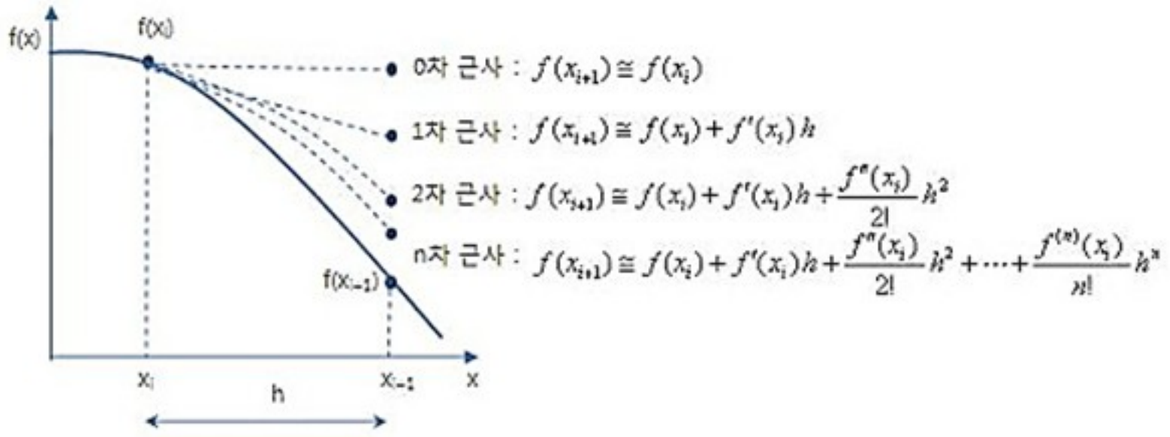
$$\widehat{y_i^0} = 0$$

$$\widehat{y_i^1} = f_1(x_i) = \widehat{y_i^1} + f_1(x_i)$$

$$\widehat{y_i^1} = f_1(x_i) + f_2(x_i) = \widehat{y_i^1} + f_2(x_i)$$

i 번째 Prediction　　$t-1$ 번째 Prediction　Added Function

$$\widehat{y_i^t} = \sum_{k=1}^{K} f_k(x_i) = \widehat{y_i^{t-1}} + f_t(x_i)$$



XGboost, A Scalable Tree Boosting System.

Gradient Tree Boosting

다음과 같은 목적함수를 최소화 하기 위해 f_i를 추가해야 됨

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y_i}^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t)$$

일반적인 환경에서 목적함수를 최적화 하기 위해, Second Order Taylor Expansion을 사용

$$\mathcal{L}^{(t)} \simeq \sum_{i=1}^{n} [l(y_i, \hat{y}^{(t-1)}) + g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i)] + \Omega(f_t)$$

$$g_i = \partial_{\hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)}) \text{ and } h_i = \partial^2_{\hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)})$$

g_i : 손실함수의 1차(First Order) Gradient 통계량 / h_i : 손실함수의 2차(Second Gradient) 통계량

XGboost, A Scalable Tree Boosting System.

Gradient Tree Boosting

상수 Terms 제거해서 다음과 같이 t에서 단순화된 목적함수를 얻을 수 있음.

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^{n} [g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i)] + \Omega(f_t)$$

잎 j의 인스턴스 집합을 $I = \{i|q(x_i) = j\}$로 정의

우리는 Regularization Term Ω를 확장하여, 수식을 다시 작성할 수 있다.

$$\begin{aligned}\tilde{\mathcal{L}}^{(t)} &= \sum_{i=1}^{n} [g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i)] + \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 \\ &= \sum_{j=1}^{T} [(\sum_{i \in I_j} g_i) w_j + \frac{1}{2} (\sum_{i \in I_j} h_i + \lambda) w_j^2] + \gamma T\end{aligned}$$

XGboost, A Scalable Tree Boosting System.

(1) 원래의 목적 함수

$$\tilde{L}^{(t)} = \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^{T} w_j^2$$

각 항목의 의미

- g_i : 손실 함수의 **일阶 도함수(Gradient)**
- h_i : 손실 함수의 **이阶 도함수(Hessian)**
- $f_t(x_i)$: 새로운 트리의 예측 값
- γT : 리프 노드의 개수 T에 대한 정규화 항 (트리의 복잡도를 줄이기 위함)
- $\lambda \sum_{j=1}^{T} w_j^2$: 리프 노드의 가중치 w_j에 대한 정규화 항

(2) 리프 노드 기준으로 변환

트리는 여러 리프(leaf) 노드로 구성되며, 각 샘플 x_i는 특정 리프 j에 속합니다. 따라서, 트리의 모든 샘플을 리프 노드 기준으로 변환하면:

$$\tilde{L}^{(t)} = \sum_{j=1}^{T} \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T$$

변형된 식의 의미

- I_j : 리프 노드 j에 속한 샘플들의 집합
- $\sum_{i \in I_j} g_i$: 해당 리프 노드에 속한 샘플들의 **일阶 도함수(Gradient) 총합**
- $\sum_{i \in I_j} h_i$: 해당 리프 노드에 속한 샘플들의 **이阶 도함수(Hessian) 총합**
- w_j : 리프 노드 j의 최적 가중치 값
- λ : 정규화 항
- γT : 리프 노드 개수 T에 대한 벌칙 항

XGboost, A Scalable Tree Boosting System.

Gradient Tree Boosting

고정된 구조(Structure) q(x)의 경우, 다음과 같이
잎 j의 최적의 가중치 w_j^*를 계산할 수 있음 : $$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda},$$

다음과 같이 해당하는 최적의 목적함수를 계산할 수 있음: $$\tilde{\mathcal{L}}^{(t)}(q) = -\frac{1}{2} \sum_{j=1}^{T} \frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T.$$

위의 식에서 트리 구조 q의 품질을 측정하기 위해 함수에 점수를 매기는 역할로 사용 될 수 있음
이 점수는 보다 광범위한 목적 함수를 위해 도출된다는 점을 제외하면, 의사결정 나무를 평가하기 위한 불순도 점수와 같음

XGboost, A Scalable Tree Boosting System.

Gradient Tree Boosting

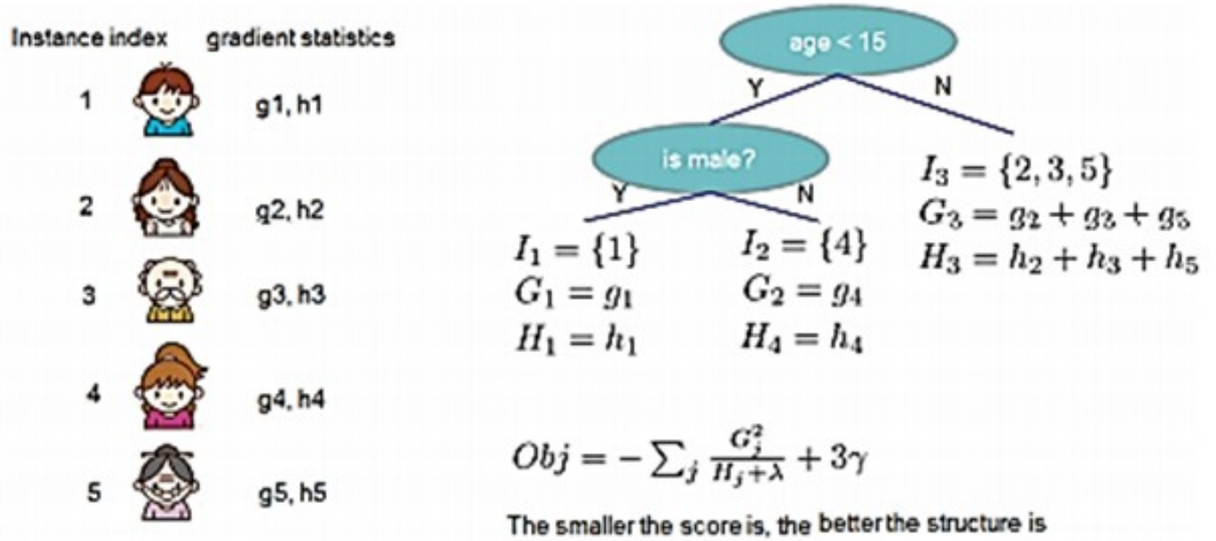


Figure 2: Structure Score Calculation. We only need to sum up the gradient and second order gradient statistics on each leaf, then apply the scoring formula to get the quality score.

XGboost, A Scalable Tree Boosting System.

Gradient Tree Boosting

보통, 가능한 모든 트리 구조 q를 나열하는 것은 불가능함.

대신, 한 잎에서 시작하여 반복적으로 트리를 가지(Branch)에 추가하는 탐욕 알고리즘이 사용됨.

I_L &I_R 이 분할 후의 왼쪽 & 오른쪽 인스턴스 집합이라고 가정하겠음.

$I = I_L \cup I_R$로 놓은 다음, 분할 후의 손실 감소는 다음과 같이 주어짐.

$$I_L : instance\ set\ of\ Left\ Node$$

$$I_R : instance\ set\ of\ Right\ Node$$

$$I_L : instance\ set\ of\ Current\ Node$$

$$\mathcal{L}_{split} = \frac{1}{2}\left[\frac{(\sum_{i\in I_L} g_i)^2}{\sum_{i\in I_L} h_i + \lambda} + \frac{(\sum_{i\in I_R} g_i)^2}{\sum_{i\in I_R} h_i + \lambda} - \frac{(\sum_{i\in I} g_i)^2}{\sum_{i\in I} h_i + \lambda}\right] - \gamma \tag{7}$$

2.3 Shrinkage and column subsampling

Regularized 목적함수 외에도, 추가적으로 Over-fitting을 예방하는 두 가지 기법이 사용됨.

1) Shrinkage

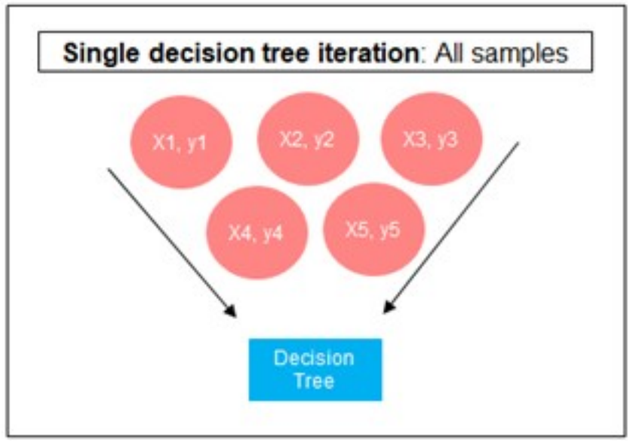
- 부스팅 트리의 각 단계 이후 마다 새롭게 추가된 가중치를 요인 η로 Scalling 함
- Stochastic 최적화의 Learning Rate와 유사하게 각 개별 트리의 영향도를 감소시키고, 모델을 향상시키기 위해 미래 트리 공간을 남겨 놓음

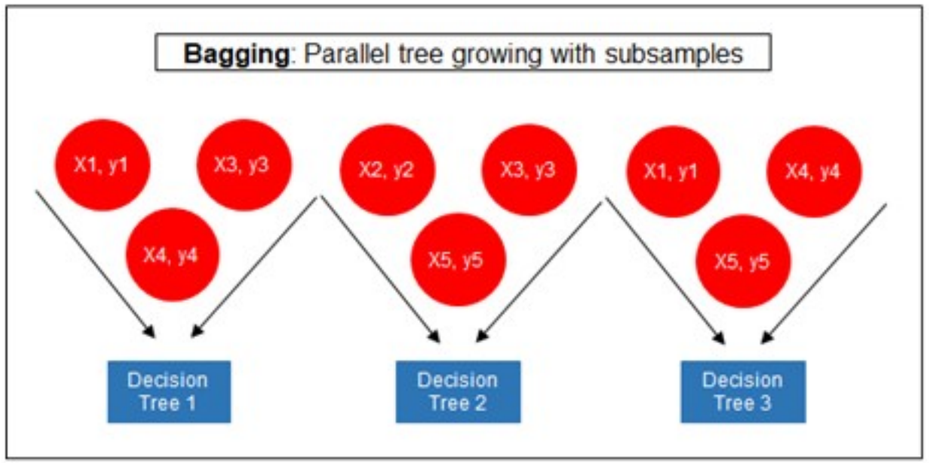
2) Column(Feature) Subsampling

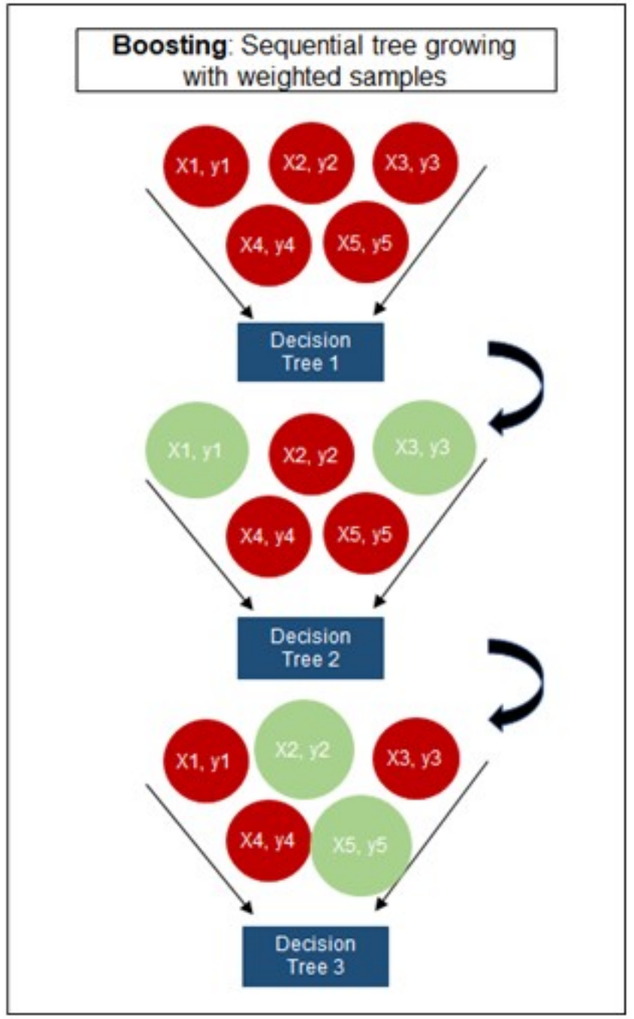
- 이 기법은 Random Forest에서 흔하게 사용되지만, 이전 부스팅 트리에서는 적용되지 않음
- 피드백에 따르면, 전통적인 Row Subsampling보다 Over-fitting을 더 잘 예방해준다고 함
- 이 기법을 사용하면 병렬 알고리즘 계산 속도를 가속화 함

XGboost, A Scalable Tree Boosting System.

2) Column(Feature) Subsampling







출처 : https://towardsdatascience.com/the-ultimate-guide-to-adaboost-random-forests-and-xgboost-7f9327061c4f#

3. Split Finding Algorithms

1) Basic Exact Greedy Algorithm

Algorithm 1: Exact Greedy Algorithm for Split Finding

Input: I, instance set of current node
Input: d, feature dimension
$gain \leftarrow 0$
$G \leftarrow \sum_{i \in I} g_i$, $H \leftarrow \sum_{i \in I} h_i$
for $k = 1$ ***to*** m **do**
  $G_L \leftarrow 0$, $H_L \leftarrow 0$
  for j *in sorted*$(I,$ *by* $\mathbf{x}_{jk})$ **do**
    $G_L \leftarrow G_L + g_j$, $H_L \leftarrow H_L + h_j$
    $G_R \leftarrow G - G_L$, $H_R \leftarrow H - H_L$
    $score \leftarrow \max(score, \frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda})$
  end
end
Output: Split with max score

XGboost, A Scalable Tree Boosting System.

Split Finding Algorithms

2) Approximate Algorithms for Split Finding

Algorithm 2: Approximate Algorithm for Split Finding

for $k = 1$ ***to*** m **do**
  Propose $S_k = \{s_{k1}, s_{k2}, \cdots s_{kl}\}$ by percentiles on feature k.
  Proposal can be done per tree (global), or per split(local).
end
for $k = 1$ ***to*** m **do**
  $G_{kv} \leftarrow= \sum_{j \in \{j | s_{k,v} \geq \mathbf{x}_{jk} > s_{k,v-1}\}} g_j$
  $H_{kv} \leftarrow= \sum_{j \in \{j | s_{k,v} \geq \mathbf{x}_{jk} > s_{k,v-1}\}} h_j$
end
Follow same step as in previous section to find max score only among proposed splits.

Split Finding Algorithms

2) Approximate Algorithms

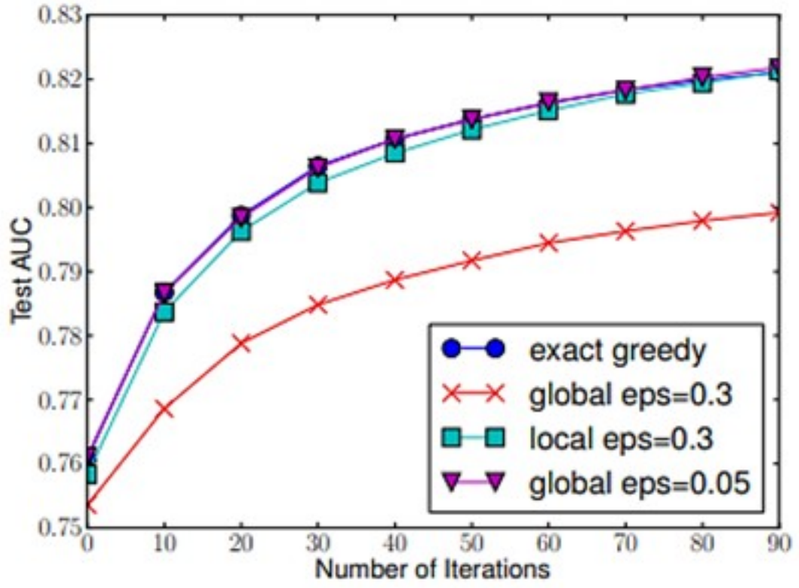
Global Variant:

초기 트리 구성 단계에서 모든 후보 분할들을 제시하고,

모든 levels에서 분할 찾기를 수행할 때 동일한 Proposals를 사용함

Local variant:

매번 분할이 진행된 후에 다시 제시



XGboost, A Scalable Tree Boosting System.

Split Finding Algorithms

3) Weighted Quantile sketch

Sketch Algorithm

- Sample Data로 Sketch하여 Original Data Distribution 파악

Quantile Sketch Algorithm

- 각 Quantile Sketch로 Original Distribution을 파악

Weighted Quantile Sketch Algorithm

- Normal Quantile :각 quantile의 데이터 개수가 같다
- Weighted quantile : 각 quantile 의 sum for weights가 같다

$$\mathcal{L}^{(t)} \simeq \sum_{i=1}^{n} [l(y_i, \hat{y}^{(t-1)}) + g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i)] + \Omega(f_t)$$

$$\sum_{i=1}^{n} \frac{1}{2} h_i (f_t(\mathbf{x}_i) - g_i/h_i)^2 + \Omega(f_t) + constant,$$

W가 weight의 역할을 함

XGboost, A Scalable Tree Boosting System.

Split Finding Algorithms

4) Sparsity-Aware Split Finding

많은 데이터들의 input variables는 sparse 한 경우가 많다.

Sparsity의 대표적인 원인

Presence of missing values in the data

Frequent zero values

Artifacts of feature engineering such as one-hot Encoding

Data

Example	Age	Gender
X1	?	male
X2	15	?
X3	25	female

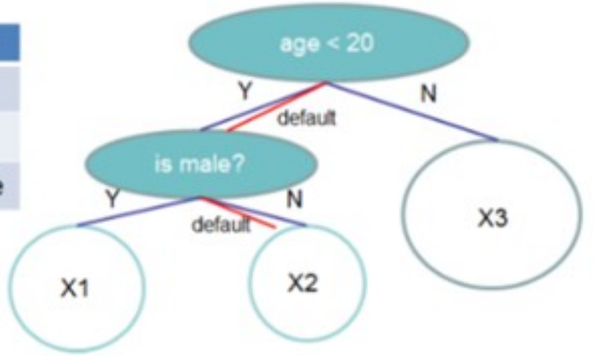


Figure 4: Tree structure with default directions. An example will be classified into the default direction when the feature needed for the split is missing.

Sparsity Aware! 〉 Add a Default Direction in each tree node.

모든 결측치는 기본 방향으로 이동한다!

갈 수 있는 방향은 두가지이며, 데이터로부터 학습한다!

XGboost, A Scalable Tree Boosting System.

Split Finding Algorithms

4) Sparsity-Aware Split Finding

초기 세팅

Input(I) : 현재 노드의 Instance Set

Input(I_K) = k , k feature의 non-missing data

Input(d) = feature dimension

Gain=0

G : x_i의 1차 미분합

H : x_i의 2차 미분합

Algorithm 3: Sparsity-aware Split Finding

Input: I, instance set of current node
Input: $I_k = \{i \in I | x_{ik} \neq \text{missing}\}$
Input: d, feature dimension
Also applies to the approximate setting, only collect statistics of non-missing entries into buckets
$gain \leftarrow 0$
$G \leftarrow \sum_{i \in I}, g_i, H \leftarrow \sum_{i \in I} h_i$
for $k = 1$ ***to*** m **do**
  // enumerate missing value goto right
  $G_L \leftarrow 0,\ H_L \leftarrow 0$
  for j *in sorted*(I_k, *ascent order by* $\mathbf{x}_{jk}$) **do**
    $G_L \leftarrow G_L + g_j,\ H_L \leftarrow H_L + h_j$
    $G_R \leftarrow G - G_L,\ H_R \leftarrow H - H_L$
    $score \leftarrow \max(score, \frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda})$
  end
  // enumerate missing value goto left
  $G_R \leftarrow 0,\ H_R \leftarrow 0$
  for j *in sorted*(I_k, *descent order by* $\mathbf{x}_{jk}$) **do**
    $G_R \leftarrow G_R + g_j,\ H_R \leftarrow H_R + h_j$
    $G_L \leftarrow G - G_R,\ H_L \leftarrow H - H_R$
    $score \leftarrow \max(score, \frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda})$
  end
end
Output: Split and default directions with max gain

XGboost, A Scalable Tree Boosting System.

Split Finding Algorithms

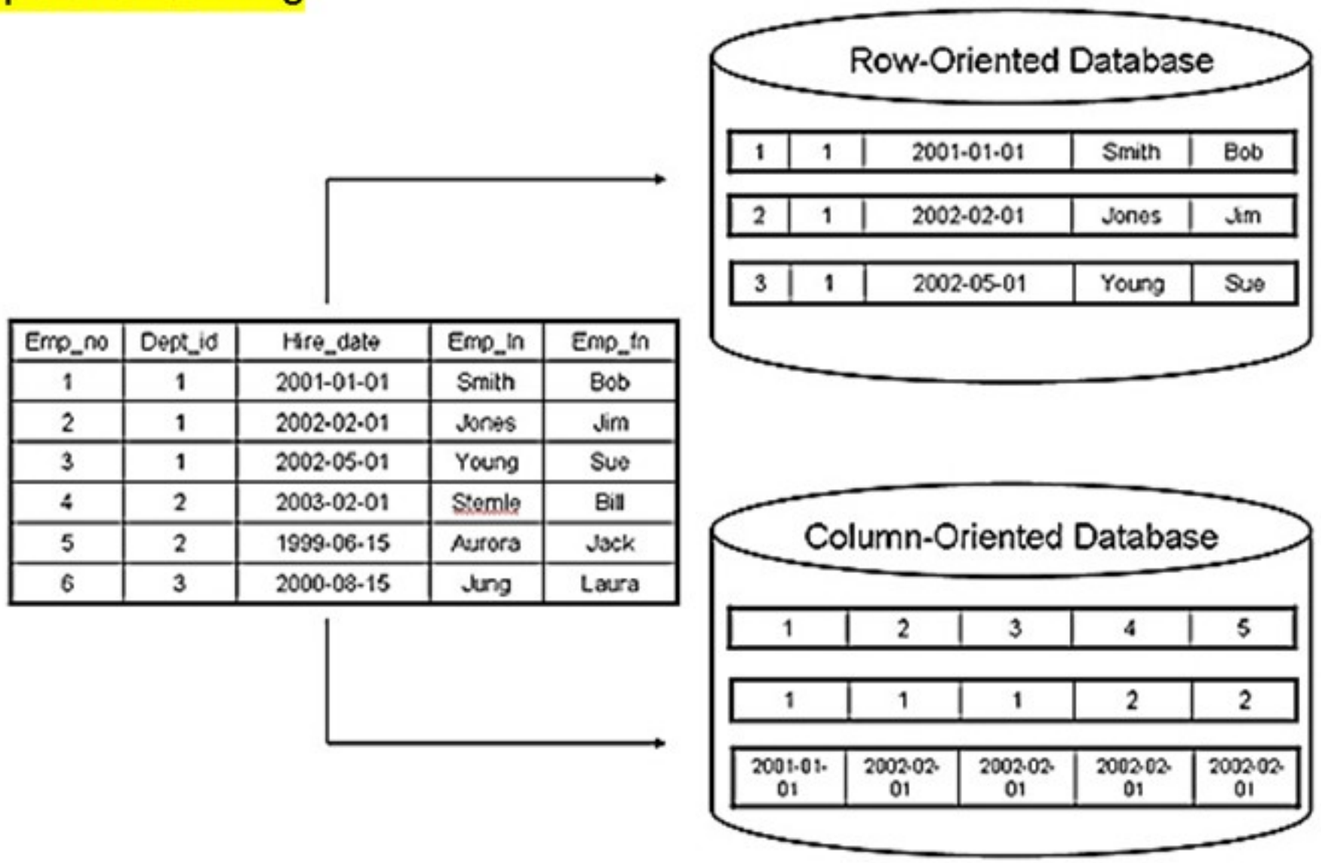
4) Sparsity-Aware Split Finding



XGboost, A Scalable Tree Boosting System.

System Design

1) Column block for parallel learning



Emp_no	Dept_id	Hire_date	Emp_ln	Emp_fn
1	1	2001-01-01	Smith	Bob
2	1	2002-02-01	Jones	Jim
3	1	2002-05-01	Young	Sue
4	2	2003-02-01	Stemle	Bill
5	2	1999-06-15	Aurora	Jack
6	3	2000-08-15	Jung	Laura

출처 : www.dbbest.com/blog/column-oriented-database-technologies

XGboost, A Scalable Tree Boosting System.

System Design

1) Column block for parallel learning

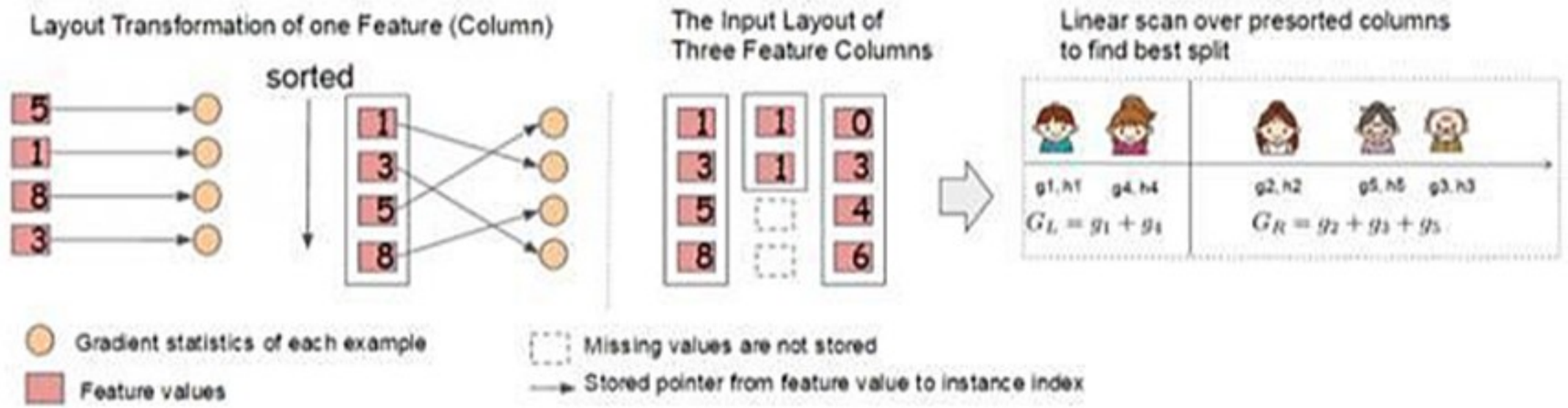
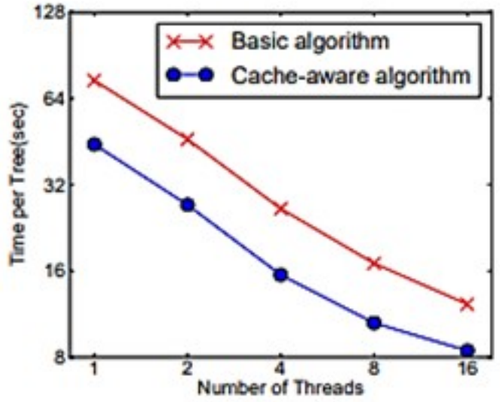


Figure 6: Block structure for parallel learning. Each column in a block is sorted by the corresponding feature value. A linear scan over one column in the block is sufficient to enumerate all the split points.

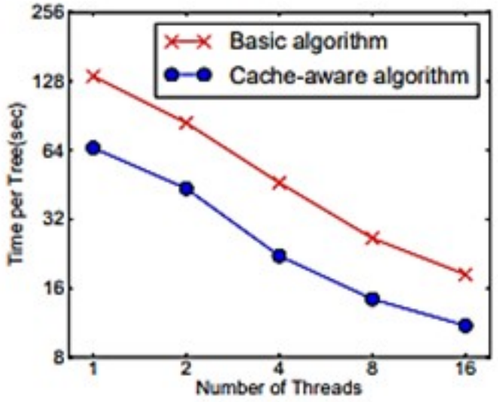
XGboost, A Scalable Tree Boosting System.

System Design

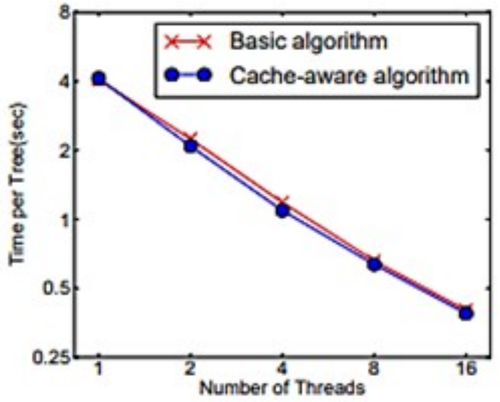
1) Column block for parallel learning



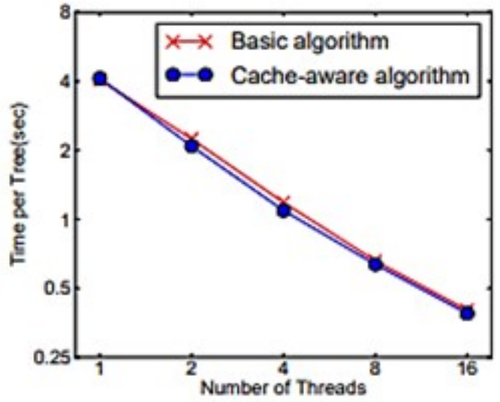
(a) Allstate 10M



(b) Higgs 10M



(c) Allstate 1M



(d) Higgs 1M

LightGBM, A highly Efficient Gradient Boosting Decision Tree

Gradient Boosting Decision Tree(GBDT)는 굉장히 유용한 머신러닝 알고리즘이며,
Xgboost 등을 효율적인 기법의 설계를 가능하게 해줌.
모든 가능한 분할점에 대해 정보 획득을 평가하기 위해 데이터 개체 전부를 스캔해야 하였기 때문에,
굉장히 시간이 많이 걸렸다.

그리하여 이 문제를 해결하기 위해 2가지 최신기술을 도입.

1. **GOSS(Gradient-based One-side Sampling)** : 기울기가 큰 데이터 개체가 정보 획득에 있어 더욱 큰 역할을 한다는 아이디어에 입각해 만들어진 테크닉.
   큰 기울기를 갖는 개체들은 유지되며, 작은 기울기를 갖는 데이터 개체들은 일정 확률에 의해 랜덤하게 제거

2. **EFB(Exclusive Feature Bundling)** : 변수 개수를 줄이기 위해 상호 배타적인 변수들을 묶는 기법.
   원 핫 인코딩된 변수와 같이 희소한(Sparse) 변수 공간에서는 많은 변수들이 상호 배타적인 경우가 많다
   (0이 굉장히 많기 때문에), 최적 묶음 문제를 그래프 색칠 문제로 치환하고 일정 근사 비율을 갖는
   Greedy 알고리즘으로 이 문제를 해결.

- GBDT는 Decision Tree의 앙상블 모델

각각의 반복에서 GBDT는 잔차 오차를 줄이면서 Decision Tree를 학습시키는데, 과정에서 가장 시간이 많이 소모되는 과정이 바로 최적의 분할점들을 찾는 것인데, 이를 대표적인 방법에는 Pre-Sorted(사전 정렬) 알고리즘과 Histogram-based 알고리즘이 있다.

Pre-sorted 알고리즘의 경우 사전 정렬한 변수 값에 대해 가능한 모든 분할점을 나열함으로써 간단하게 최적의 분할점을 찾을 수 있지만, 효율적이지 못하다는 단점이 있다. **Histogram-based 알고리즘**은 연속적인 변수 값을 이산적인 구간(bin)으로 나누고, 이 구간을 사용하여 학습과정 속에서 피쳐 히스토그램을 구성한다.

LightGBM, A highly Efficient Gradient Boosting Decision Tree

- GOSS(Gradient-based One-sided Sampling)

XGBoost

Algorithm 1: Histogram-based Algorithm

Input: I: training data, d: max depth
Input: m: feature dimension
$nodeSet \leftarrow \{0\}$ ▷ tree nodes in current level
$rowSet \leftarrow \{\{0, 1, 2, ...\}\}$ ▷ data indices in tree nodes
for $i = 1$ ***to*** d **do**
  for $node$ ***in*** $nodeSet$ **do**
    usedRows $\leftarrow rowSet[node]$
    for $k = 1$ ***to*** m **do**
      $H \leftarrow$ new Histogram()
      ▷ Build histogram
      for j ***in*** *usedRows* **do**
        bin $\leftarrow I$.f[k][j].bin
        H[bin].y $\leftarrow$ H[bin].y + I.y[j]
        H[bin].n $\leftarrow$ H[bin].n + 1
      Find the best split on histogram H.
      ...
  Update $rowSet$ and $nodeSet$ according to the best split points.
  ...

LightGBM

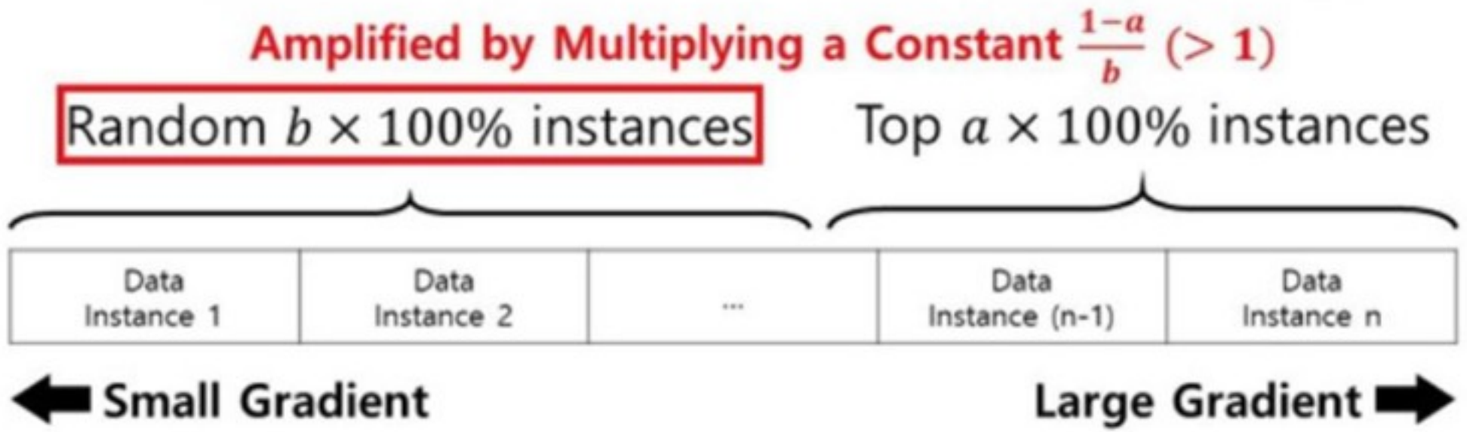
Algorithm 2: Gradient-based One-Side Sampling

Input: I: training data, d: iterations
Input: a: sampling ratio of large gradient data
Input: b: sampling ratio of small gradient data
Input: $loss$: loss function, L: weak learner
models $\leftarrow$ { }, fact $\leftarrow \frac{1-a}{b}$
topN $\leftarrow$ a $\times$ len(I) , randN $\leftarrow$ b $\times$ len(I)
for $i = 1$ ***to*** d **do**
  preds $\leftarrow$ models.predict(I)
  g $\leftarrow loss(I,$ preds), w $\leftarrow$ {1,1,...}
  sorted $\leftarrow$ GetSortedIndices(abs(g))
  topSet $\leftarrow$ sorted[1:topN]
  randSet $\leftarrow$ RandomPick(sorted[topN:len(I)], randN)
  usedSet $\leftarrow$ topSet + randSet
  w[randSet] $\times =$ fact ▷ Assign weight $fact$ to the small gradient data.
  newModel $\leftarrow$ L(I[usedSet], $-$ g[usedSet], w[usedSet])
  models.append(newModel)

출 처 : 강필성 교수님 강의

LightGBM, A highly Efficient Gradient Boosting Decision Tree

- GOSS(Gradient-based One-sided Sampling



https://cdm98.tistory.com/m/31

LightGBM, A highly Efficient Gradient Boosting Decision Tree

- Exclusive Feature Bundling (EFB)

Algorithm 3: Greedy Bundling

Input: F: features, K: max conflict count
Construct graph G
searchOrder $\leftarrow$ G.sortByDegree()
bundles $\leftarrow$ {}, bundlesConflict $\leftarrow$ {}
for i ***in*** *searchOrder* **do**
    needNew $\leftarrow$ True
    for $j = 1$ ***to*** *len(bundles)* **do**
        cnt $\leftarrow$ ConflictCnt(bundles[j],F[i])
        if *cnt + bundlesConflict[i]* $\leq K$ **then**
            bundles[j].add(F[i]), needNew $\leftarrow$ False
            break
    if *needNew* **then**
        Add $F[i]$ as a new bundle to *bundles*
Output: *bundles*

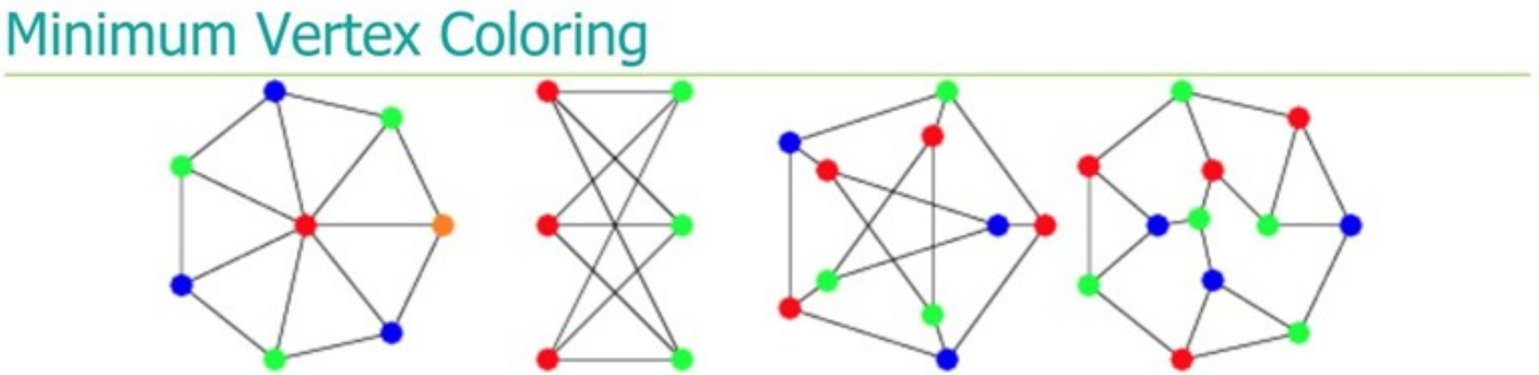
Algorithm 4: Merge Exclusive Features

Input: $numData$: number of data
Input: F: One bundle of exclusive features
binRanges $\leftarrow$ {0}, totalBin $\leftarrow$ 0
for f ***in*** F **do**
    totalBin += f.numBin
    binRanges.append(totalBin)
newBin $\leftarrow$ new Bin(numData)
for $i = 1$ ***to*** $numData$ **do**
    newBin[i] $\leftarrow$ 0
    for $j = 1$ ***to*** *len*(F) **do**
        if *F[j].bin[i]* $\neq 0$ **then**
            newBin[i] $\leftarrow$ F[j].bin[i] + binRanges[j]
Output: $newBin, binRanges$

출 처 : 강필성 교수님 강의

LightGBM, A highly Efficient Gradient Boosting Decision Tree

- Exclusive Feature Bundling (EFB)
  - ✓ Can be formulated as a Graph coloring problem
    - Construct a Graph (V, E)
      - V: feature
      - E: total conflicts between features



LightGBM, A highly Efficient Gradient Boosting Decision Tree

- Exclusive Feature Bundling (EFB)
  - ✓ Greedy bundling example

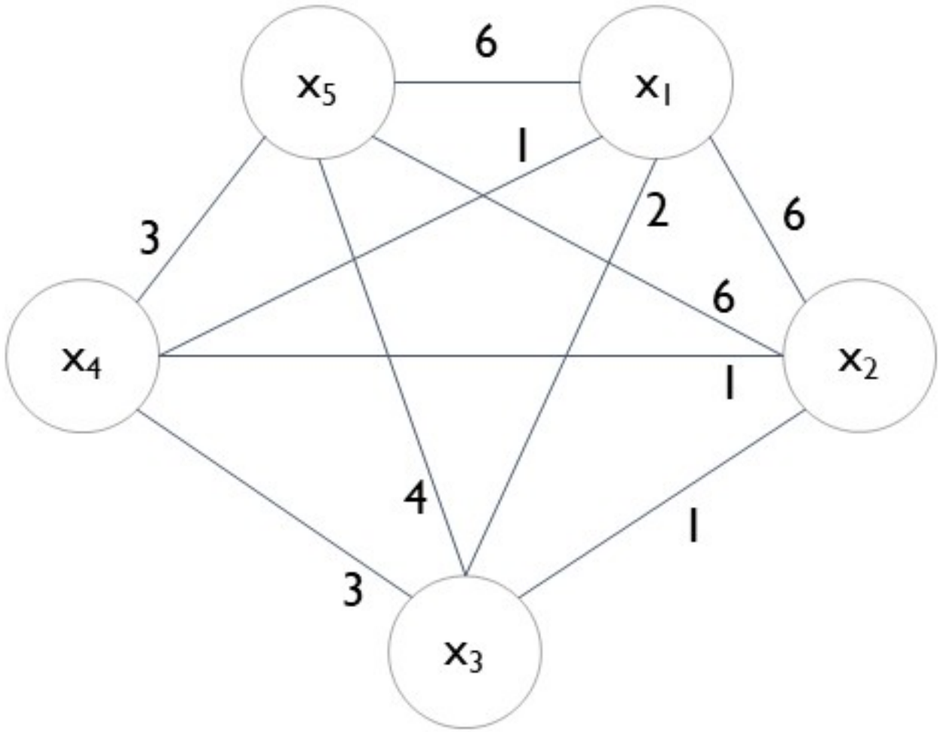
	x_1	x_2	x_3	x_4	x_5
l_1	1	1	0	0	1
l_2	0	0	1	1	1
l_3	1	2	0	0	2
l_4	0	0	2	3	1
l_5	2	1	0	0	3
l_6	3	3	0	0	1
l_7	0	0	3	0	2
l_8	1	2	3	4	3
l_9	1	0	1	0	0
l_{10}	2	3	0	0	2

	x_1	x_2	x_3	x_4	x_5
x_1	-	6	2	1	6
x_2	6	-	1	1	6
x_3	2	1	-	3	4
x_4	1	1	3	-	3
x_5	6	6	4	3	-

	x_5	x_1	x_2	x_3	x_4
d	19	15	14	10	8

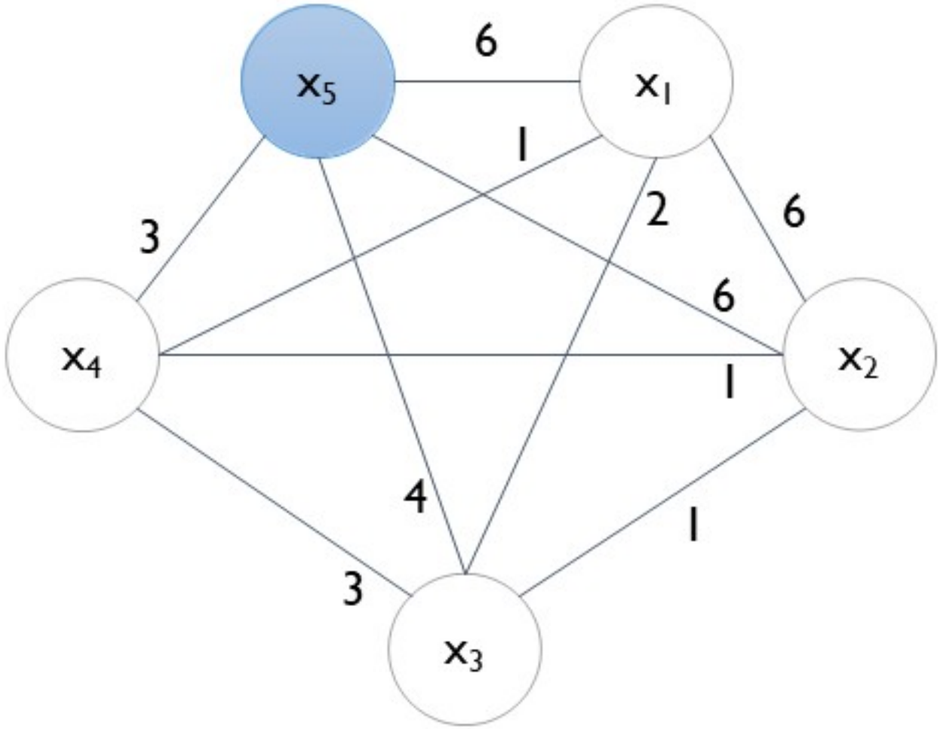
LightGBM, A highly Efficient Gradient Boosting Decision Tree

- Exclusive Feature Bundling (EFB)
  - ✓ Greedy bundling example (cut-off = 0.2)



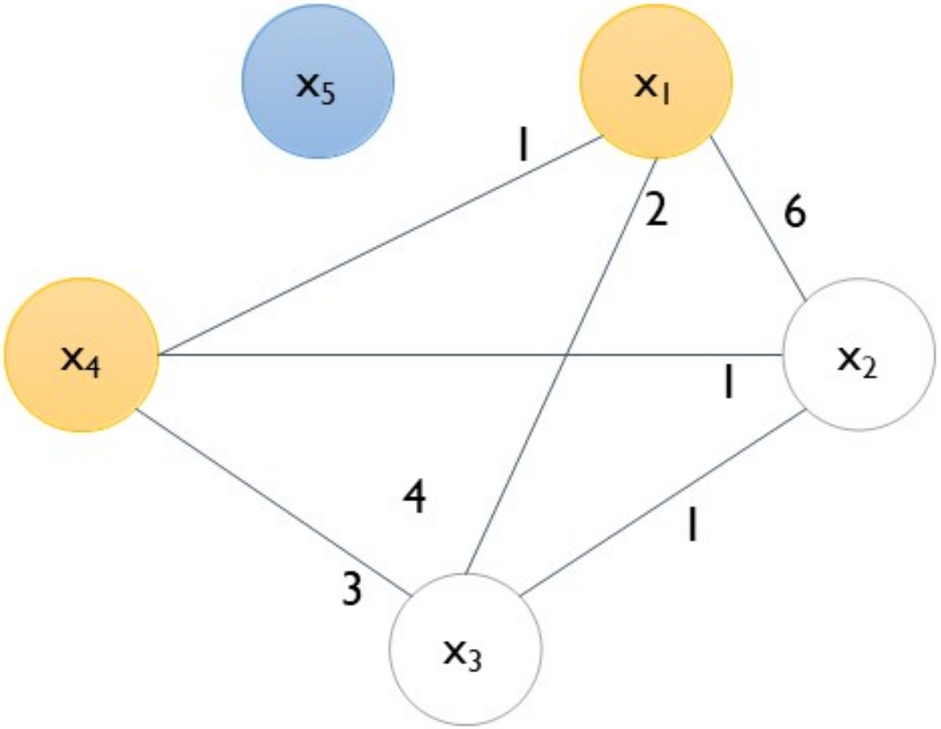
LightGBM, A highly Efficient Gradient Boosting Decision Tree

- Exclusive Feature Bundling (EFB)
  - ✓ Greedy bundling example (cut-off = 0.2)



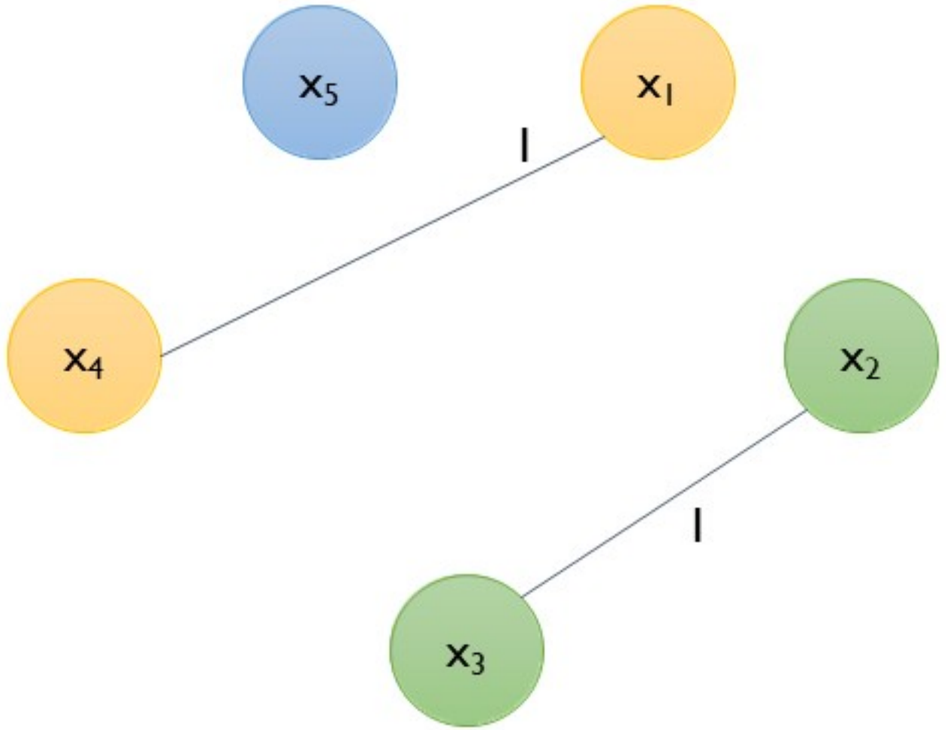
LightGBM, A highly Efficient Gradient Boosting Decision Tree

- Exclusive Feature Bundling (EFB)
  - ✓ Greedy bundling example (cut-off = 0.2)



LightGBM, A highly Efficient Gradient Boosting Decision Tree

- Exclusive Feature Bundling (EFB)
  - ✓ Greedy bundling example (cut-off = 0.2)



LightGBM, A highly Efficient Gradient Boosting Decision Tree

- Exclusive Feature Bundling (EFB)
  - ✓ Greedy bundling example (cut-off = 0.2)

	x_1	x_2	x_3	x_4	x_5
I_1	1	1	0	0	1
I_2	0	0	1	1	1
I_3	1	2	0	0	2
I_4	0	0	2	3	1
I_5	2	1	0	0	3
I_6	3	3	0	0	1
I_7	0	0	3	0	2
I_8	1	2	3	4	3
I_9	1	0	1	0	0
I_{10}	2	3	0	0	2

	x_5	x_1	x_4	x_2	x_3
I_1	1	1	0	1	0
I_2	1	0	1	0	1
I_3	2	1	0	2	0
I_4	1	0	3	0	2
I_5	3	2	0	1	0
I_6	1	3	0	3	0
I_7	2	0	0	0	3
I_8	3	1	4	2	3
I_9	0	1	0	0	1
I_{10}	2	2	0	3	0

LightGBM, A highly Efficient Gradient Boosting Decision Tree

- Exclusive Feature Bundling (EFB)
  - ✓ Exclusive feature merging
    - Add offsets to the original values of the features

	x_5	x_1	x_4	x_2	x_3
l_1	1	1	0	1	0
l_2	1	0	1	0	1
l_3	2	1	0	2	0
l_4	1	0	3	0	2
l_5	3	2	0	1	0
l_6	1	3	0	3	0
l_7	2	0	0	0	3
l_8	3	1	4	2	3
l_9	0	1	0	0	1
l_{10}	2	2	0	3	0

	x_1	x_2	x_3	x_4	x_5
x_1	-	6	2	1	6
x_2	6	-	1	1	6
x_3	2	1	-	3	4
x_4	1	1	3	-	3
x_5	6	6	4	3	-

	x_5	x_{14}	x_{23}
l_1	1	1	1
l_2	1	4	4
l_3	2	1	2
l_4	1	6	5
l_5	3	2	1
l_6	1	3	3
l_7	2	0	6
l_8	3	1	2
l_9	0	1	4
l_{10}	2	2	3

Add the offset 3 to the nonzero values of x_4

Add the offset 3 to the nonzero values of x_3

Conflict:
Use the value of x_1

Conflict:
Use the value of x_2

Catboost, Unbiased Boosting with Categorical Features

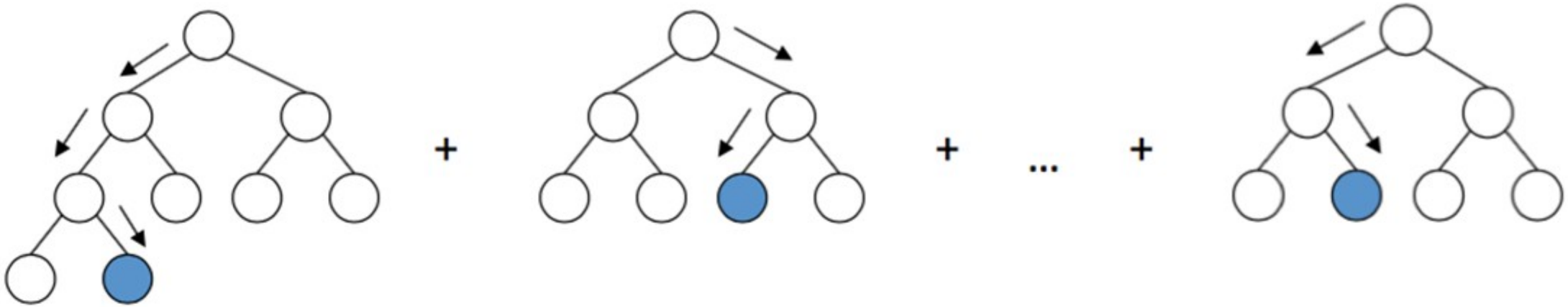


Yandex
CatBoost

Category + Gradient Boosting

Catboost, Unbiased Boosting with Categorical Features

- Gradient Boosting



CatBoost: Backgrounds

- Background in brief
  - ✓ Assume we observe a dataset of examples $\mathcal{D} = \{(\mathbf{x}_k, y_k)\}_{k=1,...,n}$
    - where $\mathbf{x}_k = (x_k^1, ..., x_k^m)$ is a random vector of m features and $y_k \in \mathbb{R}$ is a target
  - ✓ Gradient boosting builds iteratively a sequence of approximations $F^t : \mathbb{R}^m \to \mathbb{R}$ in a greedy fashion
    - $F^t : F^{t-1} + \alpha h^t$ (alpha is a step size and h is a base predictor)
    - h^t is chosen from a family of functions *H* in order to minimize the expected loss

$$h^t = \arg\min_{h \in H} \mathcal{L}(F^{t-1} + h) = \arg\min_{h \in H} \mathbb{E}L(y, F^{t-1}(\mathbf{x}) + h(\mathbf{x}))$$

    - The gradient step h^t is chosen in such a way that $h^t(\mathbf{x})$ approximates $-g^t(\mathbf{x}, y)$, where

$$g^t(\mathbf{x}, y) := \left.\frac{\partial L(y, s)}{\partial s}\right|_{s=F^{t-1}(\mathbf{x})}$$

    - Usually the least squares approximation is used

$$h^t = \arg\min_{h \in H} \mathbb{E}\big(- g^t(\mathbf{x}, y) - h(\mathbf{x})\big)^2$$

Catboost, Unbiased Boosting with Categorical Features

March, 2014 — XGBoost initially started as research project by Tianqi Chen but it actually became famous in 2016

Jan, 2017 — Microsoft released first stable version of LightGBM

April, 2017 — Yandex, one of Russia's leading tech companies open sources CatBoost

https://towardsdatascience.com/lightgbm-vs-xgboost-which-algorithm-win-the-race-1ff7dd4917d

Features

1  **Great quality without parameter tuning**

Reduce time spent on parameter tuning, because CatBoost provides great results with default parameters

2 

Categorical features support

Improve your training results with CatBoost that allows you to use non-numeric factors, instead of having to pre-process your data or spend time and effort turning it to numbers.

https://catboost.ai/

3  **Fast and scalable GPU version**

Train your model on a fast implementation of gradient-boosting algorithm for GPU. Use a multi-card configuration for large datasets.

4  **Improved accuracy**

Reduce overfitting when constructing your models with a novel gradient-boosting scheme.

5  **Fast prediction**

Apply your trained model quickly and efficiently even to latency-critical tasks using CatBoost's model applier

Catboost, Unbiased Boosting with Categorical Features

- Gradient Boosting

  ✓ Statistical Issue 1: Prediction Shift

$$\arg\min_{h \in H} \mathbb{E}\big(- g^t(\mathbf{x}, y) - h(\mathbf{x})\big)^2 \quad \approx \quad \arg\min_{h \in H} \frac{1}{n} \sum_{k=1}^{n} \big(- g^t(\mathbf{x}_k, y_k) - h(\mathbf{x}_k)\big)^2$$

$$\text{Training Dataset : } \mathcal{D} = (\mathbf{x}_k, y_k)_{k=1,\dots,n}$$

$$\text{where } \mathbf{x}_k = (x_k^1, \dots, x_k^m), \quad y_k \in \mathbb{R}$$

Conditional distribution

$F^{t-1}(\mathbf{x}_k)|\mathbf{x}_k$

for training example $\mathbf{x}_k$

is SHFTED from

$\neq$

Conditional distribution

$F^{t-1}(\mathbf{x})|\mathbf{x}$

for test example $\mathbf{x}$

출 처 : 강필성 교수님 강의

Catboost, Unbiased Boosting with Categorical Features

- Gradient Boosting
  - ✓ Statistical Issue 2: Target Leakage
    - Target Statistic (TS): Replace the categorical feature with it mean target value in the training dataset
    - (Problem) Target value of an instance is used to compute its feature value (target information is leaked)

	…	x^i	…	y
I_1	…	A	…	1
I_2	…	B	…	1
I_3	…	C	…	1
I_4	…	A	…	0
I_5	…	B	…	1
I_6	…	C	…	1
I_7	…	B	…	0
I_8	…	C	…	1
I_9	…	C	…	1
I_{10}	…	C	…	0



	…	x^i(TS)	…	y
I_1	…	0.50	…	1
I_2	…	0.67	…	1
I_3	…	0.80	…	1
I_4	…	0.50	…	0
I_5	…	0.67	…	1
I_6	…	0.80	…	1
I_7	…	0.67	…	0
I_8	…	0.80	…	1
I_9	…	0.80	…	1
I_{10}	…	0.80	…	0

Catboost, Unbiased Boosting with Categorical Features

- Solution



Target Leakage → Ordered Target Statistics

Prediction Shift → Ordered Boosting

Catboost, Unbiased Boosting with Categorical Features

- Categorical features in boosted tree
  - ✓ Basic approach:one-hot encoding

Ordered Target Statistics
->target leakage 방지

	…	x^i	…
I_1	…	A	…
I_2	…	B	…
I_3	…	C	…
I_4	…	A	…
I_5	…	B	…
I_6	…	C	…
I_7	…	B	…
I_8	…	C	…
I_9	…	C	…
I_{10}	…	C	…



	…	$x^i(A)$	$x^i(B)$	$x^i(C)$	…
I_1	…	1	0	0	…
I_2	…	0	1	0	…
I_3	…	0	0	1	…
I_4	…	1	0	0	…
I_5	…	0	1	0	…
I_6	…	0	0	1	…
I_7	…	0	1	0	…
I_8	…	0	0	1	…
I_9	…	0	0	1	…
I_{10}	…	0	0	1	…

출 처 : 강필성 교수님 강의

Catboost, Unbiased Boosting with Categorical Features

- Categorical features in boosted tree
  - ✓ Another popular method: to group categories by target statistics (TS)
    - an estimate expected target value in each category
    - Greedy TS without smoothing $\hat{x}_k^i \approx \mathbb{E}(y \mid x^i = x_k^i)$
    - i is a categorical feature while k is training example index

	…	x^i	…	y
I_1	…	A	…	1
I_2	…	B	…	1
I_3	…	C	…	1
I_4	…	A	…	0
I_5	…	B	…	1
I_6	…	C	…	1
I_7	…	B	…	0
I_8	…	C	…	1
I_9	…	C	…	1
I_{10}	…	C	…	0



	…	x^i(TS)	…	y
I_1	…	0.50	…	1
I_2	…	0.67	…	1
I_3	…	0.80	…	1
I_4	…	0.50	…	0
I_5	…	0.67	…	1
I_6	…	0.80	…	1
I_7	…	0.67	…	0
I_8	…	0.80	…	1
I_9	…	0.80	…	1
I_{10}	…	0.80	…	0

출 처 : 강필성 교수님 강의

Catboost, Unbiased Boosting with Categorical Features

- Categorical features in boosted tree
  - ✓ Another popular method:to group categories by target statistics (TS)
    - Greedy TS with smoothing

$$\hat{x}_k^i = \frac{\sum_{j=1}^{n} \mathbb{1}_{\{x_j^i = x_k^i\}} \cdot y_j + ap}{\sum_{j=1}^{n} \mathbb{1}_{\{x_j^i = x_k^i\}} + a}$$

- a > 0 is a parameter
- A common setting for p is the average target value in the dataset
- Used to remove the negative effect of low-frequency noisy categories

Catboost, Unbiased Boosting with Categorical Features

- Greedy TS with smoothing example
  - ✓ a = 0.1 (parameter), p = 0.7 (computed from the training dataset)
  - ✓ For categoryA

	…	x^i	…	y
I_1	…	A	…	I
I_2	…	B	…	I
I_3	…	C	…	I
I_4	…	A	…	0
I_5	…	B	…	I
I_6	…	C	…	I
I_7	…	B	…	0
I_8	…	C	…	I
I_9	…	C	…	0
I_{10}	…	C	…	I

$$\hat{x}_k^A = \frac{\sum_{j=1}^{n} \mathbb{1}_{\{x_j^A = x_k^A\}} \cdot y_j + ap}{\sum_{j=1}^{n} \mathbb{1}_{\{x_j^A = x_k^A\}} + a}$$

$$= \frac{1 + 0.1 \times 0.7}{2 + 0.1} = 0.5095$$

Catboost, Unbiased Boosting with Categorical Features

- Greedy TS with smoothing example
  - ✓ a = 0.1 (parameter) ,p = 0.7 (computed from the training dataset)
  - ✓ For category B

	…	x^i	…	y
I_1	…	A	…	1
I_2	…	B	…	1
I_3	…	C	…	1
I_4	…	A	…	0
I_5	…	B	…	1
I_6	…	C	…	1
I_7	…	B	…	0
I_8	…	C	…	1
I_9	…	C	…	0
I_{10}	…	C	…	1

$$\hat{x}_k^B = \frac{\sum_{j=1}^{n} \mathbb{1}_{\{x_j^B = x_k^B\}} \cdot y_j + ap}{\sum_{j=1}^{n} \mathbb{1}_{\{x_j^B = x_k^B\}} + a}$$

$$= \frac{2 + 0.1 \times 0.7}{3 + 0.1} = 0.6677$$

Catboost, Unbiased Boosting with Categorical Features

- Greedy TS with smoothing example
  - ✓ a = 0.1 (parameter) ,p = 0.7 (computed from the training dataset)
  - ✓ For category C

	…	x^i	…	y
I_1	…	A	…	1
I_2	…	B	…	1
I_3	…	C	…	1
I_4	…	A	…	0
I_5	…	B	…	1
I_6	…	C	…	1
I_7	…	B	…	0
I_8	…	C	…	1
I_9	…	C	…	0
I_{10}	…	C	…	1

$$\hat{x}_k^C = \frac{\sum_{j=1}^{n} \mathbb{1}_{\{x_j^C = x_k^C\}} \cdot y_j + ap}{\sum_{j=1}^{n} \mathbb{1}_{\{x_j^C = x_k^C\}} + a}$$

$$= \frac{4 + 0.1 \times 0.7}{5 + 0.1} = 0.7980$$

Catboost, Unbiased Boosting with Categorical Features

- Greedy TS with smoothing example
  - ✓ a = 0.1 (parameter) ,p = 0.7 (computed from the training dataset)

	…	x^i	…	y
I_1	…	A	…	1
I_2	…	B	…	1
I_3	…	C	…	1
I_4	…	A	…	0
I_5	…	B	…	1
I_6	…	C	…	1
I_7	…	B	…	0
I_8	…	C	…	1
I_9	…	C	…	0
I_{10}	…	C	…	1

	…	x^i(TS)	…	y
I_1	…	0.5095	…	1
I_2	…	0.6677	…	1
I_3	…	0.7980	…	1
I_4	…	0.5095	…	0
I_5	…	0.6677	…	1
I_6	…	0.7980	…	1
I_7	…	0.6677	…	0
I_8	…	0.7980	…	1
I_9	…	0.7980	…	0
I_{10}	…	0.7980	…	1

Catboost, Unbiased Boosting with Categorical Features

- Target leakage
  - ✓ feature $\hat{x}_k^i$ is computed using y_k , the target of $\mathbf{x}_k$
  - ✓ leads to a conditional shift: the distribution of $\hat{x}^i|y$ differs for training and test examples

We should not use this information when we predict the target of current example

$$\hat{x}_k^i = \frac{\sum_{j=1}^{n} \mathbb{1}_{\{x_j^i = x_k^i\}} \cdot y_j + ap}{\sum_{j=1}^{n} \mathbb{1}_{\{x_j^i = x_k^i\}} + a}$$

Catboost, Unbiased Boosting with Categorical Features

- **Target leakage**
  - ✓ feature $\hat{x}_k^i$ is computed using y_k , the target of $\mathbf{x}_k$
  - ✓ leads to a conditional shift: the distribution of $\hat{x}^i|y$ differs for training and test examples
    - Assume that i^{th} feature is categorical, all its values are unique, $P(y=1|x^i)=0.5, x^i \in \{A, B, C, D, E, F, ...\}$

	…	x^i	…	y
I_1	…	A	…	I
I_2	…	B	…	I
I_3	…	C	…	I
I_4	…	D	…	0
I_5	…	E	…	0
I_6	…	F	…	0

	…	x^i	…	y
I_1	…	$\frac{1+ap}{1+a}$	…	I
I_2	…	$\frac{1+ap}{1+a}$	…	I
I_3	…	$\frac{1+ap}{1+a}$	…	I
I_4	…	$\frac{0+ap}{1+a}$	…	0
I_5	…	$\frac{0+ap}{1+a}$	…	0
I_5	…	$\frac{0+ap}{1+a}$	…	0

- Training data can be perfectly classified when the split criterion is set to $x^i = \frac{0.5+ap}{1+a}$

Catboost, Unbiased Boosting with Categorical Features

- Target leakage

		x^i		y
I_1	…	A	…	I
I_2	…	B	…	I
I_3	…	C	…	I
I_4	…	D	…	0
I_5	…	E	…	0
I_6	…	F	…	0
I_7	…	G	…	I
I_8	…	H	…	I
I_9	…	I	…	0
I_{10}	…	J	…	0

Training Set

Test Set

		x^i		y
I_1	…	$\frac{1+ap}{1+a}$	…	I
I_2	…	$\frac{1+ap}{1+a}$	…	I
I_3	…	$\frac{1+ap}{1+a}$	…	I
I_4	…	$\frac{0+ap}{1+a}$	…	0
I_5	…	$\frac{0+ap}{1+a}$	…	0
I_6	…	$\frac{0+ap}{1+a}$	…	0
I_7	…	p	…	I
I_8	…	p	…	I
I_9	…	p	…	0
I_{10}	…	p	…	0

Perfect classification by setting the cut off value

$$x^i = \frac{0.5 + ap}{1 + a}$$

$$\hat{x}_k^i = \frac{\sum_{j=1}^{n} \mathbb{1}_{\{x_j^i = x_k^i\}} \cdot y_j \; + ap}{\sum_{j=1}^{n} \mathbb{1}_{\{x_j^i = x_k^i\}} \; + \; a}$$

→ All feature values of test instances become p → No discrimination ability

Catboost, Unbiased Boosting with Categorical Features

- Desired property for TS
  - ✓ Property 1:

$$\mathbb{E}(\hat{x}^i \mid y = v) = \mathbb{E}(\hat{x}^i_k \mid y_k = v)$$

$$where\ (\mathbf{x}_k, y_k)\ is\ the\ k-th\ training\ example$$

  - ✓ Property 2:
    - Effective usage of all training data for calculating TS features and for learning a model

Catboost, Unbiased Boosting with Categorical Features

- Ways to avoid conditional shift
  - ✓ General idea: compute the TS for $\mathbf{x}_k$ on a subset of examples $\mathcal{D}_k \subset \mathcal{D} \backslash \{\mathbf{x}_k\}$ excluding $\mathbf{x}_k$

$$\hat{x}_k^i = \frac{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} \cdot y_j + ap}{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} + a}$$

  - ✓ Holdout TS
    - To partition the training dataset into two part $\mathcal{D} = \hat{\mathcal{D}}_0 \cup \hat{\mathcal{D}}_1$
      - Use the former to compute the TS
      - Use the latter for training
      - Violates the Property 2

Catboost, Unbiased Boosting with Categorical Features

- Ways to avoid conditional shift
  - ✓ Leave-one-outTS
    - Take $\mathcal{D}_k = \mathcal{D} \backslash \mathbf{x}_k$ for training examples
    - Take $\mathcal{D}_k = \mathcal{D}$ for test ones
    - Does not prevent target leakage

$$\hat{x}_k^i = \frac{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} \cdot y_j + ap}{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} + a}$$

	…	x^i	…	y
I_1	…	A	…	I
I_2	…	A	…	0
I_3	…	A	…	I
I_4	…	A	…	0
I_5	…	A	…	I

	…	x^i	…	y
I_1	…	$\frac{2+ap}{4+a}$	…	I
I_2	…	$\frac{3+ap}{4+a}$	…	0
I_3	…	$\frac{2+ap}{4+a}$	…	I
I_4	…	$\frac{3+ap}{4+a}$	…	0
I_5	…	$\frac{2+ap}{4+a}$	…	I

- We can perfectly classify this training dataset by setting the cut-off value $\frac{2.5+ap}{4+a}$

Catboost, Unbiased Boosting with Categorical Features

- Ways to avoid conditional shift
  - ✓ OrderedTS: introduce an artificial time
    - a random permutation σ of the training examples

$$\mathcal{D}_k = \{\mathbf{x}_j : \sigma(j) < \sigma(k)\}$$

    - a = 0.1 (parameter) , p = 0 (computed from the training dataset)

	…	x^i	…	TS	y
I_1	…	A	…	0.000	1
I_2	…	B	…		1
I_3	…	C	…		1
I_4	…	A	…		0
I_5	…	B	…		1
I_6	…	C	…		1
I_7	…	B	…		0
I_8	…	C	…		1
I_9	…	C	…		0
I_{10}	…	C	…		1

$$\hat{x}_k^i = \frac{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} \cdot y_j \; + ap}{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} \; + \; a}$$

$$= \frac{0 \; + \; 0.1 \times 0}{0 \; + \; 0.1} \; = \; 0$$

Catboost, Unbiased Boosting with Categorical Features

- Ways to avoid conditional shift
  - ✓ Ordered TS: introduce an artificial time
    - a random permutation σ of the training examples

$$\mathcal{D}_k = \{\mathbf{x}_j : \sigma(j) < \sigma(k)\}$$

    - a = 0.1 (parameter) , p = 1 (computed from the training dataset)

	…	x^i	…	TS	y
I_1	…	A	…	0.000	1
I_2	…	B	…	1.000	1
I_3	…	C	…		1
I_4	…	A	…		0
I_5	…	B	…		1
I_6	…	C	…		1
I_7	…	B	…		0
I_8	…	C	…		1
I_9	…	C	…		0
I_{10}	…	C	…		1

$$\hat{x}_k^i = \frac{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} \cdot y_j + ap}{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} + a}$$

$$= \frac{0 + 0.1 \times 1.0}{0 + 0.1} = 1.0$$

Catboost, Unbiased Boosting with Categorical Features

- Ways to avoid conditional shift
  - ✓ Ordered TS: introduce an artificial time
    - a random permutation σ of the training examples

$$\mathcal{D}_k = \{\mathbf{x}_j : \sigma(j) < \sigma(k)\}$$

    - a = 0.1 (parameter) , p = 1 (computed from the training dataset)

	…	x^i	…	TS	y
I_1	…	A	…	0.000	1
I_2	…	B	…	1.000	1
I_3	…	C	…	1.000	1
I_4	…	A	…		0
I_5	…	B	…		1
I_6	…	C	…		1
I_7	…	B	…		0
I_8	…	C	…		1
I_9	…	C	…		0
I_{10}	…	C	…		1

$$\hat{x}_k^i = \frac{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} \cdot y_j + ap}{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} + a}$$

$$= \frac{0 + 0.1 \times 1}{0 + 0.1} = 1.0$$

Catboost, Unbiased Boosting with Categorical Features

- Ways to avoid conditional shift
  - ✓ OrderedTS: introduce an artificial time
    - a random permutation σ of the training examples

$$\mathcal{D}_k = \{\mathbf{x}_j : \sigma(j) < \sigma(k)\}$$

    - a = 0.1 (parameter) , p = 0.75 (computed from the training dataset)

	…	x^i	…	TS	y
I_1	…	A	…	0.000	1
I_2	…	B	…	1.000	1
I_3	…	C	…	1.000	1
I_4	…	A	…	1.000	0
I_5	…	B	…	0.977	1
I_6	…	C	…		1
I_7	…	B	…		0
I_8	…	C	…		1
I_9	…	C	…		0
I_{10}	…	C	…		1

$$\hat{x}_k^i = \frac{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} \cdot y_j \; + ap}{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} \; + \; a}$$

$$= \frac{1 \; + \; 0.1 \times 0.75}{1 \; + \; 0.1} \; = \; 0.977$$

Catboost, Unbiased Boosting with Categorical Features

- Ways to avoid conditional shift
  - ✓ Ordered TS: introduce an artificial time
    - a random permutation σ of the training examples

$$\mathcal{D}_k = \{\mathbf{x}_j : \sigma(j) < \sigma(k)\}$$

    - a = 0.1 (parameter) , p = 0.8 (computed from the training dataset)

	…	x^i	…	TS	y
I_1	…	A	…	0.000	1
I_2	…	B	…	1.000	1
I_3	…	C	…	1.000	1
I_4	…	A	…	1.000	0
I_5	…	B	…	0.977	1
I_6	…	C	…	0.982	1
I_7	…	B	…		0
I_8	…	C	…		1
I_9	…	C	…		0
I_{10}	…	C	…		1

$$\hat{x}_k^i = \frac{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} \cdot y_j + ap}{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} + a}$$

$$= \frac{1 + 0.1 \times 0.8}{1 + 0.1} = 0.982$$

Catboost, Unbiased Boosting with Categorical Features

- Ways to avoid conditional shift
  - ✓ Ordered TS: introduce an artificial time
    - a random permutation σ of the training examples

$$\mathcal{D}_k = \{\mathbf{x}_j : \sigma(j) < \sigma(k)\}$$

    - a = 0.1 (parameter) , p = 0.833 (computed from the training dataset)

	…	x^i	…	TS	y
I_1	…	A	…	0.000	1
I_2	…	B	…	1.000	1
I_3	…	C	…	1.000	1
I_4	…	A	…	1.000	0
I_5	…	B	…	0.977	1
I_6	…	C	…	0.982	1
I_7	…	B	…	0.992	0
I_8	…	C	…		1
I_9	…	C	…		0
I_{10}	…	C	…		1

$$\hat{x}_k^i = \frac{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} \cdot y_j \; + ap}{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} \; + \; a}$$

$$= \frac{2 \; + \; 0.1 \times 0.833}{2 \; + \; 0.1} \; = \; 0.992$$

Catboost, Unbiased Boosting with Categorical Features

- Ways to avoid conditional shift
  - ✓ Ordered TS: introduce an artificial time
    - a random permutation σ of the training examples

$$\mathcal{D}_k = \{\mathbf{x}_j : \sigma(j) < \sigma(k)\}$$

    - a = 0.1 (parameter) , p = 0.714 (computed from the training dataset)

	…	x^i	…	TS	y
I_1	…	A	…	0.000	1
I_2	…	B	…	1.000	1
I_3	…	C	…	1.000	1
I_4	…	A	…	1.000	0
I_5	…	B	…	0.977	1
I_6	…	C	…	0.982	1
I_7	…	B	…	0.992	0
I_8	…	C	…	0.986	1
I_9	…	C	…		0
I_{10}	…	C	…		1

$$\hat{x}_k^i = \frac{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} \cdot y_j + ap}{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} + a}$$

$$= \frac{2 + 0.1 \times 0.714}{2 + 0.1} = 0.986$$

Catboost, Unbiased Boosting with Categorical Features

- Ways to avoid conditional shift
  - ✓ OrderedTS: introduce an artificial time
    - a random permutation σ of the training examples

$$\mathcal{D}_k = \{\mathbf{x}_j : \sigma(j) < \sigma(k)\}$$

    - a = 0.1 (parameter) , p = 0.75 (computed from the training dataset)

	…	x^i	…	TS	y
I_1	…	A	…	0.000	1
I_2	…	B	…	1.000	1
I_3	…	C	…	1.000	1
I_4	…	A	…	1.000	0
I_5	…	B	…	0.977	1
I_6	…	C	…	0.982	1
I_7	…	B	…	0.992	0
I_8	…	C	…	0.986	1
I_9	…	C	…	0.992	0
I_{10}	…	C	…		1

$$\hat{x}_k^i = \frac{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} \cdot y_j \; + ap}{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} \; + \; a}$$

$$= \frac{3 \; + \; 0.1 \times 0.75}{3 \; + \; 0.1} \; = \; 0.992$$

Catboost, Unbiased Boosting with Categorical Features

- Ways to avoid conditional shift
  - ✓ Ordered TS: introduce an artificial time
    - a random permutation σ of the training examples

$$\mathcal{D}_k = \{\mathbf{x}_j : \sigma(j) < \sigma(k)\}$$

    - a = 0.1 (parameter) , p = 0.667 (computed from the training dataset)

	…	x^i	…	TS	y
I_1	…	A	…	0.000	1
I_2	…	B	…	1.000	1
I_3	…	C	…	1.000	1
I_4	…	A	…	1.000	0
I_5	…	B	…	0.977	1
I_6	…	C	…	0.982	1
I_7	…	B	…	0.992	0
I_8	…	C	…	0.986	1
I_9	…	C	…	0.992	0
I_{10}	…	C	…	0.748	1

$$\hat{x}_k^i = \frac{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} \cdot y_j + ap}{\sum_{\mathbf{x}_j \in \mathcal{D}_k} \mathbb{1}_{\{x_j^i = x_k^i\}} + a}$$

$$= \frac{3 + 0.1 \times 0.667}{4 + 0.1} = 0.748$$

Catboost, Unbiased Boosting with Categorical Features

- Prediction Shift

$$h^t = \arg\min_{h \in H} \; \mathbb{E}\big(- g^t(\mathbf{x}, y) - h(\mathbf{x})\big)^2$$

✓ The expectation is unknown and is usually approximated using the same dataset $\mathcal{D}$

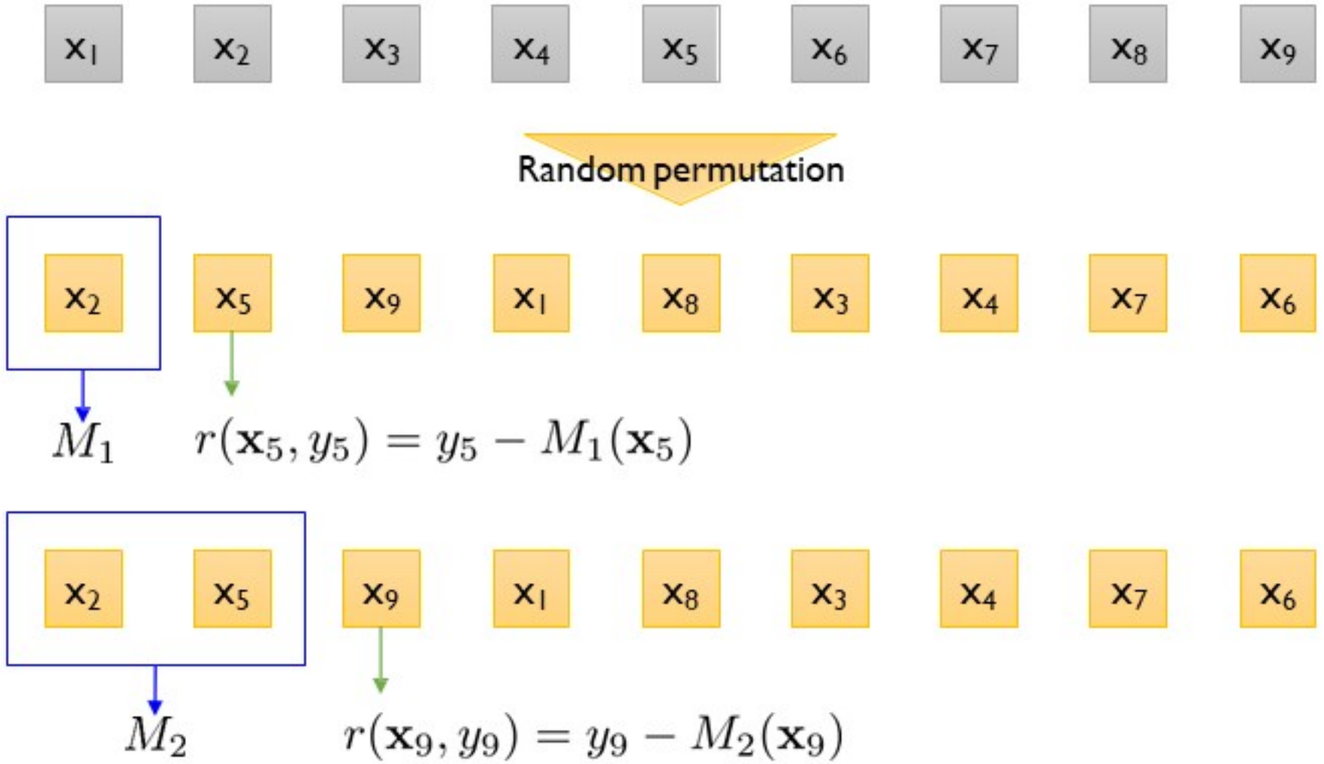
$$h^t = \arg\min_{h \in H} \; \frac{1}{n} \sum_{k=1}^{n} \big(- g^t(\mathbf{x}, y) - h(\mathbf{x})\big)^2$$

- The conditional distribution of the gradient $g^t(\mathbf{x}_k, y_k)|\mathbf{x}_k$ is shifted from that distribution on a test example $g^t(\mathbf{x}, y)|\mathbf{x}$
- In turn, base predictor h^t is biased from the solution
- Finally it affects the generalization ability of the trained model F^t

✓ When we learn a model with I trees, we need to have F^{I-1} trained without the example $\mathbf{x}_k$ to make the residual $r^{I-1}(\mathbf{x}_k, y_k)$ unshifted

Catboost, Unbiased Boosting with Categorical Features

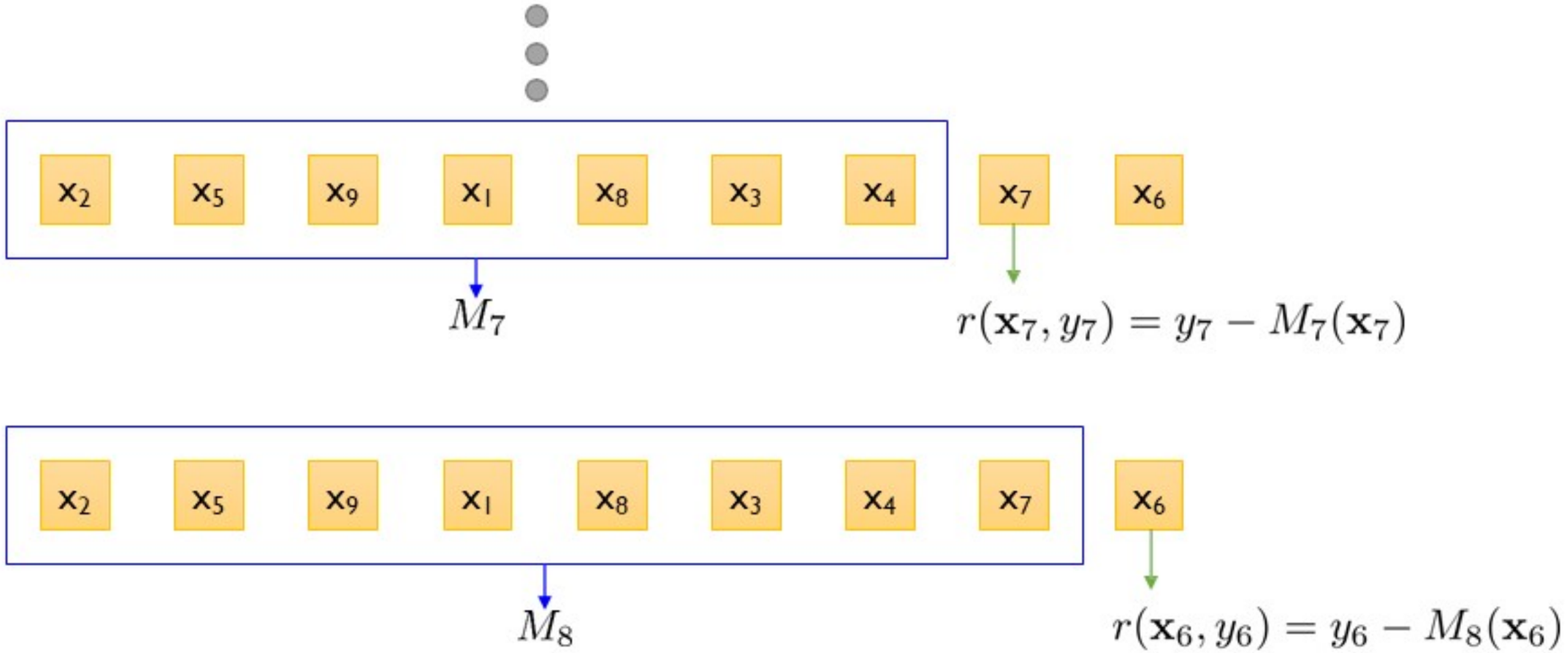
- Ordered Boosting
  - ✓ A boosting algorithm not suffering from the prediction shift problem



출 처 : 강필성 교수님 강의

Catboost, Unbiased Boosting with Categorical Features

- Ordered Boosting
  - ✓ A boosting algorithm not suffering from the prediction shift problem



  - ✓ The same permutation is used for both TS computing and ordered boosting

Catboost, Unbiased Boosting with Categorical Features

- Ordered Boosting and Building a Tree in CatBoost

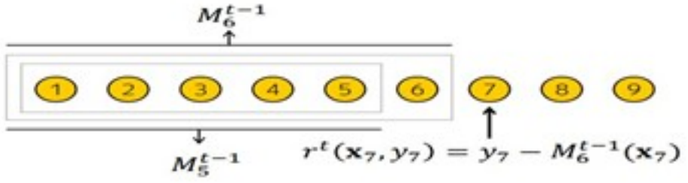


Figure 1: Ordered boosting principle, examples are ordered according to σ.

Algorithm 1: Ordered boosting

input : $\{(\mathbf{x}_k, y_k)\}_{k=1}^n$, I;
$\sigma \leftarrow$ random permutation of $[1, n]$;
$M_i \leftarrow 0$ for $i = 1..n$;
for $t \leftarrow 1$ **to** I **do**
  for $i \leftarrow 1$ **to** n **do**
    $r_i \leftarrow y_i - M_{\sigma(i)-1}(\mathbf{x}_i)$;
  for $i \leftarrow 1$ **to** n **do**
    $\Delta M \leftarrow$ $LearnModel((\mathbf{x}_j, r_j) : \sigma(j) \leq i)$;
    $M_i \leftarrow M_i + \Delta M$;
return M_n

Algorithm 2: Building a tree in CatBoost

input : M, $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, α, L, $\{\sigma_i\}_{i=1}^s$, $Mode$
$grad \leftarrow CalcGradient(L, M, y)$;
$r \leftarrow random(1, s)$;
if $Mode = Plain$ **then**
  $G \leftarrow (grad_r(i)$ for $i = 1..n)$;
if $Mode = Ordered$ **then**
  $G \leftarrow (grad_{r,\sigma_r(i)-1}(i)$ for $i = 1..n)$;
$T \leftarrow$ empty tree;
foreach *step of top-down procedure* **do**
  foreach *candidate split* c **do**
    $T_c \leftarrow$ add split c to T;
    if $Mode = Plain$ **then**
      $\Delta(i) \leftarrow \text{avg}(grad_r(p)$ for $p : leaf_r(p) = leaf_r(i))$ for $i = 1..n$;
    if $Mode = Ordered$ **then**
      $\Delta(i) \leftarrow \text{avg}(grad_{r,\sigma_r(i)-1}(p)$ for $p : leaf_r(p) = leaf_r(i), \sigma_r(p) < \sigma_r(i))$ for $i = 1..n$;
    $loss(T_c) \leftarrow \cos(\Delta, G)$
  $T \leftarrow \arg\min_{T_c}(loss(T_c))$
if $Mode = Plain$ **then**
  $M_{r'}(i) \leftarrow M_{r'}(i) - \alpha\,\text{avg}(grad_{r'}(p)$ for $p : leaf_{r'}(p) = leaf_{r'}(i))$ for $r' = 1..s$, $i = 1..n$;
if $Mode = Ordered$ **then**
  $M_{r',j}(i) \leftarrow M_{r',j}(i) - \alpha\,\text{avg}(grad_{r',j}(p)$ for $p : leaf_{r'}(p) = leaf_{r'}(i), \sigma_{r'}(p) \leq j)$ for $r' = 1..s$, $i = 1..n$, $j \geq \sigma_{r'}(i) - 1$;
return T, M

출 처 : 강필성 교수님 강의

Catboost, Unbiased Boosting with Categorical Features

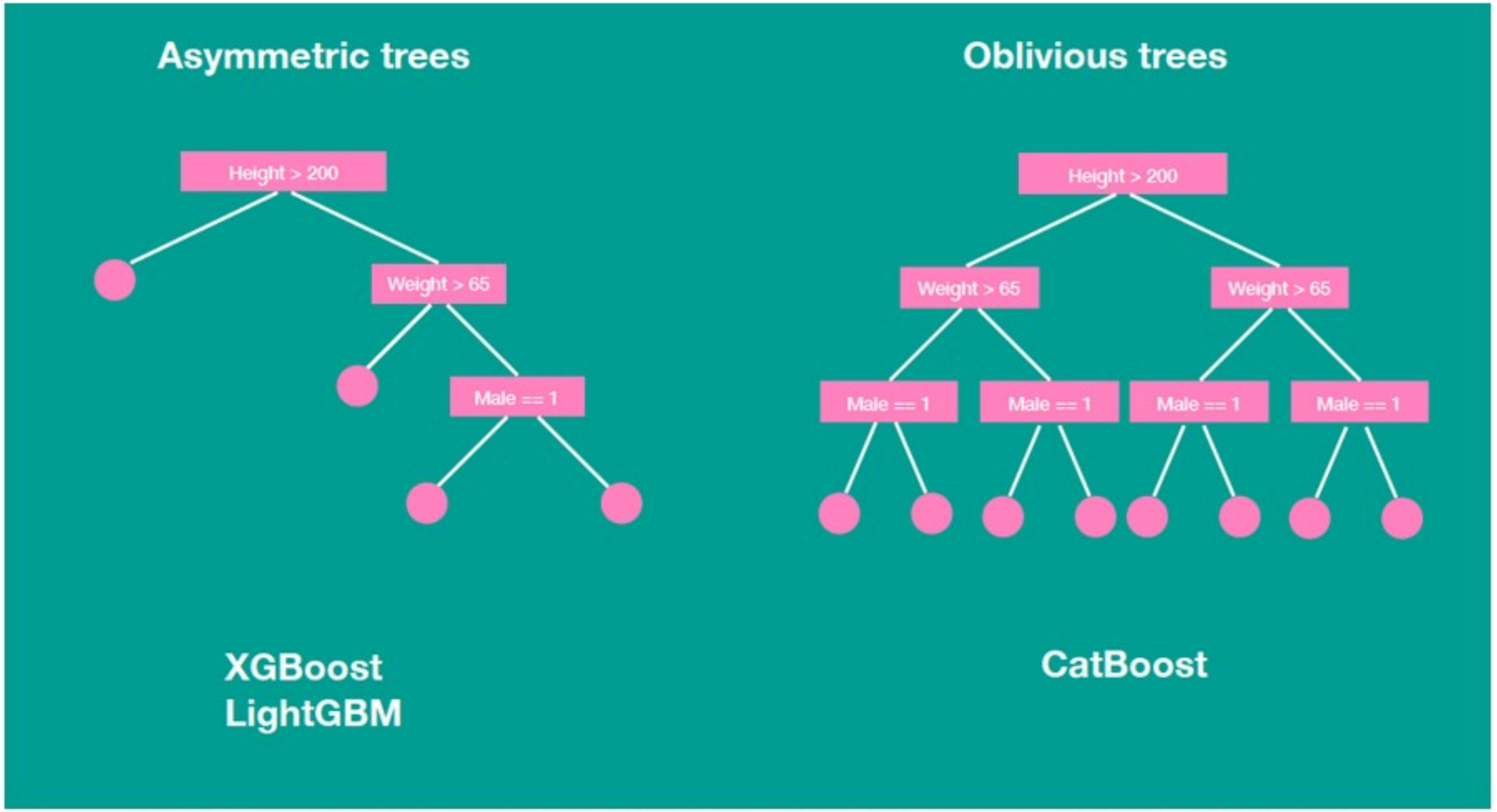
- CatBoost Procedure
  - ✓ Initialization: generate (s+1) independent random permutations of the training dataset
    - s permutations to evaluate the split
    - 1 permutation to compute the leaf value of the obtained trees
  - ✓ Both TS and predictions used by ordered boosting have a high variance
  - ✓ Using only one permutation may increase the variance of the final model predictions, while several permutations can reduce it

Catboost, Unbiased Boosting with Categorical Features

- CatBoost Procedure
  - ✓ Building an oblivious tree the same splitting criterion is used across an entire level of the tree in the ordered boosting mode
    - Maintain the supporting models $M_{r,j}$
    - $M_{r,j}(i)$: the current prediction for the ith example based on the first j examples in the permutation r
    - At each iteration t, sample a random permutation σ_r and construct a tree T_t
      - TS are computed according to this permutation σ_r
      - The permutation affects the tree learning procedure
    - For each example i, take the gradient $grad_{r,\sigma_r(i)-1}(i)$
    - While constructing a tree, the loss of the candidate split c is computed by the cosine similarity between the leaf value and the gradient of each example
      - At the candidate splits evaluation step, the leaf value $\Delta(i)$ for example i is obtained individually by averaging the gradients $grad_{r,\sigma_r(i)-1}$ of the preceding example p lying in the same leaf node of i

Catboost, Unbiased Boosting with Categorical Features

- Oblivious tree



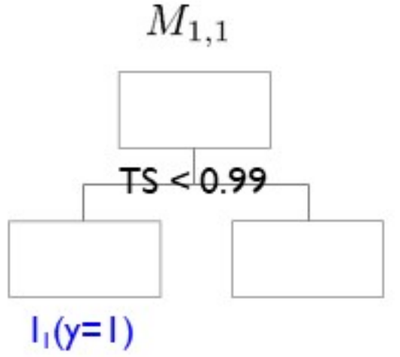
https://towardsdatascience.com/introduction-to-gradient-boosting-on-decision-trees-with-catboost-d511a9ccbd14

출 처 : 강필성 교수님 강의

Catboost, Unbiased Boosting with Categorical Features

- Ordered Boosting Example
  - ✓ Assumption: squared loss (gradient: $f(x) - y$)
  - ✓ random permutation is conducted

	…	x^i	…	TS	y	G	Δ
I_1	…	A	…	0.000	1	0	0
I_2	…	B	…	1.000	1		
I_3	…	C	…	1.000	1		
I_4	…	A	…	1.000	0		
I_5	…	B	…	0.977	1		
I_6	…	C	…	0.982	1		
I_7	…	B	…	0.992	0		
I_8	…	C	…	0.986	1		
I_9	…	C	…	0.992	0		
I_{10}	…	C	…	0.748	1		



$G(I_1) = 0$

$\Delta(I_1) = 0$

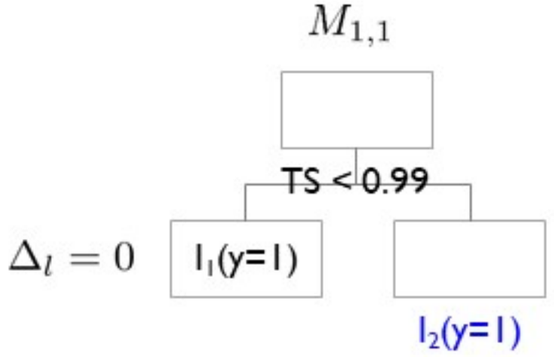
Lecturer's guess

- Initialize both the first gradient and leaf value to 0

Catboost, Unbiased Boosting with Categorical Features

- Ordered Boosting Example
  - ✓ Assumption: squared loss (gradient: $f(x) - y$)
  - ✓ random permutation is conducted

	…	x^i	…	TS	y	G	Δ
I_1	…	A	…	0.000	1	0	0
I_2	…	B	…	1.000	1	0	0
I_3	…	C	…	1.000	1		
I_4	…	A	…	1.000	0		
I_5	…	B	…	0.977	1		
I_6	…	C	…	0.982	1		
I_7	…	B	…	0.992	0		
I_8	…	C	…	0.986	1		
I_9	…	C	…	0.992	0		
I_{10}	…	C	…	0.748	1		



$$G(I_2) = 0$$

$$\Delta(I_2) = 0$$

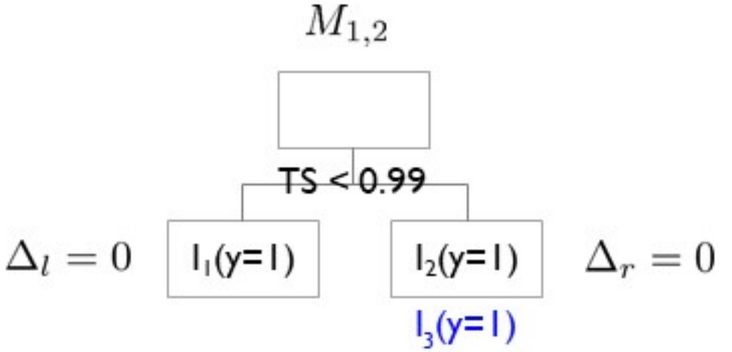
Lecturer's guess

- Initialize both the first gradient and leaf value to 0

Catboost, Unbiased Boosting with Categorical Features

- Ordered Boosting Example
  - ✓ Assumption: squared loss (gradient: $f(x) - y$)
  - ✓ random permutation is conducted

	…	x^i	…	TS	y	G	Δ
I_1	…	A	…	0.000	1	0	0
I_2	…	B	…	1.000	1	0	0
I_3	…	C	…	1.000	1	-1	0
I_4	…	A	…	1.000	0		
I_5	…	B	…	0.977	1		
I_6	…	C	…	0.982	1		
I_7	…	B	…	0.992	0		
I_8	…	C	…	0.986	1		
I_9	…	C	…	0.992	0		
I_{10}	…	C	…	0.748	1		



$$G(I_3) = grad_{1,\sigma_1(3)-1}(I_3)$$
$$= \Delta_r - y_3 = 0 - 1 = -1$$

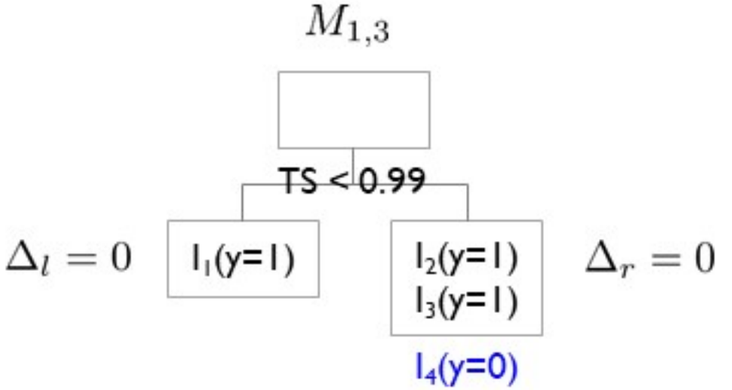
$$\Delta(I_3) = grad_{1,\sigma_1(3)-1}(I_2) = 0$$

출 처 : 강필성 교수님 강의

Catboost, Unbiased Boosting with Categorical Features

- Ordered Boosting Example
  - ✓ Assumption: squared loss (gradient: $f(x) - y$)
  - ✓ random permutation is conducted

	…	x^i	…	TS	y	G	Δ
I_1	…	A	…	0.000	1	0	0
I_2	…	B	…	1.000	1	0	0
I_3	…	C	…	1.000	1	-1	0
I_4	…	A	…	1.000	0	0	-0.5
I_5	…	B	…	0.977	1		
I_6	…	C	…	0.982	1		
I_7	…	B	…	0.992	0		
I_8	…	C	…	0.986	1		
I_9	…	C	…	0.992	0		
I_{10}	…	C	…	0.748	1		



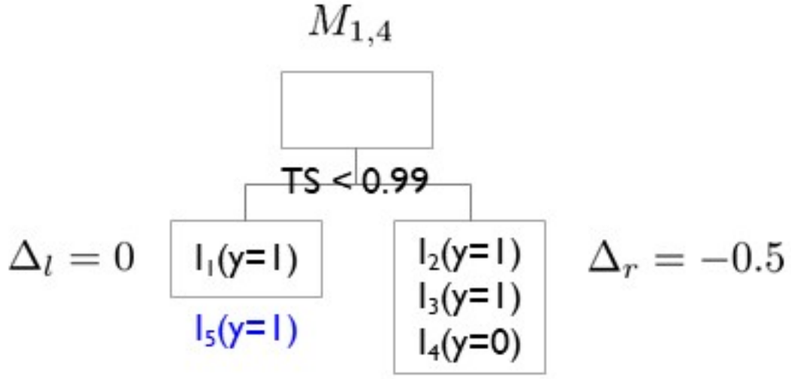
$$G(I_4) = grad_{1,\sigma_1(4)-1}(I_4)$$
$$= \Delta_r - y_4 = 0 - 0 = 0$$

$$\Delta(I_4) = \frac{1}{2}(grad_{1,\sigma_1(5)-1}(I_2) + grad_{1,\sigma_1(5)-1}(I_3))$$
$$= \frac{1}{2}(0 - 1) = -0.5$$

Catboost, Unbiased Boosting with Categorical Features

- Ordered Boosting Example
  - ✓ Assumption: squared loss (gradient: $f(x) - y$)
  - ✓ random permutation is conducted

	…	x^i	…	TS	y	G	Δ
I_1	…	A	…	0.000	1	0	0
I_2	…	B	…	1.000	1	0	0
I_3	…	C	…	1.000	1	-1	0
I_4	…	A	…	1.000	0	0	-0.5
I_5	…	B	…	0.977	1	-1	0
I_6	…	C	…	0.982	1		
I_7	…	B	…	0.992	0		
I_8	…	C	…	0.986	1		
I_9	…	C	…	0.992	0		
I_{10}	…	C	…	0.748	1		



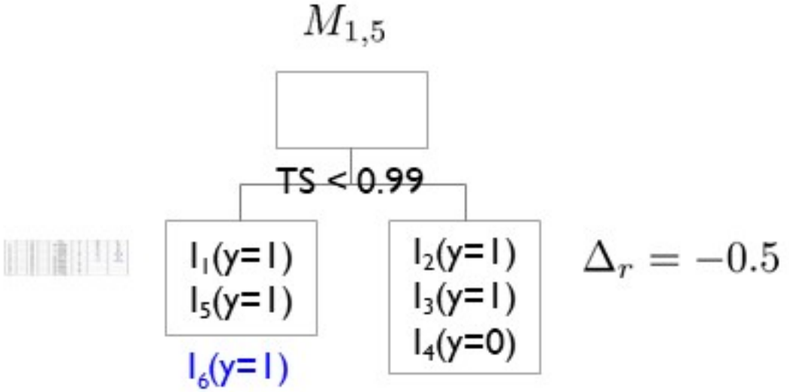
$$G(I_5) = grad_{1,\sigma_1(5)-1}(I_5)$$
$$= \Delta_l - y_5 = 0 - 1 = -1$$

$$\Delta(I_5) = grad_{1,\sigma_1(5)-1}(I_1)$$
$$= 0$$

Catboost, Unbiased Boosting with Categorical Features

- Ordered Boosting Example
  - ✓ Assumption: squared loss (gradient: $f(x) - y$)
  - ✓ random permutation is conducted

	…	x^i	…	TS	y	G	Δ
I_1	…	A	…	0.000	1	0	0
I_2	…	B	…	1.000	1	0	0
I_3	…	C	…	1.000	1	-1	0
I_4	…	A	…	1.000	0	0	-0.5
I_5	…	B	…	0.977	1	-1	0
I_6	…	C	…	0.982	1	-1	-0.5
I_7	…	B	…	0.992	0		
I_8	…	C	…	0.986	1		
I_9	…	C	…	0.992	0		
I_{10}	…	C	…	0.748	1		



$$G(I_6) = grad_{1,\sigma_1(6)-1}(I_6)$$

$$= \Delta_l - y_6 = 0 - 1 = -1$$

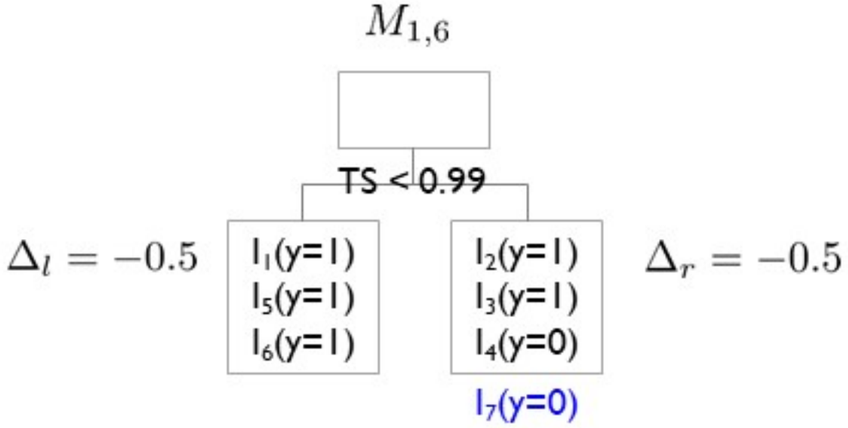
$$\Delta(I_6) = \frac{1}{2}(grad_{1,\sigma_1(6)-1}(I_1) + grad_{1,\sigma_1(6)-1}(I_5))$$

$$= \frac{1}{2}(0 - 1) = -0.5$$

Catboost, Unbiased Boosting with Categorical Features

- Ordered Boosting Example
  - ✓ Assumption: squared loss (gradient: $f(x) - y$)
  - ✓ random permutation is conducted

	…	x^i	…	TS	y	G	Δ
I_1	…	A	…	0.000	1	0	0
I_2	…	B	…	1.000	1	0	0
I_3	…	C	…	1.000	1	-1	0
I_4	…	A	…	1.000	0	0	-0.5
I_5	…	B	…	0.977	1	-1	0
I_6	…	C	…	0.982	1	-1	-0.5
I_7	…	B	…	0.992	0	-0.5	-0.33
I_8	…	C	…	0.986	1		
I_9	…	C	…	0.992	0		
I_{10}	…	C	…	0.748	1		



$$G(I_7) = grad_{1,\sigma_1(7)-1}(I_7)$$

$$= \Delta_r - y_7 = -0.5 - 0 = -0.5$$

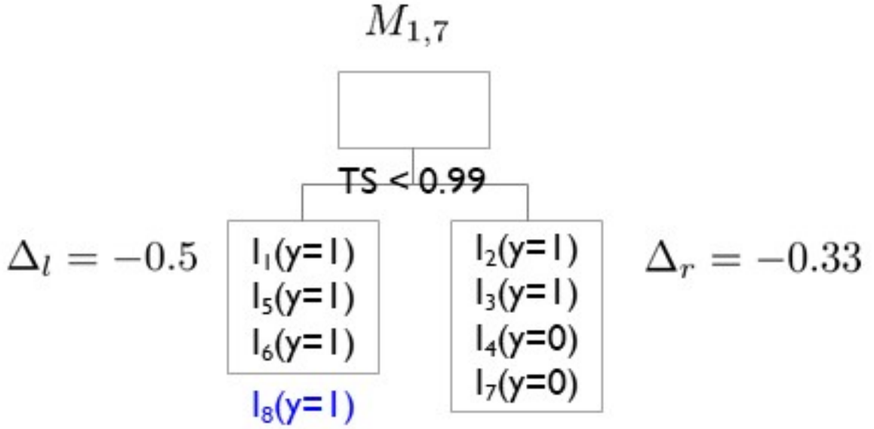
$$\Delta(I_7) = \frac{1}{3}\big(grad_{1,\sigma_1(7)-1}(I_2) + grad_{1,\sigma_1(7)-1}(I_3) + grad_{1,\sigma_1(7)-1}(I_4)\big)$$

$$= \frac{1}{3}(0 - 1 - 0) = -0.33$$

Catboost, Unbiased Boosting with Categorical Features

- Ordered Boosting Example
  - ✓ Assumption: squared loss (gradient: $f(x) - y$)
  - ✓ random permutation is conducted

	…	x^i	…	TS	y	G	Δ
I_1	…	A	…	0.000	1	0	0
I_2	…	B	…	1.000	1	0	0
I_3	…	C	…	1.000	1	-1	0
I_4	…	A	…	1.000	0	0	-0.5
I_5	…	B	…	0.977	1	-1	0
I_6	…	C	…	0.982	1	-1	-0.5
I_7	…	B	…	0.992	0	-0.5	-0.33
I_8	…	C	…	0.986	1	-1.5	-0.67
I_9	…	C	…	0.992	0		
I_{10}	…	C	…	0.748	1		



$$G(I_8) = grad_{1,\sigma_1(8)-1}(I_8)$$
$$= \Delta_l - y_8 = -0.5 - 1 = -1.5$$

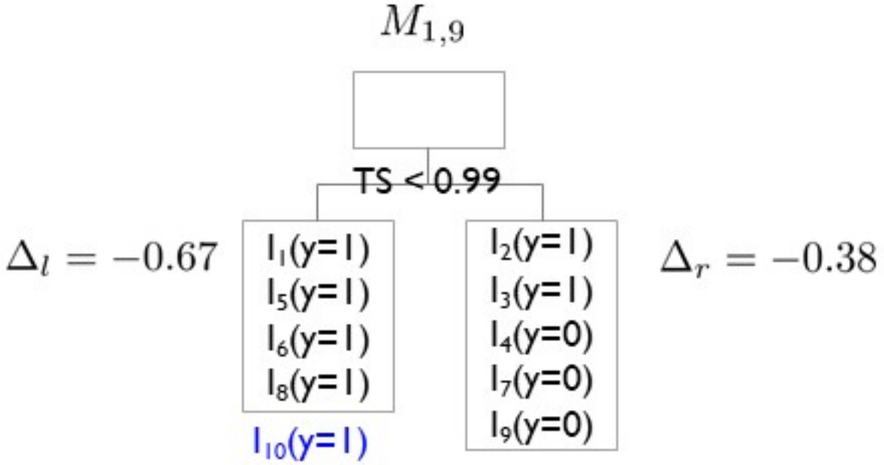
$$\Delta(I_8) = \frac{1}{3}(grad_{1,\sigma_1(8)-1}(I_1) + grad_{1,\sigma_1(8)-1}(I_5) + grad_{1,\sigma_1(8)-1}(I_6))$$
$$= \frac{1}{3}(0 - 1 - 1) = -0.67$$

출 처 : 강필성 교수님 강의

Catboost, Unbiased Boosting with Categorical Features

- Ordered Boosting Example
  - ✓ Assumption: squared loss (gradient: $f(x) - y$)
  - ✓ random permutation is conducted

	…	x^i	…	TS	y	G	Δ
I_1	…	A	…	0.000	1	0	0
I_2	…	B	…	1.000	1	0	0
I_3	…	C	…	1.000	1	-1	0
I_4	…	A	…	1.000	0	0	-0.5
I_5	…	B	…	0.977	1	-1	0
I_6	…	C	…	0.982	1	-1	-0.5
I_7	…	B	…	0.992	0	-0.5	-0.33
I_8	…	C	…	0.986	1	-1.5	-0.67
I_9	…	C	…	0.992	0	-0.33	-0.38
I_{10}	…	C	…	0.748	1	-1.67	-0.58



$$G(I_{10}) = grad_{1,\sigma_1(10)-1}(I_{10})$$

$$= \Delta_l - y_{10} = -0.67 - 1 = -1.67$$

$$\Delta(I_9) = \frac{1}{4}\big(grad_{1,\sigma_1(10)-1}(I_1) + grad_{1,\sigma_1(10)-1}(I_5)$$

$$+ grad_{1,\sigma_1(10)-1}(I_6) + grad_{1,\sigma_1(10)-1}(I_8)\big)$$

$$= \frac{1}{4}(0 - 1 - 1 - 0.33) = -0.58$$

Catboost, Unbiased Boosting with Categorical Features

- Ordered Boosting Example
  - ✓ Assumption: squared loss (gradient: $f(x) - y$)
  - ✓ random permutation is conducted

	…	x^i	…	TS	y	G	Δ
I_1	…	A	…	0.000	1	0	0
I_2	…	B	…	1.000	1	0	0
I_3	…	C	…	1.000	1	-1	0
I_4	…	A	…	1.000	0	0	-0.5
I_5	…	B	…	0.977	1	-1	0
I_6	…	C	…	0.982	1	-1	-0.5
I_7	…	B	…	0.992	0	-0.5	-0.33
I_8	…	C	…	0.986	1	-1.5	-0.67
I_9	…	C	…	0.992	0	-0.33	-0.38
I_{10}	…	C	…	0.748	1	-1.67	-0.58

foreach *step of top-down procedure* **do**
  foreach *candidate split* c **do**
    $T_c \leftarrow$ add split c to T;
    if $Mode = Plain$ **then**
      $\Delta(i) \leftarrow \text{avg}(grad_r(p)$ for $p : leaf_r(p) = leaf_r(i))$ for $i = 1..n$;
    if $Mode = Ordered$ **then**
      $\Delta(i) \leftarrow \text{avg}(grad_{r,\sigma_r(i)-1}(p)$ for $p : leaf_r(p) = leaf_r(i), \sigma_r(p) < \sigma_r(i))$ for $i = 1..n$;
    $loss(T_c) \leftarrow \cos(\Delta, G)$
  $T \leftarrow \arg\min_{T_c}(loss(T_c))$

$$loss(T_c) = cos(\Delta, G) = 0.3575$$

(TS < 0.99)

Catboost, Unbiased Boosting with Categorical Features

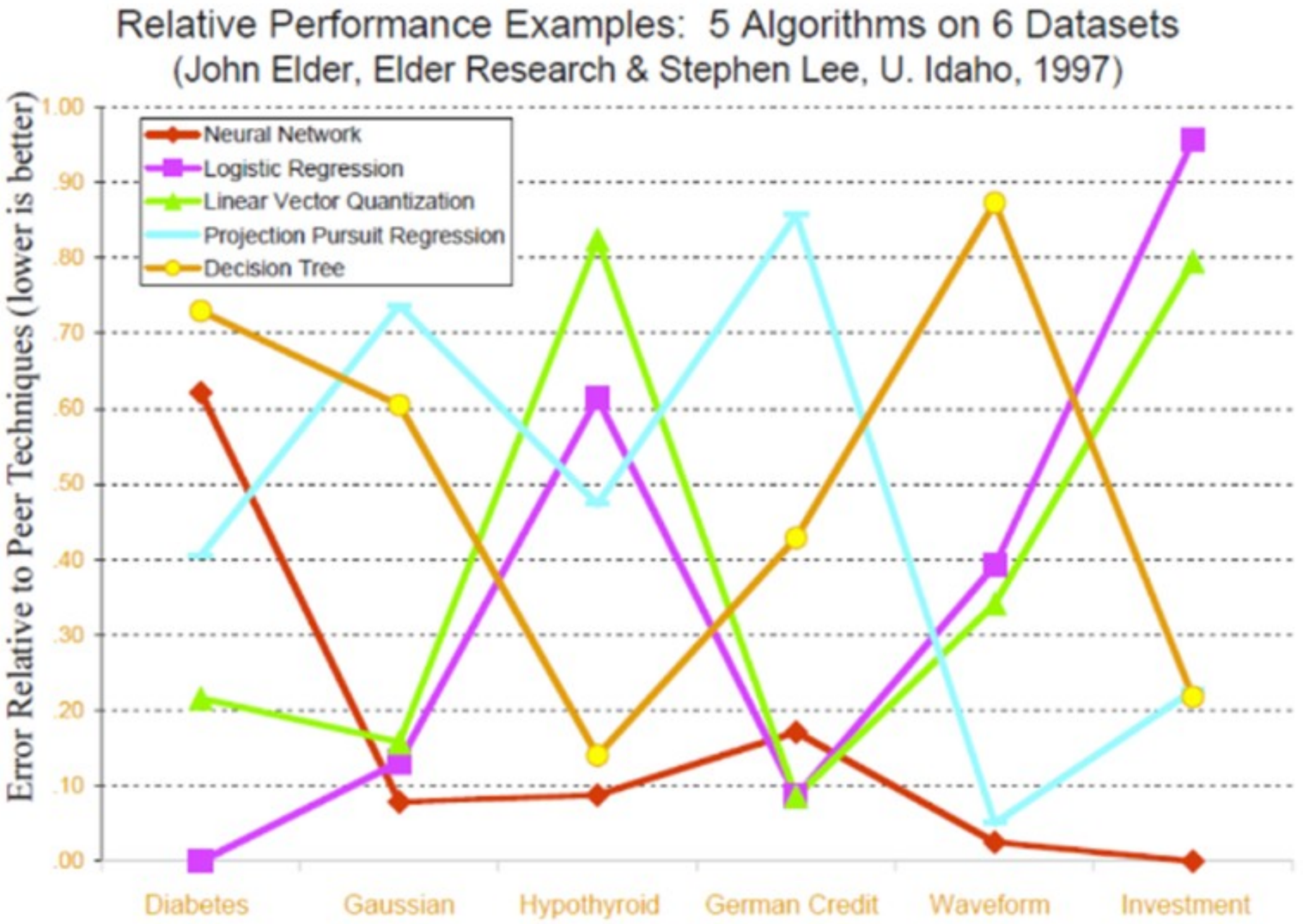
- Choosing leaf values
  - ✓ Given all the trees constructed, the leaf values of the final model F are calculated by the standard gradient boosting procedure
  - ✓ Training examples i are matched to leaves $leaf_0(i)$ using permutation σ_0 to calculate TS
  - ✓ When the final model F is applied to a new example at testing time, TS is calculated on the whole training data

7. Ensemble

- Bagging
- Boosting
- Voting
- Stacking



7. Ensemble 의 배경

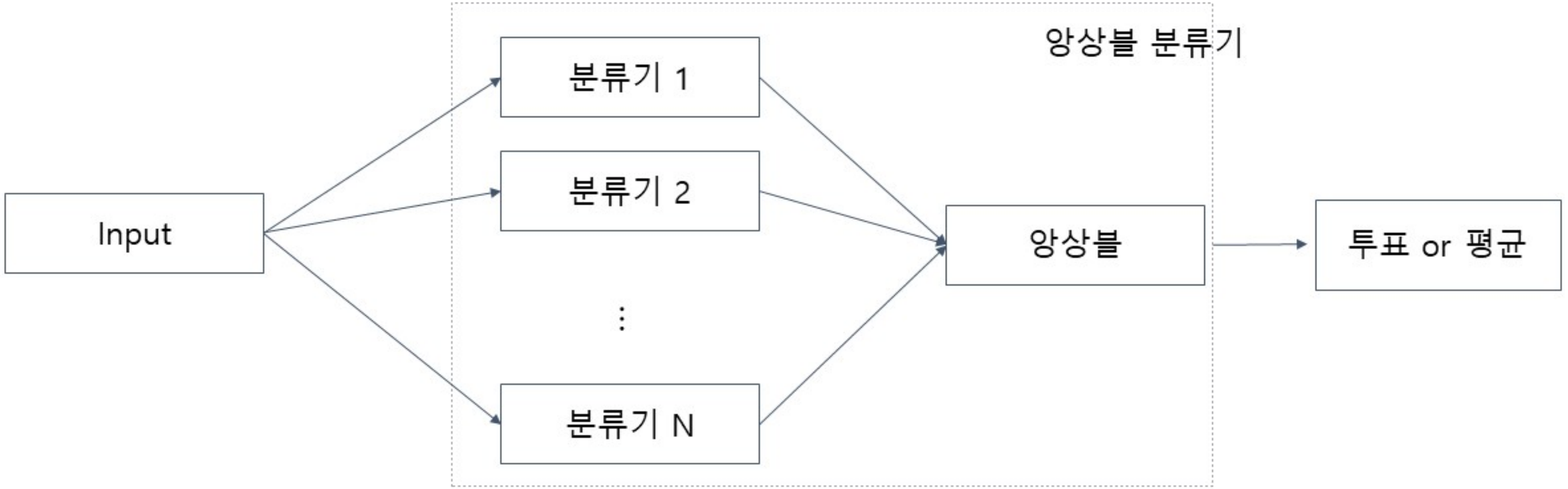


Seni and Elder (2010)

7. Ensemble 이란?

- 여러 개의 결정 트리(Decision Tree)를 결합하여 하나의 결정 트리보다 더 좋은 성능을 내는 머신러닝 기법
- 앙상블 학습의 핵심은 여러 개의 여러 개의 약 분류기(Weak Classifier)를 결합하여 강 분류기(Strong Classifier) 만드는 과정.
- 여러 개의 단일 모델들의 평균치를 내거나, 투표를 해서 다수결에 의한 결정을 하는 등 여러 모델들의 집단 지성을 활용하여 더 나은 결과를 도출해 내는 것에 주 목적이 있음.

7. Ensemble 이란?



장점 : 1. 성능을 분산시키기 때문에 과적합(Overfitting) 감소 효과
2. 개별 모델 성능이 잘 안나올 때 앙상블 학습을 이용하면 성능이 향상될 수 있음.

7. Ensemble 의 기법

1. Bagging - Boostrap Aggregation(샘플을 다양하게 생성)
2. Voting(투표) - 투표를 통해 결과 도출
3. Boosting - 이전 오차를 보완 하며 가중치 부여
4. Stacking - 여러 모델을 기반으로 meta 모델

7. Ensemble의 기법_Voting

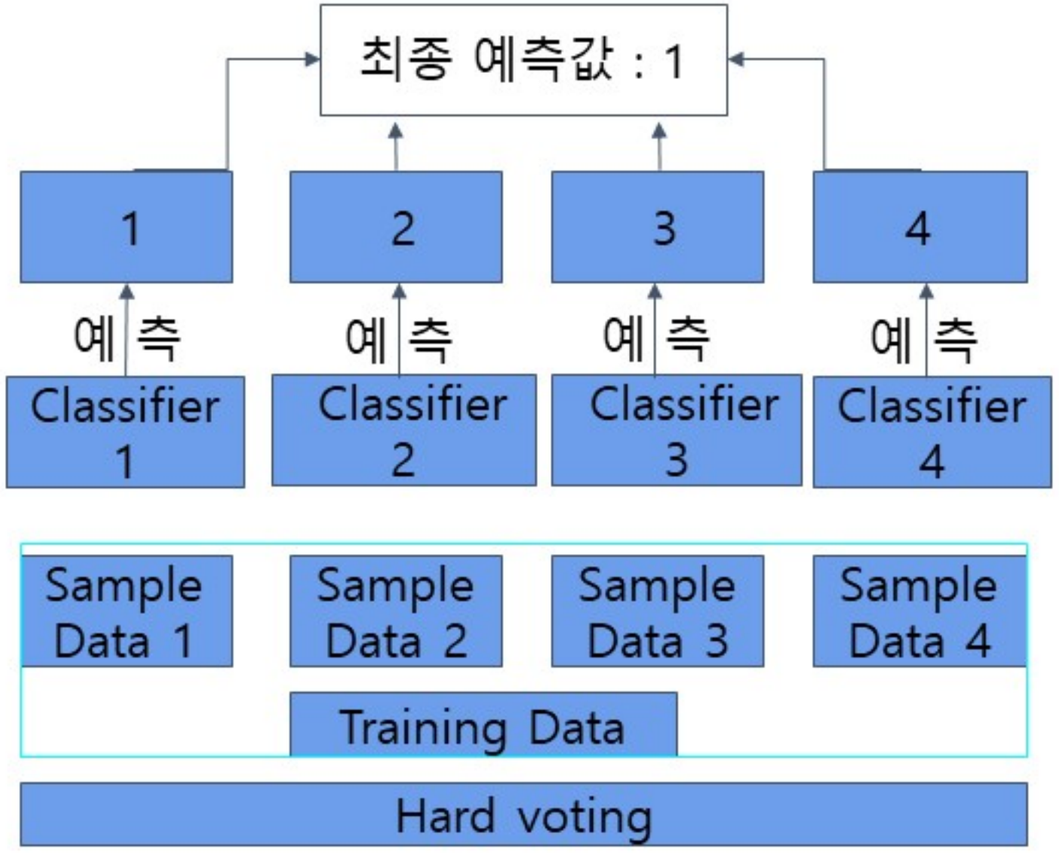
- 투표를 통해 결정하는 방식. Voting은 Bagging과 같은 투표방식이라는 점에서 유사하지만, 다음과 같은 큰 차이점이 존재

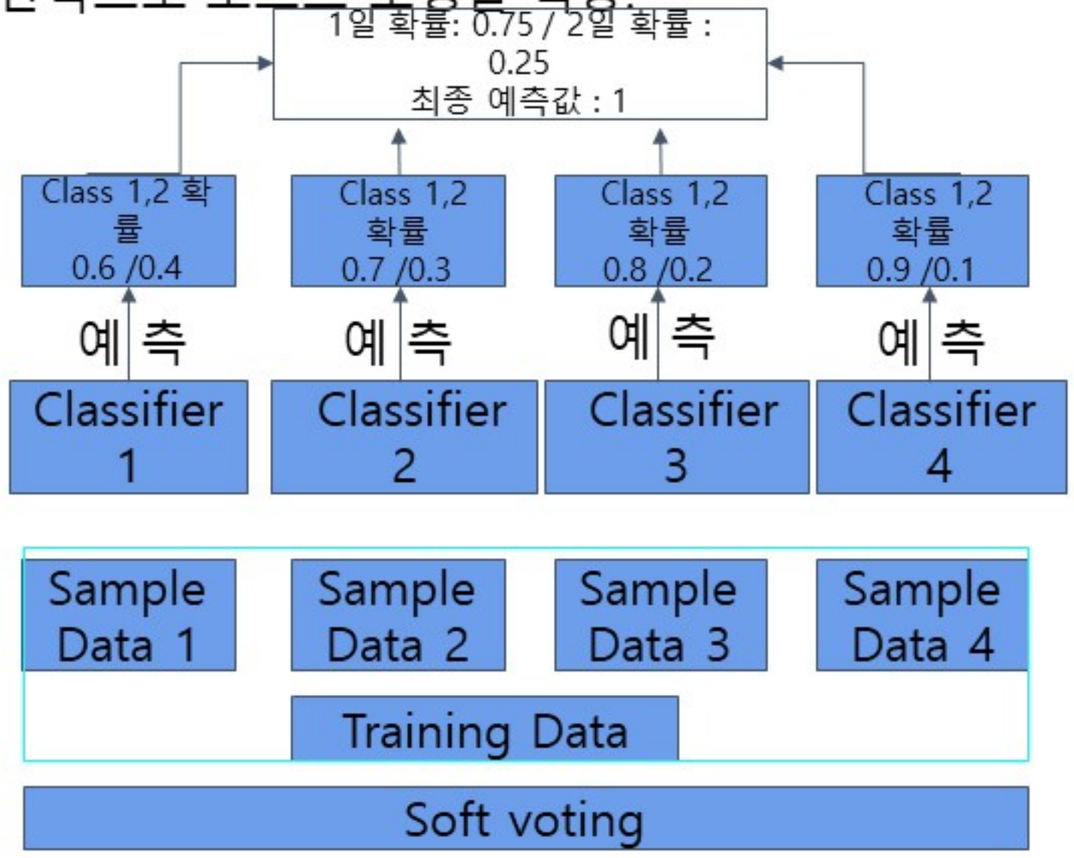
- Voting은 다른 알고리즘 model을 조합해서 사용
- Bagging은 같은 알고리즘 내에서 다른 Sample 조합을 사용

- Voting은 서로 다른 알고리즘이 도출해 낸 결과물에 대하여 최종 투표하는 방식
- hard vote와 soft vote로 나뉨.
- hard vote는 결과물에 대한 최종 값을 투표해서 결정하고, soft vote는 최종 결과물이 나올 확률 값이 다 더해서 최종 결과물에 대한 각각의 확률을 구한 뒤 최종값을 도출.

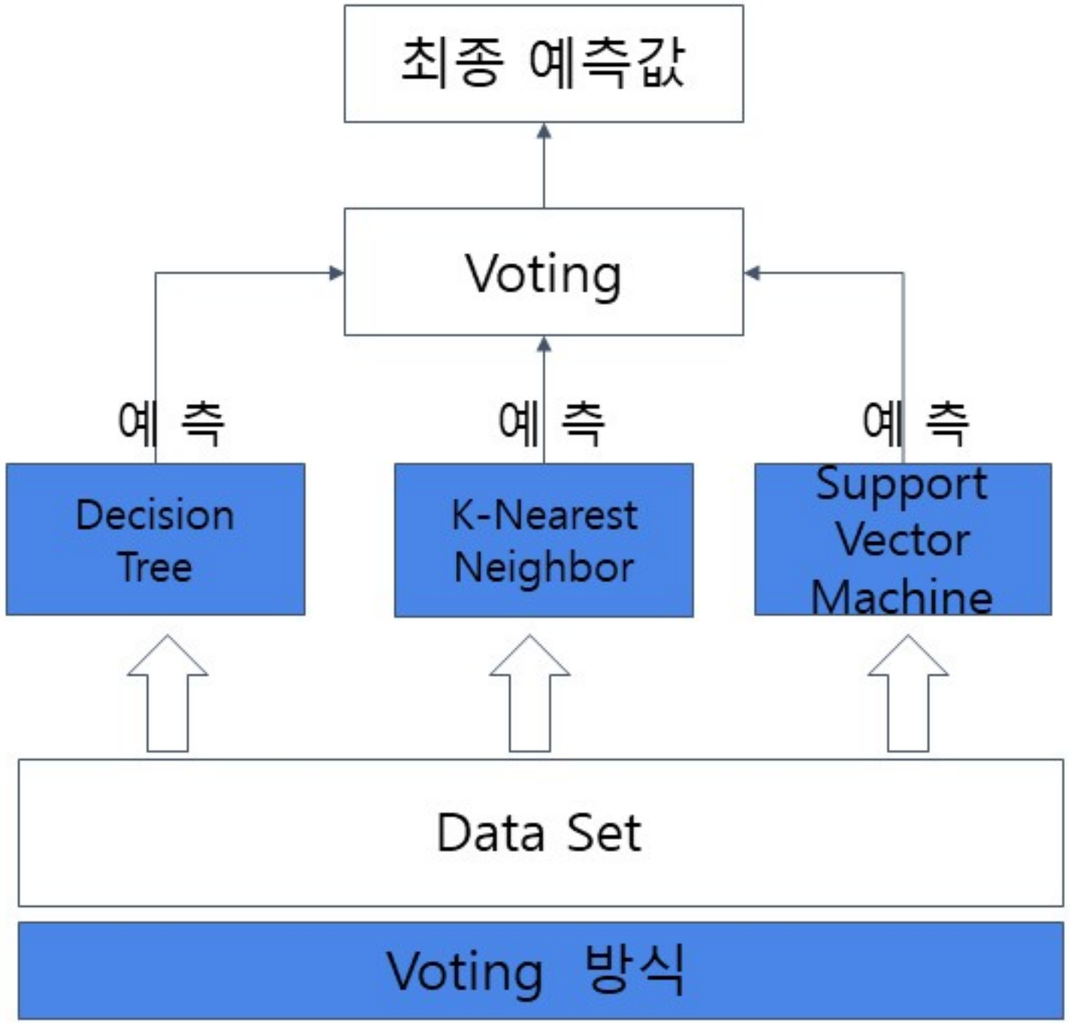
7. Ensemble 의 기법_Voting

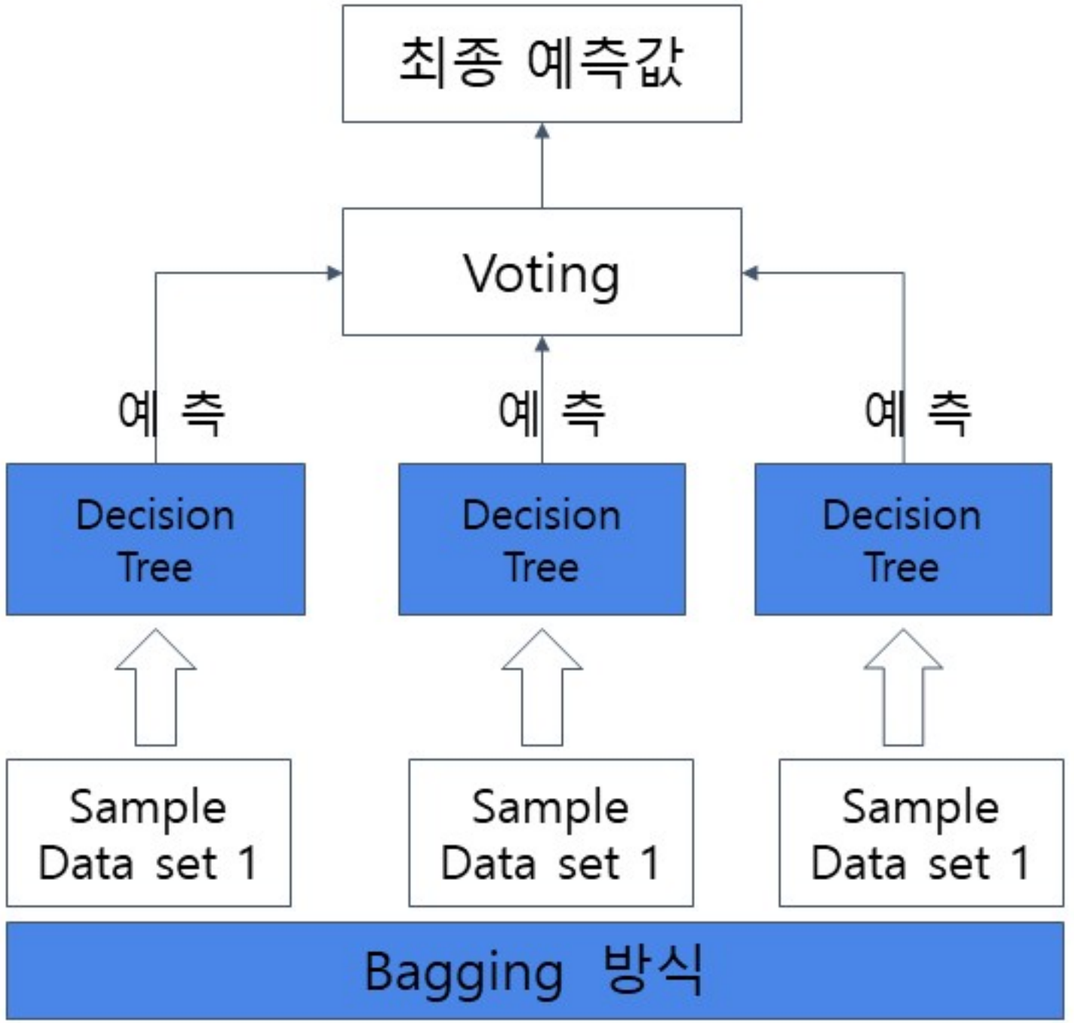
- Hard voting : 다수결 원칙과 비슷
- Soft voting : 분류기들의 레이블 값 결정 확률을 모두 더하고 이를 평균해서 이들 중 확률이 가장 높은 레이블 값을 최종 보팅 결괏값 으로 선정. 일반적으로 소프트 보팅을 적용.





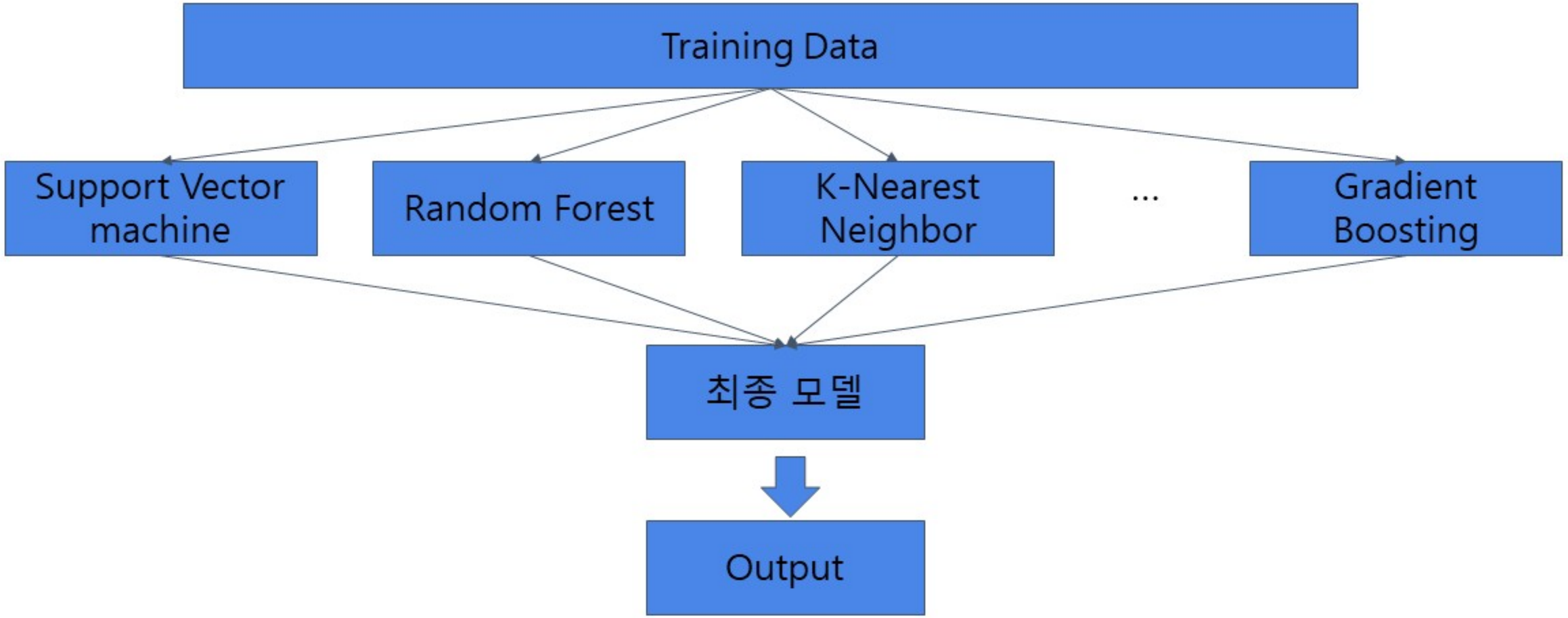
7. Ensemble 의 기법_ Bagging VS Voting





7. Ensemble 의 기법_Stacking

- 여러 모델들을 활용해 각각의 예측 결과를 도출한 뒤 그 예측 결과를 결합해 최종 예측 결과를 만들어 내는 것.



6. TabNet

Tabular Data를 위한 딥러닝 모델 TabNet

TabNet: Attentive Interpretable Tabular Learning

Sercan Ö. Arık
Google Cloud AI
soarik@google.com

Tomas Pfister
Google Cloud AI
tpfister@google.com

Key point

1. TabNet은 전처리 과정이 필요하지 않음.
2. 정형 데이터에 대해서는 기존의 Decision tree-based gradient boosting(xgboost, lgbm, catboost)와 같은 모델에 비해 신경망 모델은 아직 성능이 안정적이지 못함. 두 구조의 장점을 모두 갖는 신경망 모델
3. Feature selection, interpretability(local, global)가 가능한 신경망 모델, 설명가능한 모델
4. feature 값을 예측하는 Unsupervised pretrain 단계를 적용하여 상당한 성능 향상을 보여줌.

8. TabNet

- TabNet은 순차적인 어텐션(Sequential Attention)을 사용하여 각 의사 결정 단계에서 추론할 특징을 선택하여 학습 능력이 가장 두드러진 특징을 사용
  - 기존의 머신러닝 방법에서는 이러한 특징 선택과 모델 학습의 과정이 나누어져 있지만, TabNet에서는 한번에 가능
  - 특징 선택이 이루어지므로 어떠한 특징이 중요한지 설명이 가능함.

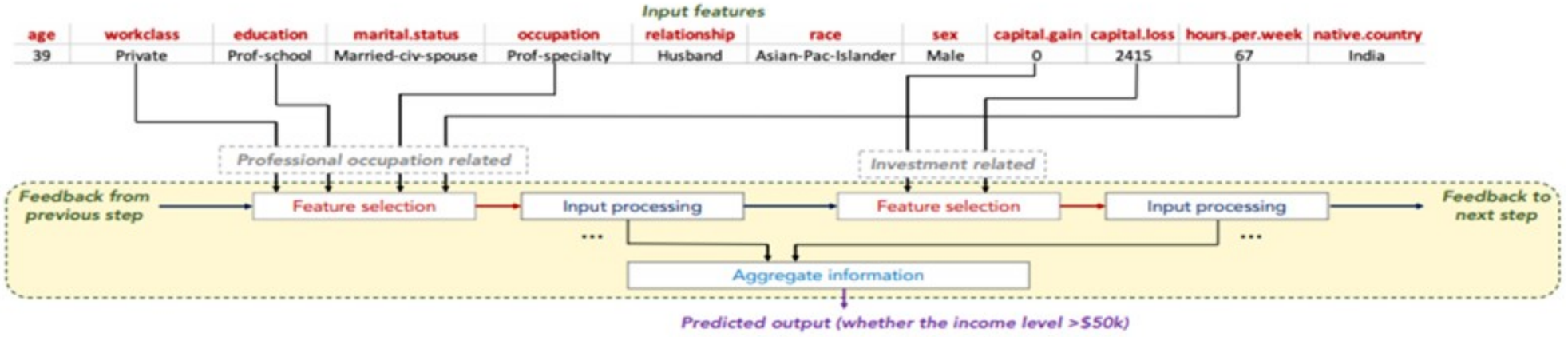


Figure 1: Depiction of TabNet's sparse feature selection for Adult Census Income prediction (Dua & Graff, 2017). TabNet employs multiple decision blocks that focus on processing a subset of input features for overall decision making. Two decision blocks shown as examples process the features that are professional occupation related and investment related in order to predict the income level.

출처 : https://arxiv.org/pdf/1908.07442.pdf

8. TabNet

Unsupervised pre-training

Age	Cap. gain	Education	Occupation	Gender	Relationship
53	200000	?	Exec-managerial	F	Wife
19	0	?	Farming-fishing	M	?
?	5000	Doctorate	Prof-specialty	M	Husband
25	?	?	Handlers-cleaners	F	Wife
59	300000	Bachelors	?	?	Husband
33	0	Bachelors	?	F	?
?	0	High-school	Armed-Forces	?	Husband

TabNet encoder

TabNet decoder

Age	Cap. gain	Education	Occupation	Gender	Relationship
		Masters			
		High-school			Unmarried
43					
	0	High-school		F	
			Exec-managerial	M	
			Adm-clerical		Wife
39				M	

Supervised fine-tuning

Age	Cap. gain	Education	Occupation	Gender	Relationship
60	200000	Bachelors	Exec-managerial	M	Husband
23	0	High-school	Farming-fishing	M	Unmarried
45	5000	Doctorate	Prof-specialty	M	Husband
23	0	High-school	Handlers-cleaners	F	Wife
56	300000	Bachelors	Exec-managerial	M	Husband
38	10000	Bachelors	Prof-specialty	F	Wife
23	0	High-school	Armed-Forces	M	Husband

TabNet encoder

Decision making

Income > $50k
True
False
True
False
True
True
False

출처 : https://arxiv.org/pdf/1908.07442.pdf

Thank you.

분류 문제 / 류영표 강사
ryp1662@gmail.com

© 2025 Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.